Data Science, Machine Learning & Deep Learning

A placement booklet for online assessments and interviews

Claude Opus 5.5

October 2026

Contents

How to use this booklet **ix**

I Foundations **1**

1 Linear Algebra, Calculus and Optimisation **2**

1.1 Vectors, matrices and the geometry behind them 2

1.1.1 Dot product, norms, angles 2

1.1.2 Matrix multiplication and shapes 2

1.1.3 Rank, span, independence 3

1.1.4 Determinant and inverse 3

1.2 Eigen-decomposition and SVD 3

1.2.1 Eigenvalues and eigenvectors 3

1.2.2 Singular Value Decomposition (SVD) 3

1.3 Calculus for machine learning 4

1.3.1 Derivatives, partial derivatives, gradient 4

1.3.2 Chain rule — the heart of backpropagation 4

1.3.3 Matrix calculus identities (memorise) 4

1.3.4 Derivatives of common functions 4

1.3.5 Taylor expansion 5

1.4 Optimisation 5

1.4.1 Convexity 5

1.4.2 Gradient descent family 5

1.4.3 Newton's method 5

1.4.4 Constrained optimisation: Lagrange multipliers and KKT 5

1.5 Interview questions 6

2 Probability, Statistics and Information Theory **7**

2.1 Probability essentials 7

2.1.1 Rules 7

2.1.2 Random variables, expectation, variance 7

2.1.3 Distributions you must know 8

2.1.4 Limit theorems 8

2.2 Estimation . . . 8
2.2.1 Maximum likelihood (MLE) . . . 8
2.2.2 MAP and priors . . . 9
2.2.3 Properties of estimators . . . 9
2.3 Hypothesis testing . . . 9
2.3.1 Framework . . . 9
2.3.2 Which test? . . . 9
2.4 A/B testing (core for DS roles) . . . 9
2.5 Information theory . . . 10
2.6 Classic probability puzzles . . . 10

II Classical Machine Learning 12

3 Machine Learning Fundamentals 13
3.1 The learning problem . . . 13
3.2 Bias–variance trade-off . . . 13
3.2.1 Derivation (squared loss) . . . 14
3.3 Data splitting and validation . . . 14
3.3.1 Data leakage . . . 14
3.4 Preprocessing and feature engineering . . . 15
3.4.1 Missing values . . . 15
3.4.2 Categorical encoding . . . 15
3.4.3 Scaling . . . 15
3.4.4 Feature engineering ideas interviewers like . . . 15
3.4.5 Feature selection . . . 16
3.4.6 Curse of dimensionality . . . 16
3.5 Class imbalance . . . 16
3.6 Evaluation metrics . . . 16
3.6.1 Confusion matrix and derived metrics . . . 16
3.6.2 ROC and PR curves . . . 17
3.6.3 Probabilistic metrics and calibration . . . 17
3.6.4 Regression metrics . . . 17
3.6.5 Ranking metrics . . . 17
3.7 Interview questions . . . 18

4 Linear Models 19
4.1 Linear regression . . . 19
4.1.1 Model and OLS solution . . . 19
4.1.2 Assumptions (classical linear model) . . . 19
4.1.3 Inference . . . 20
4.1.4 Interpreting coefficients under transforms . . . 20

4.2 Regularised regression . . . 20
4.2.1 Ridge (L_2) . . . 20
4.2.2 Lasso (L_1) . . . 20
4.2.3 Elastic net . . . 21
4.2.4 Bayesian interpretation (derivation) . . . 21
4.3 Logistic regression . . . 21
4.3.1 Model . . . 21
4.3.2 Loss and gradient (derive this on a whiteboard) . . . 21
4.3.3 Multiclass: softmax regression . . . 22
4.4 Generalised linear models (GLMs) . . . 22
4.5 Model selection criteria . . . 22
4.6 Interview questions . . . 22

5 Classical Classifiers 24
5.1 k-Nearest Neighbours . . . 24
5.2 Naive Bayes . . . 24
5.3 Linear and Quadratic Discriminant Analysis . . . 25
5.4 Support Vector Machines . . . 25
5.4.1 Hard-margin SVM (derivation) . . . 25
5.4.2 Soft margin and hinge loss . . . 25
5.4.3 The dual and the kernel trick . . . 26
5.5 Decision trees . . . 26
5.5.1 Splitting criteria . . . 26
5.5.2 Algorithm and complexity . . . 26
5.5.3 Controlling overfitting . . . 27
5.5.4 Pros and cons . . . 27
5.6 Interview questions . . . 27

6 Ensembles 28
6.1 Why averaging helps . . . 28
6.2 Bagging . . . 28
6.3 Random forests . . . 28
6.4 Boosting . . . 29
6.4.1 AdaBoost . . . 29
6.4.2 Gradient boosting (derivation) . . . 29
6.4.3 XGBoost: second-order boosting (derivation) . . . 30
6.4.4 LightGBM . . . 30
6.4.5 CatBoost . . . 30
6.4.6 Tuning gradient boosting — a practical order . . . 31
6.5 Stacking and blending . . . 31
6.6 Model interpretability . . . 31

6.7 Interview questions . . . 31

7 Unsupervised Learning **33**
7.1 k-means clustering . . . 33
7.2 Hierarchical clustering . . . 34
7.3 DBSCAN . . . 34
7.4 Gaussian mixture models and EM . . . 34
7.5 Principal Component Analysis . . . 35
7.5.1 Derivation . . . 35
7.5.2 Practical points . . . 35
7.6 Non-linear embeddings for visualisation . . . 35
7.7 Anomaly detection . . . 36
7.8 Association rules (market basket analysis) . . . 36
7.9 Interview questions . . . 36

8 Time Series, Recommenders, NLP **38**
8.1 Time series . . . 38
8.1.1 Components and stationarity . . . 38
8.1.2 ACF and PACF . . . 38
8.1.3 Classical models . . . 38
8.1.4 Machine learning for forecasting . . . 39
8.1.5 Validation and metrics . . . 39
8.2 Recommender systems . . . 39
8.2.1 Approaches . . . 39
8.2.2 Problems and metrics . . . 40
8.3 Classical NLP . . . 40
8.4 Interview questions . . . 40

III Deep Learning **42**

9 Neural Networks and How to Train Them **43**
9.1 From the perceptron to multilayer networks . . . 43
9.2 Backpropagation . . . 43
9.2.1 Derivation in matrix form . . . 44
9.2.2 Why softmax with cross-entropy gives prediction minus target . . . 44
9.2.3 A tiny worked example . . . 44
9.2.4 Automatic differentiation . . . 44
9.3 Activation functions . . . 44
9.4 Vanishing and exploding gradients . . . 45
9.5 Weight initialisation . . . 45
9.6 Optimisers . . . 45

9.6.1 Learning-rate schedules . . . 46
9.6.2 Batch size . . . 46
9.7 Normalisation . . . 46
9.8 Regularisation in deep learning . . . 47
9.9 Training at scale (ML engineer topics) . . . 47
9.10 Debugging a network that won't learn . . . 47
9.11 Interview questions . . . 48

10 Convolutional Neural Networks and Computer Vision 49
10.1 Why convolutions? . . . 49
10.2 The convolution layer . . . 49
10.2.1 Output size and parameter count (asked in almost every OA) . . . 49
10.2.2 Other ingredients . . . 50
10.2.3 Receptive field . . . 50
10.3 Landmark architectures . . . 50
10.4 Transfer learning . . . 51
10.5 Beyond classification . . . 51
10.5.1 Object detection . . . 51
10.5.2 Segmentation . . . 51
10.6 Interview questions . . . 51

11 Recurrent Networks and Sequence Models 53
11.1 Recurrent neural networks . . . 53
11.1.1 Backpropagation through time and vanishing gradients . . . 53
11.2 LSTM . . . 53
11.3 GRU . . . 54
11.4 Variants and practicalities . . . 54
11.5 Sequence-to-sequence and attention . . . 54
11.6 Interview questions . . . 55

12 Transformers and Large Language Models 56
12.1 Self-attention . . . 56
12.1.1 Scaled dot-product attention . . . 56
12.1.2 Multi-head attention . . . 56
12.1.3 Complexity . . . 57
12.2 Positional information . . . 57
12.3 The Transformer block . . . 57
12.4 Three families of Transformers . . . 58
12.5 Tokenisation . . . 58
12.6 Pre-training at scale . . . 58
12.7 Adapting LLMs . . . 59
12.7.1 Prompting and in-context learning . . . 59

12.7.2 Fine-tuning . . . 59
12.7.3 Alignment: RLHF and DPO . . . 59
12.8 Inference and decoding . . . 60
12.8.1 Decoding strategies . . . 60
12.8.2 KV cache . . . 60
12.8.3 Making models cheaper . . . 60
12.9 Retrieval-augmented generation (RAG) . . . 60
12.10Evaluating language models . . . 61
12.11Interview questions . . . 61

13 Generative Models and Representation Learning 63
13.1 Autoencoders . . . 63
13.2 Variational autoencoders (VAE) . . . 63
13.2.1 ELBO derivation . . . 63
13.3 Generative adversarial networks (GANs) . . . 64
13.4 Diffusion models . . . 64
13.5 Comparing generative models . . . 65
13.6 Self-supervised and contrastive learning . . . 65
13.7 Interview questions . . . 65

IV Engineering, Coding and Interview Practice 67

14 MLOps and ML System Design 68
14.1 A framework for any ML design question . . . 68
14.2 Core MLOps concepts . . . 68
14.3 Worked design 1: real-time card fraud detection . . . 69
14.4 Worked design 2: product recommendations on an e-commerce home page . . . 70
14.5 Worked design 3: customer churn for a telecom / subscription business . . . 70
14.6 Interview questions . . . 70

15 Python, NumPy and pandas for Assessments 72
15.1 Python fundamentals that get asked . . . 72
15.1.1 Data structures and complexity . . . 72
15.1.2 Gotchas . . . 72
15.1.3 Idioms worth having at your fingertips . . . 73
15.2 NumPy . . . 74
15.2.1 Vectorisation and broadcasting . . . 74
15.2.2 Numerically stable softmax and log-sum-exp . . . 74
15.3 pandas . . . 74
15.3.1 Core operations . . . 74
15.3.2 Time-aware and window-style operations . . . 75

15.3.3 pandas pitfalls . . . 76
15.4 scikit-learn: leakage-safe pipelines . . . 76
15.5 Implement-from-scratch questions . . . 76
15.5.1 Linear regression with gradient descent (and closed form) . . . 76
15.5.2 Logistic regression . . . 77
15.5.3 k-means . . . 77
15.5.4 k-nearest neighbours . . . 78
15.5.5 Metrics by hand: confusion matrix and ROC-AUC . . . 78
15.5.6 A two-layer neural network with manual backprop . . . 78
15.5.7 PCA via SVD . . . 79
15.6 Typical DSA patterns in DS coding rounds . . . 79
15.7 Interview questions . . . 80

16 SQL for Data Science Interviews **81**
16.1 Sample schema used in this chapter . . . 81
16.2 Logical order of execution . . . 81
16.3 Joins . . . 81
16.4 NULL semantics . . . 82
16.5 Window functions . . . 82
16.6 Classic interview queries . . . 82
16.6.1 Second-highest (and N-th highest) salary . . . 82
16.6.2 Top-N per group . . . 83
16.6.3 Employees earning more than their manager (self join) . . . 83
16.6.4 Customers who never ordered (anti join) . . . 83
16.6.5 Duplicates: find and keep one . . . 83
16.6.6 Department aggregates with HAVING and share of total . . . 83
16.6.7 Running total, moving average, previous value . . . 84
16.6.8 Month-over-month revenue growth . . . 84
16.6.9 Pivot with conditional aggregation . . . 84
16.6.10 Consecutive days (gaps and islands) . . . 85
16.6.11 Retention: share of users who returned the next day . . . 85
16.6.12 Monthly cohort analysis . . . 85
16.6.13 Median . . . 85
16.6.14 Customers whose every order exceeded 200 (relational division style) . . . 86
16.7 Other things you may be asked . . . 86
16.8 Interview questions . . . 86

17 Rapid-Fire Question Bank **88**
17.1 Statistics and probability . . . 88
17.2 Classical machine learning . . . 89
17.3 Deep learning . . . 90

17.4 Tricky MCQ-style items 92
17.5 Case and open-ended questions 92
17.6 Talking about your own projects 93

A Formula Cheat Sheet **94**

How to use this booklet

What this booklet is, and what it is not

This booklet targets two overlapping role families: **Data Scientist** (statistics, classical ML, experimentation, business framing) and **ML Engineer / DL-heavy** roles (neural networks, Transformers, training at scale, deployment). Every major topic is written in the same order:

1. **Intuition** — what the method is trying to do, in plain words, usually with a real-world example.
2. **Formal treatment** — definitions, the objective function, and derivations of the results interviewers actually ask you to reproduce on a whiteboard.
3. **Interview angle** — the follow-up questions, trade-offs and pitfalls that separate a prepared candidate from one who memorised definitions.

An honest limitation: no single document covers every question every company asks. Firms with a strong research bent (quant funds, AI labs) go deeper into specific areas than this booklet does; product and analytics firms lean more on SQL, experimentation and business cases. Treat this as the core you must own, then add depth in whatever the job description emphasises.

How online assessments are usually structured

Campus OAs for DS/ML roles in India commonly mix some of the following (the mix varies a lot by company):

- **MCQs on ML/DL/statistics** — definitions, which metric to use, what happens if a hyperparameter changes, output-size of a convolution, number of parameters in a layer, probability puzzles.
- **Aptitude / quant** — probability and combinatorics are where DS candidates are expected to be stronger than average.
- **Coding** — usually one or two DSA-style problems (arrays, strings, hashing) and sometimes a pandas / NumPy / SQL task.
- **SQL** — joins, GROUP BY, window functions; frequently auto-graded on HackerRank-style platforms.
- **Case / subjective** — design a model for a business problem, choose a metric, explain trade-offs.

Suggested study plan

Phase	What to do
Week 1	Chapters 1–4. Be able to derive OLS, logistic regression gradient, bias–variance decomposition, and compute every classification metric from a confusion matrix by hand.
Week 2	Chapters 5–8. Trees, boosting (derive the gradient-boosting update), SVM dual, PCA via SVD, k-means, EM.
Week 3	Chapters 9–13. Backprop by hand for a 2-layer net, Adam, BatchNorm, conv output sizes and parameter counts, LSTM gates, self-attention end to end.
Week 4	Chapters 14–17. SQL daily (20–30 queries), pandas drills, one ML system-design case per day, then the question bank under time pressure.

Conventions

Vectors are bold lower-case ($\mathbf{x}$), matrices bold upper-case ($\mathbf{X}$). The design matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ has n rows (samples) and d columns (features). log is the natural log unless stated. "Interview angle" boxes contain the questions you should be able to answer aloud in under a minute.

Part I

Foundations

Chapter 1

Linear Algebra, Calculus and Optimisation

Almost every ML model is "a matrix multiplication, a nonlinearity and an optimiser". Interviewers use linear algebra and calculus questions to check whether you understand what the libraries are doing.

1.1 Vectors, matrices and the geometry behind them

Intuition. A vector is a point (or an arrow) in space; a matrix is a *function* that moves points around — it stretches, rotates, projects or shears. Reading $\mathbf{Ax}$ as "apply transformation $\mathbf{A}$ to $\mathbf{x}$" makes PCA, SVD and neural network layers much easier to reason about.

1.1.1 Dot product, norms, angles

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$: $\mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i = \|\mathbf{x}\|\|\mathbf{y}\| \cos\theta$.

- L_2 norm $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$; L_1 norm $\|\mathbf{x}\|_1 = \sum |x_i|$; $L_\infty = \max |x_i|$; L_0 "norm" = number of non-zeros (not a true norm).
- **Cosine similarity** $= \mathbf{x}^\top \mathbf{y} / (\|\mathbf{x}\|\|\mathbf{y}\|) \in [-1, 1]$. Used for text embeddings because it ignores magnitude (document length).
- **Projection** of $\mathbf{x}$ onto unit vector $\mathbf{u}$: $(\mathbf{x}^\top \mathbf{u})\mathbf{u}$. PCA is "find the $\mathbf{u}$ that keeps the most variance after projection".

Real-world example. In a recommendation system, each user and each movie is a vector of, say, 64 numbers. The predicted rating is the dot product. Two movies with high cosine similarity are "similar" even if one is a blockbuster (large norm) and one is niche (small norm).

1.1.2 Matrix multiplication and shapes

$(\mathbf{AB})_{ij} = \sum_k A_{ik} B_{kj}$, defined when $\mathbf{A} \in \mathbb{R}^{m \times k}, \mathbf{B} \in \mathbb{R}^{k \times n}$, result $m \times n$. Cost: $O(mkn)$ multiplications. Not commutative; associative, so the order of evaluation matters for cost: for $\mathbf{A} \in \mathbb{R}^{1000 \times 2}, \mathbf{B} \in \mathbb{R}^{2 \times 1000}, \mathbf{x} \in \mathbb{R}^{1000}$, computing $\mathbf{A}(\mathbf{Bx})$ costs about 4000 operations, $(\mathbf{AB})\mathbf{x}$ about 3 million.

Useful identities: $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$, $\mathrm{tr}(\mathbf{ABC}) = \mathrm{tr}(\mathbf{CAB})$ (cyclic), $\mathbf{x}^\top \mathbf{Ax} = \mathrm{tr}(\mathbf{Axx}^\top)$.

1.1.3 Rank, span, independence

- Vectors are **linearly independent** if no one is a linear combination of the others.
- **Rank** = number of independent columns = number of independent rows = number of non-zero singular values.
- $\mathbf{X}^\top\mathbf{X}$ is invertible iff $\mathbf{X}$ has full column rank. *This is why perfect multicollinearity breaks OLS*: two identical columns make $\mathbf{X}^\top\mathbf{X}$ singular.
- Rank–nullity: $\operatorname{rank}(\mathbf{A}) + \dim\operatorname{null}(\mathbf{A}) =$ number of columns.

1.1.4 Determinant and inverse

$\det(\mathbf{A})$ is the factor by which $\mathbf{A}$ scales volume. $\det = 0$ means the transformation squashes space into a lower dimension, so it cannot be undone (no inverse). $\det(\mathbf{AB}) = \det\mathbf{A}\det\mathbf{B}$; $\det(\mathbf{A}^{-1}) = 1/\det\mathbf{A}$; determinant equals the product of eigenvalues; trace equals the sum of eigenvalues.

In practice we almost never compute an inverse explicitly: we solve $\mathbf{Ax} = \mathbf{b}$ with a factorisation (LU, Cholesky, QR). Explicit inversion is slower and numerically less stable.

1.2 Eigen-decomposition and SVD

1.2.1 Eigenvalues and eigenvectors

$\mathbf{Av} = \lambda\mathbf{v}$: $\mathbf{v}$ is a direction that $\mathbf{A}$ only stretches (by λ), never rotates. Found by solving $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$.

Example 1.1. $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. $\det\begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 1 = 0 \Rightarrow \lambda \in \{1, 3\}$. For $\lambda = 3$: $\mathbf{v} = (1,1)^\top/\sqrt{2}$; for $\lambda = 1$: $\mathbf{v} = (1,-1)^\top/\sqrt{2}$. Check: trace $= 4 = 1 + 3$, det $= 3 = 1 \times 3$.

Theorem 1.2 (Spectral theorem)**.** A real symmetric matrix $\mathbf{S}$ has real eigenvalues and orthonormal eigenvectors: $\mathbf{S} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$.

Covariance matrices are symmetric and positive semi-definite (PSD), so they always have this decomposition with $\lambda_i \geq 0$. That is the foundation of PCA.

Positive definiteness. $\mathbf{S}$ is PSD if $\mathbf{x}^\top\mathbf{Sx} \geq 0$ for all $\mathbf{x}$ (equivalently all eigenvalues ≥ 0); positive definite (PD) if > 0 for $\mathbf{x} \neq 0$. A twice-differentiable function is convex iff its Hessian is PSD everywhere.

Proof that a covariance matrix is PSD: $\mathbf{x}^\top\mathbf{\Sigma}\mathbf{x} = \mathbf{x}^\top\mathbb{E}[(\mathbf{z}-\boldsymbol{\mu})(\mathbf{z}-\boldsymbol{\mu})^\top]\mathbf{x} = \mathbb{E}[(\mathbf{x}^\top(\mathbf{z}-\boldsymbol{\mu}))^2] \geq 0$.

1.2.2 Singular Value Decomposition (SVD)

Any $\mathbf{A} \in \mathbb{R}^{m\times n}$ factorises as $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ where $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ are orthogonal and $\mathbf{\Sigma}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

Intuition. Every linear map is: rotate ($\mathbf{V}^\top$), stretch along axes ($\mathbf{\Sigma}$), rotate again ($\mathbf{U}$).

Connections you will be asked about:

- $\mathbf{A}^\top\mathbf{A} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top$: the right singular vectors are eigenvectors of $\mathbf{A}^\top\mathbf{A}$, and σ_i^2 are its eigenvalues.

- **Eckart–Young theorem**: the best rank-k approximation (in Frobenius or spectral norm) is $\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$. This underlies PCA, latent semantic analysis, image compression and low-rank adapters (LoRA).
- Condition number $\kappa = \sigma_{\max}/\sigma_{\min}$. Large κ means the problem is ill-conditioned: small input changes cause large output changes, and gradient descent zig-zags.
- Pseudo-inverse $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ gives the minimum-norm least-squares solution even when $\mathbf{X}^\top\mathbf{X}$ is singular.

Real-world example. Netflix-prize style collaborative filtering approximates the user × movie rating matrix by a low-rank product: tastes are driven by a few latent factors ("action vs. drama", "mainstream vs. indie"), which is exactly what a truncated SVD captures.

1.3 Calculus for machine learning

1.3.1 Derivatives, partial derivatives, gradient

The gradient $\nabla f(\mathbf{x}) = (\partial f/\partial x_1, \ldots, \partial f/\partial x_d)^\top$ points in the direction of steepest increase; its magnitude is the rate of increase. Gradient descent steps in $-\nabla f$.

Why $-\nabla f$ is steepest descent: the directional derivative along unit $\mathbf{u}$ is $\nabla f^\top \mathbf{u} = \|\nabla f\| \cos\theta$, minimised at $\theta = \pi$.

1.3.2 Chain rule — the heart of backpropagation

If $y = f(g(x))$ then $dy/dx = f'(g(x))\, g'(x)$. For vectors, $\partial \mathbf{z}/\partial \mathbf{x} = (\partial \mathbf{z}/\partial \mathbf{y})(\partial \mathbf{y}/\partial \mathbf{x})$ as a product of Jacobians.

1.3.3 Matrix calculus identities (memorise)

With $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{A}$ constant:

$$\nabla_{\mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a} \qquad \nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x} \ (= 2\mathbf{A}\mathbf{x} \text{ if symmetric})$$

$$\nabla_{\mathbf{x}}\|\mathbf{x}\|_2^2 = 2\mathbf{x} \qquad \nabla_{\mathbf{x}}\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 = 2\mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$$

$$\nabla_{\mathbf{A}} \operatorname{tr}(\mathbf{A}\mathbf{B}) = \mathbf{B}^\top \qquad \nabla_{\mathbf{A}} \log\det \mathbf{A} = \mathbf{A}^{-\top}$$

1.3.4 Derivatives of common functions

Function	Formula	Derivative
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$	$\sigma(z)(1-\sigma(z))$, max 0.25 at $z=0$
Tanh	$\tanh z$	$1 - \tanh^2 z$, max 1
ReLU	$\max(0, z)$	$\mathbb{1}[z > 0]$
Softplus	$\log(1+e^z)$	$\sigma(z)$
Softmax	$p_i = e^{z_i}/\sum_j e^{z_j}$	$\partial p_i/\partial z_j = p_i(\delta_{ij} - p_j)$
Log-sum-exp	$\log \sum_j e^{z_j}$	$\operatorname{softmax}(\mathbf{z})$

Derivation of the sigmoid derivative: $\sigma = (1+e^{-z})^{-1} \Rightarrow \sigma' = e^{-z}(1+e^{-z})^{-2} = \sigma \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(1-\sigma)$.

1.3.5 Taylor expansion

$f(\mathbf{x}+\boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top \mathbf{H}\boldsymbol{\delta}$. The first-order term justifies gradient descent; the second-order term justifies Newton's method and XGBoost's split gain formula (Chapter 6).

1.4 Optimisation

1.4.1 Convexity

f is convex if $f(\lambda\mathbf{x}+(1-\lambda)\mathbf{y}) \le \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y})$. For convex f, every local minimum is global.

- Convex: MSE for linear regression, log-loss for logistic regression, hinge loss for SVM, L_1 and L_2 penalties.
- Non-convex: neural network losses, k-means objective, GMM likelihood, matrix factorisation.
- Sums of convex functions are convex; a convex function composed with an affine map is convex.

1.4.2 Gradient descent family

Batch GD: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla L(\boldsymbol{\theta})$ using all n samples. Deterministic, expensive per step.

Stochastic GD: use one sample; noisy but cheap and the noise can help escape saddle points.

Mini-batch GD: batch of size B (32–4096). Standard in deep learning; vectorises on GPUs.

Convergence (convex, L-smooth): with $\eta \le 1/L$, GD reaches $f(\boldsymbol{\theta}_t) - f^* = O(1/t)$; with strong convexity, linear (geometric) convergence. Too large a learning rate diverges; too small crawls.

Momentum, RMSProp and Adam are covered in Chapter 9 where they matter most.

1.4.3 Newton's method

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \mathbf{H}^{-1}\nabla L$. Uses curvature, so converges quadratically near the optimum, but each step needs an $O(d^3)$ Hessian solve, impractical for millions of parameters. Logistic regression's classic solver IRLS is Newton's method. Quasi-Newton (BFGS, L-BFGS) approximates $\mathbf{H}^{-1}$ from gradients; L-BFGS is scikit-learn's default for logistic regression.

1.4.4 Constrained optimisation: Lagrange multipliers and KKT

To minimise $f(\mathbf{x})$ subject to $g_i(\mathbf{x}) \le 0$, $h_j(\mathbf{x}) = 0$, form the Lagrangian

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\nu}) = f(\mathbf{x}) + \sum_i \alpha_i g_i(\mathbf{x}) + \sum_j \nu_j h_j(\mathbf{x}), \quad \alpha_i \ge 0.$$

KKT conditions at the optimum: stationarity $\nabla_{\mathbf{x}}\mathcal{L} = 0$; primal feasibility; dual feasibility $\alpha_i \ge 0$; complementary slackness $\alpha_i g_i(\mathbf{x}) = 0$. Complementary slackness is why only *support vectors* have non-zero α_i in an SVM.

Example 1.3 (PCA as a Lagrangian problem)**.** Maximise $\mathbf{u}^\top\boldsymbol{\Sigma}\mathbf{u}$ s.t. $\mathbf{u}^\top\mathbf{u} = 1$. $\mathcal{L} = \mathbf{u}^\top\boldsymbol{\Sigma}\mathbf{u} - \lambda(\mathbf{u}^\top\mathbf{u} - 1)$; $\nabla_{\mathbf{u}} = 2\boldsymbol{\Sigma}\mathbf{u} - 2\lambda\mathbf{u} = 0 \Rightarrow \boldsymbol{\Sigma}\mathbf{u} = \lambda\mathbf{u}$. The variance captured is $\mathbf{u}^\top\boldsymbol{\Sigma}\mathbf{u} = \lambda$, so pick the eigenvector with the largest eigenvalue.

1.5 Interview questions

Q1.1. Why is computing $(\mathbf{X}^\top\mathbf{X})^{-1}$ a bad idea for large or collinear data?

Answer. It costs $O(d^3)$, and when columns are nearly collinear the matrix is ill-conditioned, so rounding errors explode. Use QR/SVD-based solvers, add ridge regularisation ($\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ is always invertible for $\lambda > 0$), or use gradient methods.

Q1.2. What does it mean geometrically if a matrix has a zero eigenvalue?

Answer. Some direction is mapped to zero: the matrix collapses space, is singular and not invertible; determinant is zero.

Q1.3. Relationship between eigen-decomposition and SVD?

Answer. For symmetric PSD matrices they coincide. In general, singular values of $\mathbf{A}$ are square roots of eigenvalues of $\mathbf{A}^\top\mathbf{A}$; SVD exists for any rectangular matrix, eigen-decomposition only for (diagonalisable) square ones.

Q1.4. Is the Hessian of the logistic regression loss PSD? Why does it matter?

Answer. Yes: $\mathbf{H} = \mathbf{X}^\top\mathbf{D}\mathbf{X}$ with $\mathbf{D} = \mathrm{diag}(p_i(1-p_i)) \succeq 0$, so $\mathbf{u}^\top\mathbf{H}\mathbf{u} = \|\mathbf{D}^{1/2}\mathbf{X}\mathbf{u}\|^2 \geq 0$. The loss is convex, so any local minimum is global.

Q1.5. What is a saddle point and why are they a bigger issue than local minima in deep learning?

Answer. A point with zero gradient where the Hessian has both positive and negative eigenvalues. In high dimensions, random critical points are overwhelmingly likely to be saddles rather than minima (each of millions of eigenvalues would have to be positive). SGD noise and momentum help escape them.

Chapter 2

Probability, Statistics and Information Theory

You did not ask for this chapter specifically, but data-science interviews test it constantly and many ML derivations later in the booklet (MLE, cross-entropy, Naive Bayes, VAEs) depend on it. It is kept compact and interview-focused.

2.1 Probability essentials

2.1.1 Rules

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.
- Conditional: $P(A \mid B) = P(A \cap B)/P(B)$. Independence: $P(A \cap B) = P(A)P(B)$.
- Total probability: $P(A) = \sum_i P(A \mid B_i)P(B_i)$ for a partition $\{B_i\}$.
- **Bayes:** $P(H \mid D) = \dfrac{P(D \mid H)P(H)}{P(D)}$ — posterior $\propto$ likelihood $\times$ prior.

Example 2.1 (The base-rate trap — asked constantly)**.** A fraud model flags 99% of fraudulent transactions and wrongly flags 2% of genuine ones. Fraud rate is 0.5%. Given a flag, what is $P(\text{fraud})$?

$$P(F \mid +) = \frac{0.99 \times 0.005}{0.99 \times 0.005 + 0.02 \times 0.995} = \frac{0.00495}{0.00495 + 0.0199} \approx 0.199.$$

Only about 20% of flags are real fraud. This is why precision collapses on rare classes even with a "99% accurate" model, and why a bank's fraud team cares about precision at a fixed review capacity.

2.1.2 Random variables, expectation, variance

- $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$ always (no independence needed).
- $\operatorname{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2$; $\operatorname{Var}(aX + b) = a^2 \operatorname{Var} X$.
- $\operatorname{Var}(X + Y) = \operatorname{Var} X + \operatorname{Var} Y + 2\operatorname{Cov}(X, Y)$.
- $\operatorname{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}X\mathbb{E}Y$; correlation $\rho = \operatorname{Cov}/(\sigma_X \sigma_Y) \in [-1, 1]$.
- Independence $\Rightarrow$ zero covariance, but **not** the reverse: $X \sim U(-1, 1)$, $Y = X^2$ has $\operatorname{Cov} = 0$ yet Y is a function of X.

- Law of total expectation $\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$; law of total variance $\operatorname{Var} X = \mathbb{E}[\operatorname{Var}(X \mid Y)] + \operatorname{Var}(\mathbb{E}[X \mid Y])$.

Portfolio connection: for weights $\mathbf{w}$ and covariance $\mathbf{\Sigma}$, the portfolio variance is $\mathbf{w}^\top \mathbf{\Sigma} \mathbf{w}$ — the same quadratic form as in PCA.

2.1.3 Distributions you must know

Distribution	PMF / PDF	Mean, Var	Where it shows up
Bernoulli(p)	$p^x(1-p)^{1-x}$	$p,\ p(1-p)$	click / no click; logistic regression target
Binomial(n, p)	$\binom{n}{x}p^x(1-p)^{n-x}$	$np,\ np(1-p)$	conversions out of n visitors
Geometric(p)	$(1-p)^{x-1}p$	$1/p,\ (1-p)/p^2$	trials until first success
Poisson(λ)	$e^{-\lambda}\lambda^x/x!$	$\lambda,\ \lambda$	calls per hour, claims per policy
Uniform(a, b)	$1/(b-a)$	$\frac{a+b}{2},\ \frac{(b-a)^2}{12}$	random init, simulations
Exponential(λ)	$\lambda e^{-\lambda x}$	$1/\lambda,\ 1/\lambda^2$	time between events; memoryless
Normal(μ, σ^2)	$\frac{1}{\sqrt{2\pi}\sigma}e^{-(x-\mu)^2/2\sigma^2}$	$\mu,\ \sigma^2$	noise, CLT, linear regression errors
Beta(α, β)	$\propto x^{\alpha-1}(1-x)^{\beta-1}$	$\frac{\alpha}{\alpha+\beta}$	prior on a probability (Bayesian A/B)

Normal facts: 68–95–99.7 rule; sum of independent normals is normal; standardise $Z = (X-\mu)/\sigma$. Poisson approximates Binomial when n large and p small with $\lambda = np$.

2.1.4 Limit theorems

Law of large numbers: $\bar{X}_n \to \mu$ as $n \to \infty$. **Central limit theorem:** for i.i.d. X_i with finite variance, $\sqrt{n}(\bar{X}_n - \mu)/\sigma \to N(0,1)$. The CLT is why confidence intervals and z-tests on averages work even when individual data (revenue per user) is very skewed — given enough samples. With heavy tails, "enough" can be very large.

2.2 Estimation

2.2.1 Maximum likelihood (MLE)

Choose $\boldsymbol{\theta}$ that makes the observed data most probable: $\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \sum_i \log p(x_i \mid \boldsymbol{\theta})$.

Example 2.2 (MLE for a coin)**.** n flips, k heads. $\ell(p) = k \log p + (n-k)\log(1-p)$; $\ell'(p) = k/p - (n-k)/(1-p) = 0 \Rightarrow \hat{p} = k/n$.

Example 2.3 (MLE for a Gaussian)**.** $\ell = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum(x_i-\mu)^2$. Setting derivatives to zero gives $\hat{\mu} = \bar{x}$ and $\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2$. This variance estimator is **biased** ($\mathbb{E}\hat{\sigma}^2 = \frac{n-1}{n}\sigma^2$); dividing by $n-1$ (Bessel's correction) removes the bias because one degree of freedom was used to estimate $\bar{x}$.

Key link: minimising MSE $\Leftrightarrow$ MLE under Gaussian noise; minimising cross-entropy $\Leftrightarrow$ MLE under a Bernoulli/categorical model; minimising MAE $\Leftrightarrow$ MLE under Laplace noise.

2.2.2 MAP and priors

$\hat{\boldsymbol{\theta}}_{\text{MAP}} = \arg\max[\log p(D \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})]$. A Gaussian prior on weights gives the L_2 (ridge) penalty; a Laplace prior gives the L_1 (lasso) penalty. Derivation in Chapter 4.

2.2.3 Properties of estimators

Bias $= \mathbb{E}\hat{\theta} - \theta$; MSE $=$ Bias^2 + Var. Consistency: $\hat{\theta} \to \theta$ as $n \to \infty$. MLE is consistent and asymptotically efficient under regularity conditions.

2.3 Hypothesis testing

2.3.1 Framework

- H_0: no effect. H_1: effect.
- **p-value**: probability, *assuming H_0 is true*, of a test statistic at least as extreme as observed. It is *not* the probability that H_0 is true.
- Type I error (α): reject a true H_0 (false positive). Type II (β): fail to reject a false H_0. **Power** $= 1 - \beta$.
- A 95% confidence interval: if we repeated the experiment many times, 95% of the intervals constructed this way would contain the true parameter. It does *not* mean "95% probability the parameter is in this particular interval" (that is a Bayesian credible interval).

2.3.2 Which test?

Situation	Test
Mean vs known value, σ known or n large	z-test
Mean vs value, σ unknown, small n	one-sample t-test
Two independent group means	two-sample t (Welch if variances differ)
Before/after on same units	paired t-test
Two proportions (conversion rates)	two-proportion z-test or χ^2
Association of two categorical variables	χ^2 test of independence
Means of 3+ groups	one-way ANOVA (F-test), then post-hoc (Tukey)
Non-normal, small samples	Mann–Whitney U, Wilcoxon signed-rank
Distribution equality	Kolmogorov–Smirnov (also used for drift detection)

Two-sample t statistic: $t = \dfrac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$.

2.4 A/B testing (core for DS roles)

Real-world example. A food-delivery app tests a new checkout button. Control converts at 10%. Product wants to detect a lift to 10.5% (5% relative). How many users per arm?

Sample size for two proportions (two-sided $\alpha = 0.05$, power 80%):

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \left[p_1(1-p_1) + p_2(1-p_2)\right]}{(p_2 - p_1)^2}$$
$$= \frac{(1.96 + 0.84)^2 (0.0900 + 0.0940)}{0.005^2} \approx 57{,}700 \text{ per arm.}$$

A rule of thumb is $n \approx 16\sigma^2/\delta^2$ per arm. Halving the detectable effect quadruples the sample size.

Process: define a primary metric and guardrail metrics (latency, cancellations) → choose randomisation unit (user, not session, to avoid one user seeing both) → compute sample size and duration (cover full weekly cycles) → run an A/A test or sample-ratio-mismatch check → analyse once at the planned end → decide with practical as well as statistical significance.

Pitfalls interviewers probe:

- **Peeking**: checking daily and stopping when $p < 0.05$ inflates false positives far above 5%. Fix: fixed horizon, or sequential tests (alpha spending, mSPRT).
- **Multiple comparisons**: testing 20 metrics at $\alpha = 0.05$ gives roughly a 64% chance of at least one false positive ($1 - 0.95^{20}$). Fix: Bonferroni (α/m) or Benjamini–Hochberg (controls false discovery rate).
- **Sample ratio mismatch**: a 50/50 split arriving as 52/48 signals a bug; check with χ^2.
- **Novelty / primacy effects**: users react to change itself; run long enough.
- **Network effects / interference**: in marketplaces (riders and drivers) treatment leaks into control; use cluster or switchback (time-sliced) randomisation.
- **Simpson's paradox**: an effect can reverse when data is aggregated across segments with different sizes.
- **Variance reduction**: CUPED uses pre-experiment metric X as a covariate: $Y' = Y - \theta(X - \bar{X})$ with $\theta = \mathrm{Cov}(X, Y)/\operatorname{Var} X$, cutting variance by a factor $(1 - \rho^2)$.

2.5 Information theory

- **Entropy** $H(p) = -\sum p_i \log p_i$: average surprise; maximal for uniform distributions. A fair coin has 1 bit (base 2).
- **Cross-entropy** $H(p, q) = -\sum p_i \log q_i$: cost of encoding data from p using code optimised for q. Classification loss with one-hot p is $-\log q_{\text{true}}$.
- **KL divergence** $D_{\mathrm{KL}}(p\|q) = \sum p_i \log(p_i/q_i) = H(p, q) - H(p) \geq 0$ (Gibbs' inequality, via Jensen). Not symmetric. Minimising cross-entropy w.r.t. q equals minimising KL because $H(p)$ is constant.
- **Mutual information** $I(X; Y) = H(X) - H(X \mid Y) = D_{\mathrm{KL}}(p(x, y)\|p(x)p(y))$: zero iff independent; used for feature selection.
- **Information gain** in decision trees = parent entropy − weighted child entropy = mutual information between the split and the label.

Forward KL $D_{\mathrm{KL}}(p\|q)$ is "mean-seeking" (q covers all modes of p); reverse KL $D_{\mathrm{KL}}(q\|p)$, used in variational inference, is "mode-seeking".

2.6 Classic probability puzzles

Q2.1. Two children, at least one is a boy. Probability both are boys?

Answer. Sample space $\{BB, BG, GB\}$ equally likely: 1/3. (If you know *the older* is a boy: 1/2.)

Q2.2. Expected number of die rolls to see a 6?

Answer. Geometric with $p = 1/6$: 6. Expected rolls to see all six faces (coupon collector): $6(1 + \frac{1}{2} + \cdots + \frac{1}{6}) = 14.7$.

Q2.3. Monty Hall: should you switch?

Answer. Yes. Switching wins with probability 2/3: you initially pick the car with 1/3; the host's reveal transfers the remaining 2/3 to the other closed door.

Q2.4. Birthday problem: how many people for $> 50\%$ chance of a shared birthday?

Answer. 23. $P(\text{no match}) = \prod_{k=0}^{22}(365-k)/365 \approx 0.493$.

Q2.5. You have a stick broken at two uniform random points. Probability the three pieces form a triangle?

Answer. 1/4. Each piece must be shorter than half the stick.

Q2.6. Generate a fair coin from a biased one.

Answer. Von Neumann: flip twice; HT $\to$ heads, TH $\to$ tails, otherwise repeat. $P(HT) = P(TH) = p(1-p)$.

Q2.7. Expected value of the maximum of two independent $U(0,1)$?

Answer. $P(\max \le x) = x^2$, density $2x$, mean $\int_0^1 2x^2 dx = 2/3$.

Q2.8. A random variable has mean 0, variance 1. Can $P(|X| \ge 3)$ be 0.2?

Answer. No. Chebyshev: $P(|X-\mu| \ge k\sigma) \le 1/k^2 = 1/9 \approx 0.111$.

Q2.9. Sampling from a stream of unknown length uniformly at random?

Answer. Reservoir sampling: keep the i-th item with probability $1/i$ (or k/i for a sample of k). Each item ends up with probability $1/n$.

Q2.10. Correlation is 0.9 between ice-cream sales and drownings. Interpretation?

Answer. Confounder (temperature/season). Correlation is not causation; establish causality with experiments or causal methods (difference-in-differences, instrumental variables, regression discontinuity, propensity matching).

Part II

Classical Machine Learning

Chapter 3

Machine Learning Fundamentals

3.1 The learning problem

Intuition. We have examples $(\mathbf{x}_i, y_i)$ drawn from some unknown process. We want a function f that does well on *new* examples from the same process, not just the ones we saw. Everything in ML — regularisation, validation, early stopping, ensembling — is a tool for closing the gap between "good on what I saw" and "good on what I will see".

Types of learning.

- **Supervised**: labels given. Regression (continuous y) and classification (discrete y).
- **Unsupervised**: no labels. Clustering, dimensionality reduction, density estimation, anomaly detection.
- **Semi-supervised**: few labels, many unlabelled points (pseudo-labelling, consistency regularisation).
- **Self-supervised**: labels manufactured from the data itself (predict the next word, the masked patch). Basis of LLM pre-training.
- **Reinforcement learning**: an agent acts, receives rewards, learns a policy.

Empirical risk minimisation (ERM). True risk $R(f) = \mathbb{E}_{(\mathbf{x},y)\sim P}[\ell(f(\mathbf{x}), y)]$ is unknowable. We minimise the empirical risk $\hat{R}(f) = \frac{1}{n}\sum_i \ell(f(\mathbf{x}_i), y_i)$ plus usually a complexity penalty $\lambda\Omega(f)$.

Parametric vs non-parametric. Parametric models have a fixed number of parameters (linear/logistic regression, a fixed neural net). Non-parametric models grow with the data (kNN, kernel SVM, decision trees grown fully, Gaussian processes).

Discriminative vs generative. Discriminative models learn $p(y \mid \mathbf{x})$ or a decision boundary directly (logistic regression, SVM). Generative models learn $p(\mathbf{x}, y) = p(\mathbf{x} \mid y)p(y)$ and use Bayes (Naive Bayes, LDA, GMM). Generative models can generate data and handle missing features; discriminative models usually have better accuracy with enough data.

No free lunch theorem. Averaged over all possible problems, no learner beats any other. Every algorithm wins because its assumptions (inductive bias) match the problem: CNNs assume locality and translation invariance; linear models assume additivity; trees assume axis-aligned thresholds matter.

3.2 Bias–variance trade-off

Intuition. Bias: the model is systematically wrong because it is too simple (a straight line through a curve). Variance: the model changes a lot depending on which training sample it saw (a degree-15

polynomial). You want both low, but reducing one typically increases the other.

3.2.1 Derivation (squared loss)

Assume $y = f(\mathbf{x}) + \varepsilon$ with $\mathbb{E}\varepsilon = 0$, $\operatorname{Var}\varepsilon = \sigma^2$, and $\hat{f}$ is learned from a random training set D. At a fixed $\mathbf{x}$, write $\bar{f} = \mathbb{E}_D[\hat{f}(\mathbf{x})]$.

$$\begin{aligned}\mathbb{E}_{D,\varepsilon}[(y-\hat{f})^2] &= \mathbb{E}[(f+\varepsilon-\hat{f})^2] = \mathbb{E}[(f-\hat{f})^2] + \sigma^2 \quad \text{(cross term vanishes since } \varepsilon \perp D\text{)} \\ &= \mathbb{E}[(f-\bar{f}+\bar{f}-\hat{f})^2] + \sigma^2 \\ &= \underbrace{(f-\bar{f})^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}-\bar{f})^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}} .\end{aligned}$$

The middle cross term $2(f-\bar{f})\mathbb{E}[\bar{f}-\hat{f}] = 0$ because $\mathbb{E}[\hat{f}] = \bar{f}$.

Symptom	Diagnosis	Remedies
High train error, high val error	High bias (underfit)	More features, more complex model, less regularisation, train longer
Low train error, high val error	High variance (overfit)	More data, regularisation, simpler model, dropout, bagging, early stopping, augmentation

Which models reduce what? Bagging/random forests mainly reduce variance; boosting mainly reduces bias. Increasing k in kNN increases bias and lowers variance. Deeper trees lower bias and raise variance.

Double descent (modern nuance). In very over-parameterised models (large neural nets), test error can fall again after the interpolation threshold, so the classical U-curve is incomplete. Worth mentioning in DL interviews; do not overclaim it.

3.3 Data splitting and validation

- **Train / validation / test**: fit on train, tune hyperparameters on validation, report once on test. Touching the test set repeatedly turns it into a validation set and makes the reported number optimistic.
- **k-fold CV**: average performance over k folds; lower variance estimate. $k = 5$ or 10 typical. LOOCV ($k = n$) is nearly unbiased but high variance and expensive.
- **Stratified k-fold**: preserves class proportions — use for imbalanced classification.
- **Group k-fold**: all rows of a customer/patient stay in one fold; otherwise the model memorises the person.
- **Time-series split**: train on the past, validate on the future (expanding or rolling window). Never shuffle time series.
- **Nested CV**: inner loop tunes hyperparameters, outer loop estimates generalisation of the whole tuning procedure.

3.3.1 Data leakage

Information unavailable at prediction time sneaks into training. It produces great offline scores and failure in production. Typical forms:

- **Target leakage**: feature derived from the label. E.g. "number of collection calls made" when predicting loan default — calls happen *after* default starts.
- **Train–test contamination**: fitting a scaler, imputer, PCA or target encoder on the full data before splitting. Fit preprocessing inside the CV fold (scikit-learn `Pipeline`).
- **Temporal leakage**: random split on time-ordered data; using future prices to predict today's.
- **Duplicate leakage**: the same entity appears in train and test.

Real-world example. A hospital pneumonia-mortality model learned that asthma patients had *lower* risk. True in the data — because asthmatics were sent straight to intensive care. The model captured treatment policy, not biology. Lesson: a feature correlated with an intervention can mislead badly; always sanity-check feature effects with domain experts.

3.4 Preprocessing and feature engineering

3.4.1 Missing values

Types: MCAR (completely at random), MAR (depends on observed variables), MNAR (depends on the missing value itself, e.g. high earners not disclosing income). Options: drop rows/columns; mean/median/mode imputation; model-based (kNN, iterative/MICE); add a "was missing" indicator (often highly predictive under MNAR); tree libraries like XGBoost/LightGBM learn a default direction for missing values.

3.4.2 Categorical encoding

- **One-hot**: safe for low cardinality; drop one level for linear models with an intercept (dummy-variable trap).
- **Ordinal / label encoding**: only when categories are ordered, or for tree models.
- **Target (mean) encoding**: replace category by mean target; must be computed out-of-fold with smoothing to avoid leakage.
- **Frequency encoding**, **hashing trick** (fixed-size, collisions acceptable), **learned embeddings** (neural nets, high cardinality like product IDs).

3.4.3 Scaling

Standardisation $z = (x - \mu)/\sigma$; min–max to $[0, 1]$; robust scaling with median and IQR (outlier-resistant); log transform for right-skewed data like income.

Needs scaling: kNN, k-means, SVM (RBF), PCA, linear/logistic regression with regularisation, neural networks, anything gradient-based. **Does not need it**: tree-based models (splits are invariant to monotone transforms).

3.4.4 Feature engineering ideas interviewers like

Ratios (debt/income), differences, date parts (day-of-week, month-end, holiday flags), lags and rolling statistics for time series, aggregations per entity (customer's average spend in last 30 days), interaction terms, binning, text features (TF-IDF, embeddings), geospatial distances.

3.4.5 Feature selection

- **Filter**: correlation, mutual information, χ^2, variance threshold. Fast, model-agnostic, ignores interactions.
- **Wrapper**: forward/backward selection, recursive feature elimination. Expensive, risk of overfitting the selection.
- **Embedded**: lasso, tree importances.
- Multicollinearity check: variance inflation factor $\text{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 comes from regressing feature j on the others. VIF $>$ 5–10 is a common warning threshold.

3.4.6 Curse of dimensionality

As d grows, data becomes sparse: the volume of a unit hypercube concentrates near the boundary, and distances between random points become nearly equal (ratio of nearest to farthest distance $\to 1$). Distance-based methods (kNN, k-means) degrade; more data is needed exponentially. Remedies: feature selection, PCA/embeddings, regularisation, models with strong inductive bias.

3.5 Class imbalance

- Use the right metric (PR-AUC, recall at fixed precision, F1) — not accuracy. A 99.5%-accurate fraud model can be one that predicts "not fraud" always.
- **Class weights** in the loss (`class_weight='balanced'`, `scale_pos_weight` in XGBoost).
- **Resampling**: random undersampling of the majority; oversampling the minority; **SMOTE** creates synthetic minority points by interpolating between a point and its minority-class neighbours. Resample only the training fold, never validation/test.
- **Threshold tuning**: the default 0.5 is rarely optimal; pick the threshold that maximises expected business value.
- Note that resampling and class weights distort predicted probabilities; recalibrate if you need true probabilities.

3.6 Evaluation metrics

3.6.1 Confusion matrix and derived metrics

	Predicted +	Predicted −
Actual +	TP	FN
Actual −	FP	TN

- Accuracy $= \frac{TP+TN}{\text{all}}$.
- **Precision** $= \frac{TP}{TP+FP}$ — of those I flagged, how many were right? (cost of false alarms)
- **Recall / sensitivity / TPR** $= \frac{TP}{TP+FN}$ — of all real positives, how many did I catch? (cost of misses)
- Specificity / TNR $= \frac{TN}{TN+FP}$; FPR $= 1-$specificity.
- $F_1 = \frac{2PR}{P+R}$ (harmonic mean, punishes imbalance between P and R); $F_\beta = (1+\beta^2)\frac{PR}{\beta^2 P+R}$, $\beta > 1$ weights recall more.

- MCC $= \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$ — balanced even with imbalance.

Example 3.1. 1000 loan applicants, 100 actually default. The model flags 80, of which 60 truly default. TP= 60, FP= 20, FN= 40, TN= 880. Accuracy = 94%; precision = 75%; recall = 60%; $F_1 = 0.667$; specificity = 880/900 = 97.8%.

Which to prioritise? Cancer screening, fraud detection where reviews are cheap: recall. Spam filtering, recommending a loan rejection that triggers legal review: precision. Always tie it to the cost of an FP versus an FN.

3.6.2 ROC and PR curves

- **ROC curve**: TPR vs FPR across thresholds. **AUC** = probability that a random positive gets a higher score than a random negative. 0.5 = random, 1 = perfect. Threshold-independent and invariant to class prior.
- With heavy imbalance ROC-AUC can look great while precision is poor (FPR stays tiny because TN is huge). **PR curve / average precision** focuses on the positive class; its random baseline equals the positive rate, not 0.5.
- **Gini coefficient** (credit scoring) $= 2 \cdot \text{AUC} - 1$. **KS statistic** = max separation between cumulative score distributions of goods and bads.

3.6.3 Probabilistic metrics and calibration

Log-loss $= -\frac{1}{n}\sum[y \log p + (1-y)\log(1-p)]$ punishes confident wrong answers heavily. Brier score $= \frac{1}{n}\sum(p-y)^2$. A model is **calibrated** if among cases predicted 0.7, about 70% are positive. Check with a reliability diagram; fix with Platt scaling (a logistic regression on scores) or isotonic regression. Random forests and boosted trees are often mis-calibrated; logistic regression is usually well-calibrated.

3.6.4 Regression metrics

- MSE, RMSE (same units as y, punishes large errors). MAE (robust to outliers; optimal constant prediction is the median, whereas for MSE it is the mean).
- MAPE $= \frac{1}{n}\sum |y - \hat{y}|/|y|$: scale-free but undefined at $y = 0$ and asymmetric. sMAPE, WAPE are alternatives used in demand forecasting.
- $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$: share of variance explained. Can be negative on test data (worse than predicting the mean). Never decreases as you add features on training data; adjusted $R^2 = 1 - (1-R^2)\frac{n-1}{n-d-1}$ penalises useless features.
- Huber loss: quadratic for small errors, linear for large — a compromise between MSE and MAE.
- Quantile (pinball) loss trains models for prediction intervals.

3.6.5 Ranking metrics

Precision@k, Recall@k, MAP (mean of average precision across queries), MRR (mean reciprocal rank of the first relevant item), and NDCG@k: DCG $= \sum_{i=1}^{k} \frac{2^{rel_i}-1}{\log_2(i+1)}$ normalised by the ideal ordering's DCG.

3.7 Interview questions

Q3.1. Your model has 98% training accuracy and 70% validation accuracy. What do you do?

Answer. Overfitting. Check for leakage or distribution mismatch between splits first; then add regularisation, reduce complexity, get more data or augment, use early stopping, ensembling. Also check that the validation set is representative (same time period, same customer mix).

Q3.2. Validation score is great but production performance is poor. Causes?

Answer. Leakage; train–serve skew (feature computed differently online); data drift or concept drift; label definition differences; the offline metric not matching the business metric; selection bias in historical data (e.g. only approved loans have default labels — "reject inference" problem).

Q3.3. When is accuracy a bad metric?

Answer. Under class imbalance, and whenever FP and FN costs differ.

Q3.4. Why does R^2 increase when you add a random noise feature?

Answer. OLS can only reduce training SSE by adding a column (it can set the coefficient to zero at worst), so R^2 cannot fall in-sample. Use adjusted R^2, out-of-sample R^2, AIC/BIC.

Q3.5. Explain AUC to a business stakeholder.

Answer. "If I pick one customer who churned and one who didn't, AUC is the chance the model ranks the churner as riskier."

Q3.6. How would you handle a categorical feature with 50,000 levels?

Answer. Target encoding with out-of-fold computation and smoothing, frequency encoding, hashing, grouping rare levels into "other", or learned embeddings. Not one-hot for linear models unless sparse and regularised.

Q3.7. Why stratify the split?

Answer. To keep class proportions the same across folds so that rare-class metrics are estimated stably.

Chapter 4

Linear Models

4.1 Linear regression

Intuition. Fit the flat surface that sits as close as possible (in squared vertical distance) to the data cloud. Each coefficient answers: "holding the other features fixed, how much does y change for a one-unit increase in this feature?"

Real-world example. A used-car platform predicts price from age, kilometres driven, engine size and brand. A coefficient of -0.4 lakh on age means each extra year lowers the expected price by about 40,000 rupees, holding kilometres and the rest fixed. Note the "holding fixed": age and kilometres are correlated, so the raw relationship of price with age is steeper than the coefficient.

4.1.1 Model and OLS solution

$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, with $\mathbf{X} \in \mathbb{R}^{n\times(d+1)}$ including a column of ones. Minimise $L(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$.

Derivation. Expand: $L = \mathbf{y}^\top\mathbf{y} - 2\boldsymbol{\beta}^\top\mathbf{X}^\top\mathbf{y} + \boldsymbol{\beta}^\top\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta}$. Gradient: $\nabla L = -2\mathbf{X}^\top\mathbf{y} + 2\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} = 0$, giving the **normal equations**

$$\mathbf{X}^\top\mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^\top\mathbf{y} \quad \Rightarrow \quad \hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}.$$

The Hessian $2\mathbf{X}^\top\mathbf{X}$ is PSD, so this is a minimum.

Simple regression special case: $\hat{\beta}_1 = \dfrac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \dfrac{\mathrm{Cov}(x,y)}{\mathrm{Var}(x)} = r\dfrac{s_y}{s_x}$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1\bar{x}$. The fitted line passes through $(\bar{x}, \bar{y})$. Note the CAPM beta is exactly this slope of stock returns on market returns.

Geometric view. $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{H}\mathbf{y}$ with hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$, the orthogonal projection onto the column space of $\mathbf{X}$. Residuals $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ are orthogonal to every column: $\mathbf{X}^\top\mathbf{e} = 0$. Consequences: with an intercept, residuals sum to zero; $SS_{tot} = SS_{reg} + SS_{res}$. Diagonal h_{ii} of $\mathbf{H}$ is the **leverage** of point i; Cook's distance combines leverage and residual size to find influential points.

Probabilistic view. If $\varepsilon_i \sim N(0, \sigma^2)$ i.i.d., the log-likelihood is $-\frac{1}{2\sigma^2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \text{const}$, so OLS = MLE.

4.1.2 Assumptions (classical linear model)

1. **Linearity** in parameters (features can be non-linear transforms: x^2, $\log x$).
2. **Exogeneity**: $\mathbb{E}[\varepsilon \mid \mathbf{X}] = 0$ (no omitted confounders). Violated ⇒ biased coefficients.

3. **No perfect multicollinearity**: $\mathbf{X}$ full column rank.
4. **Homoskedasticity**: constant $\mathrm{Var}(\varepsilon_i)$. Violated $\Rightarrow$ coefficients still unbiased, but standard errors wrong; use robust (White/HC) errors or weighted least squares. Detect with residual-vs-fitted plots, Breusch–Pagan.
5. **No autocorrelation** of errors (time series). Detect with Durbin–Watson; use Newey–West errors.
6. **Normality** of errors — only needed for exact small-sample t/F inference, not for unbiasedness. Check with a Q–Q plot.

Theorem 4.1 (Gauss–Markov)**.** Under assumptions 1–5, OLS is BLUE: the Best (minimum variance) Linear Unbiased Estimator. Normality is not required.

4.1.3 Inference

$\mathrm{Var}(\hat{\boldsymbol{\beta}} \mid \mathbf{X}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$; estimate $\hat{\sigma}^2 = SS_{res}/(n-d-1)$. $t_j = \hat{\beta}_j/\mathrm{SE}(\hat{\beta}_j)$ tests $\beta_j = 0$. The F-test checks whether all slopes are jointly zero. A non-significant coefficient with a significant overall F often signals multicollinearity: the features matter together but their individual effects cannot be separated, and standard errors inflate.

4.1.4 Interpreting coefficients under transforms

Model	Interpretation of β
$y = \beta x$	$+1$ in $x \Rightarrow +\beta$ in y
$\log y = \beta x$	$+1$ in $x \Rightarrow$ approx. $100\beta\%$ change in y
$y = \beta \log x$	$+1\%$ in $x \Rightarrow +\beta/100$ in y
$\log y = \beta \log x$	elasticity: $+1\%$ in $x \Rightarrow +\beta\%$ in y

4.2 Regularised regression

Intuition. When you have many features or correlated features, OLS coefficients become large and unstable (they offset each other). Regularisation adds a "price" for big coefficients, trading a little bias for a large drop in variance.

4.2.1 Ridge (L_2)

$\min \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_2^2$. Gradient: $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + 2\lambda\boldsymbol{\beta} = 0 \Rightarrow \hat{\boldsymbol{\beta}}_{ridge} = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$.

- $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ has eigenvalues $\lambda_i + \lambda > 0$, so it is always invertible — fixes multicollinearity.
- SVD view: ridge shrinks the component along singular direction i by $\sigma_i^2/(\sigma_i^2 + \lambda)$; low-variance directions are shrunk most.
- Shrinks coefficients toward zero but never exactly zero.
- Do not penalise the intercept; standardise features first (the penalty depends on scale).

4.2.2 Lasso (L_1)

$\min \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_1$. No closed form; solved by coordinate descent. For one standardised coordinate the update is the **soft-thresholding** operator:

$$\beta_j \leftarrow S(\rho_j, \lambda) = \mathrm{sign}(\rho_j)\max(|\rho_j| - \lambda, 0),$$

where ρ_j is the correlation of feature j with the partial residual. Any feature with $|\rho_j| \le \lambda$ is set *exactly* to zero: built-in feature selection.

Why L_1 gives sparsity, geometrically. Write the problem as minimising squared error subject to $\|\boldsymbol{\beta}\|_1 \le t$. The L_1 ball is a diamond with corners on the axes; the elliptical error contours usually first touch it at a corner, where some coordinates are zero. The L_2 ball is round, so the touching point generically has all coordinates non-zero.

Lasso weaknesses. With correlated features it picks one somewhat arbitrarily; with $d > n$ it selects at most n features.

4.2.3 Elastic net

$\lambda_1\|\boldsymbol{\beta}\|_1 + \lambda_2\|\boldsymbol{\beta}\|_2^2$: sparsity plus grouping of correlated features.

4.2.4 Bayesian interpretation (derivation)

With $y \mid \mathbf{x}, \boldsymbol{\beta} \sim N(\mathbf{x}^\top\boldsymbol{\beta}, \sigma^2)$ and prior $\beta_j \sim N(0, \tau^2)$:

$$-\log p(\boldsymbol{\beta} \mid D) = \frac{1}{2\sigma^2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \frac{1}{2\tau^2}\|\boldsymbol{\beta}\|^2 + \text{const}$$

— ridge with $\lambda = \sigma^2/\tau^2$. A Laplace prior $p(\beta_j) \propto e^{-|\beta_j|/b}$ gives the lasso. A tighter prior (small τ) means stronger regularisation.

4.3 Logistic regression

Intuition. We want a probability, so squeeze a linear score into $(0, 1)$ with the sigmoid. Equivalently, the model says the *log-odds* of the event are linear in the features.

4.3.1 Model

$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top\mathbf{x} + b)$, so $\log\frac{p}{1-p} = \mathbf{w}^\top\mathbf{x} + b$. The decision boundary $\mathbf{w}^\top\mathbf{x} + b = 0$ is linear. e^{w_j} is the **odds ratio**: a one-unit increase in x_j multiplies the odds by e^{w_j}.

Real-world example. In a credit-scoring scorecard, if the coefficient on "number of missed payments in the last year" is 0.35, each extra missed payment multiplies the odds of default by $e^{0.35} \approx 1.42$, i.e. 42% higher odds. Scorecards in banks are often still logistic regressions because regulators want this kind of explainability.

4.3.2 Loss and gradient (derive this on a whiteboard)

Likelihood $\prod p_i^{y_i}(1-p_i)^{1-y_i}$. Negative log-likelihood (binary cross-entropy):

$$L(\mathbf{w}) = -\sum_i \big[y_i \log p_i + (1 - y_i)\log(1 - p_i)\big], \quad p_i = \sigma(z_i),\ z_i = \mathbf{w}^\top\mathbf{x}_i.$$

For one sample, using $\sigma' = \sigma(1-\sigma)$:

$$\frac{\partial L}{\partial z} = -\Big[\frac{y}{p} - \frac{1-y}{1-p}\Big]p(1-p) = -[y(1-p) - (1-y)p] = p - y.$$

Hence $\nabla_{\mathbf{w}} L = \sum_i (p_i - y_i)\mathbf{x}_i = \mathbf{X}^\top(\mathbf{p} - \mathbf{y})$ — the same form as linear regression's gradient with predictions replaced by probabilities.

Hessian $\mathbf{H} = \mathbf{X}^\top \mathbf{D}\mathbf{X}$, $\mathbf{D} = \mathrm{diag}(p_i(1-p_i))$: PSD, so the loss is convex. Newton's update $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{H}^{-1}\mathbf{X}^\top(\mathbf{p}-\mathbf{y})$ is **IRLS** (iteratively reweighted least squares).

Why not use MSE for logistic regression? MSE composed with a sigmoid is non-convex in $\mathbf{w}$, and its gradient $(p-y)p(1-p)$ vanishes when the model is confidently wrong ($p \approx 0$, $y = 1$), so learning stalls. Cross-entropy gives gradient $p - y$, large exactly when wrong.

Perfect separation. If classes are linearly separable, the MLE does not exist: $\|\mathbf{w}\| \to \infty$ keeps increasing the likelihood. Regularisation fixes it.

4.3.3 Multiclass: softmax regression

$p(y = k \mid \mathbf{x}) = \dfrac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_j e^{\mathbf{w}_j^\top \mathbf{x}}}$, loss $-\sum_i \log p(y_i \mid \mathbf{x}_i)$, gradient w.r.t. logits $= \mathbf{p} - \mathbf{y}_{\text{one-hot}}$. Alternative: one-vs-rest with K binary classifiers. Softmax is invariant to adding a constant to all logits, so implementations subtract $\max_j z_j$ for numerical stability.

4.4 Generalised linear models (GLMs)

A GLM has: a distribution from the exponential family, a linear predictor $\eta = \mathbf{x}^\top \boldsymbol{\beta}$, and a link $g(\mathbb{E}[y]) = \eta$.

- Gaussian + identity link = linear regression.
- Bernoulli + logit = logistic regression.
- Poisson + log link = Poisson regression for counts (insurance claim frequency). Use negative binomial if overdispersed (variance > mean).
- Gamma + log link for positive skewed amounts (claim severity). Insurance pricing often models frequency × severity, or uses a Tweedie distribution for the combined pure premium.

4.5 Model selection criteria

AIC $= 2k - 2\log \hat{L}$; BIC $= k \log n - 2 \log \hat{L}$ (stronger penalty for large n, favours smaller models). Lower is better. Useful for statistical models; for predictive ML, cross-validation is the norm.

4.6 Interview questions

Q4.1. Derive the closed-form solution of linear regression.

Answer. See above: set $\nabla_\beta \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = 0$.

Q4.2. What happens to OLS with perfectly correlated features? Highly correlated?

Answer. Perfect: $\mathbf{X}^\top\mathbf{X}$ singular, infinitely many solutions. High: solution exists but coefficients have huge variance and may flip signs; predictions can still be fine. Fix with ridge, dropping/combining features, PCA.

Q4.3. Lasso vs ridge — when to use which?

Answer. Lasso when you believe few features matter and want selection/interpretability; ridge when many small effects or correlated features; elastic net when both. Tune λ by CV.

Q4.4. Is logistic regression a classification or regression algorithm?

Answer. It regresses the log-odds (a continuous quantity) and becomes a classifier once you apply a threshold. Its output is a probability.

Q4.5. Why is logistic regression's decision boundary linear even though the sigmoid is non-linear?

Answer. $p = 0.5 \iff \mathbf{w}^\top \mathbf{x} + b = 0$, a hyperplane. The sigmoid is monotone, so thresholds on p are thresholds on the linear score.

Q4.6. If you duplicate every row in the dataset, what happens to OLS coefficients and their standard errors?

Answer. Coefficients unchanged. Standard errors shrink by about $\sqrt{2}$ because the software believes n doubled — false confidence. Same intuition explains why non-independent observations (repeated customers) understate uncertainty.

Q4.7. If you add a feature identical to an existing one with ridge regularisation?

Answer. The weight splits equally between the two copies (ridge is "grouping"). Lasso would arbitrarily keep one.

Q4.8. How do you handle a non-linear relationship in linear regression?

Answer. Transform features (polynomials, logs, splines), add interactions, or move to a non-linear model.

Q4.9. Gradient of the MSE loss for linear regression and update rule?

Answer. $\nabla = \frac{2}{n}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$; $\mathbf{w} \leftarrow \mathbf{w} - \eta\nabla$.

Chapter 5

Classical Classifiers: kNN, Naive Bayes, LDA, SVM and Decision Trees

5.1 k-Nearest Neighbours

Intuition. "Tell me who your neighbours are and I'll tell you who you are." Predict by majority vote (classification) or average (regression) of the k closest training points.

- No training (lazy learner); prediction costs $O(nd)$ per query by brute force. KD-trees and ball trees speed this up in low dimensions; approximate nearest neighbour indexes (HNSW, IVF, product quantisation as in FAISS) are used for millions of embeddings, e.g. in vector databases for retrieval.
- Small k: low bias, high variance (jagged boundary). Large k: smooth, high bias. $k = 1$ has zero training error.
- Needs feature scaling; suffers badly from the curse of dimensionality.
- Distance-weighted voting gives closer neighbours more say.
- Distances: Euclidean, Manhattan, Minkowski, cosine, Hamming (binary), Mahalanobis (accounts for correlation).

5.2 Naive Bayes

Intuition. Use Bayes' rule, and make the "naive" assumption that features are independent given the class. It is almost always false, yet the classifier ranks classes well because only the *argmax* matters, not calibrated probabilities.

$$\hat{y} = \arg\max_c \; P(c) \prod_{j=1}^{d} P(x_j \mid c) = \arg\max_c \Big[\log P(c) + \sum_j \log P(x_j \mid c) \Big].$$

Log-space avoids underflow from multiplying many small probabilities.

Variants: **Gaussian** NB (continuous features, per-class mean/variance), **Multinomial** NB (word counts — text), **Bernoulli** NB (word present/absent).

Laplace (add-one) smoothing: $P(w \mid c) = \dfrac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha |V|}$. Without it, a word never seen in spam training data gives $P = 0$ and wipes out the whole product.

Example 5.1 (Spam filter)**.** Training: 40 spam and 60 ham emails. "offer" appears in 20 spam and 3 ham emails (Bernoulli view). For an email containing only "offer" (ignoring other words): $P(\text{spam} \mid \text{offer}) \propto 0.4 \times \frac{20}{40} = 0.2$, $P(\text{ham} \mid \text{offer}) \propto 0.6 \times \frac{3}{60} = 0.03$. Normalised: spam with probability $0.2/0.23 \approx 0.87$.

Strengths: very fast, works with little data, strong baseline for text. Weaknesses: correlated features are double-counted, so probabilities are over-confident.

5.3 Linear and Quadratic Discriminant Analysis

Generative: model $p(\mathbf{x} \mid c) = N(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$. If all classes share $\boldsymbol{\Sigma}$ (LDA), the log-ratio of posteriors is linear in $\mathbf{x} \Rightarrow$ linear boundary. Separate $\boldsymbol{\Sigma}_c$ (QDA) gives quadratic boundaries. LDA also gives a supervised dimensionality reduction: project onto at most $K-1$ directions maximising between-class over within-class variance (Fisher's criterion $\max \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$). Do not confuse with Latent Dirichlet Allocation (topic modelling), also abbreviated LDA.

5.4 Support Vector Machines

Intuition. Among all lines that separate two classes, pick the one with the widest "street" between them. Only the points on the edge of the street (support vectors) determine it; points far away don't matter. A wide margin means a small change in the data is unlikely to flip predictions — good generalisation.

5.4.1 Hard-margin SVM (derivation)

Labels $y_i \in \{-1, +1\}$, classifier $\text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. The distance from $\mathbf{x}_i$ to the hyperplane is $|\mathbf{w}^\top \mathbf{x}_i + b|/\|\mathbf{w}\|$. Scale $\mathbf{w}, b$ so that the closest points satisfy $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$; the margin width is then $2/\|\mathbf{w}\|$. Maximising it is equivalent to

$$\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \ \forall i.$$

5.4.2 Soft margin and hinge loss

Allow violations with slacks $\xi_i \geq 0$:

$$\min \tfrac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i.$$

At the optimum $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$, so this is the unconstrained problem

$$\min \sum_i \max\left(0, 1 - y_i f(\mathbf{x}_i)\right) + \tfrac{1}{2C}\|\mathbf{w}\|^2,$$

i.e. **hinge loss** + L_2 regularisation. Large C: few violations, narrow margin, risk of overfitting. Small C: wider margin, more violations, more regularisation.

Hinge vs log loss: hinge is exactly zero for confidently correct points (beyond the margin), giving sparsity in support vectors; logistic loss is never zero and gives probabilities. SVM outputs are not probabilities (Platt scaling needed).

5.4.3 The dual and the kernel trick

Lagrangian: $\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1]$. Stationarity gives $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and $\sum_i \alpha_i y_i = 0$. Substituting back yields the dual

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t. } 0 \le \alpha_i \le C, \; \sum_i \alpha_i y_i = 0.$$

Complementary slackness: $\alpha_i > 0$ only for points on or inside the margin — the support vectors. Prediction $f(\mathbf{x}) = \sum_i \alpha_i y_i \, \mathbf{x}_i^\top \mathbf{x} + b$.

Kernel trick. Data appears only through dot products, so replace $\mathbf{x}_i^\top \mathbf{x}_j$ with $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ — a dot product in a (possibly infinite-dimensional) feature space, computed without ever building ϕ.

- Polynomial $K = (\mathbf{x}^\top \mathbf{z} + c)^p$. For $p = 2$, $d = 2$: ϕ contains $x_1^2, x_2^2, \sqrt{2}x_1 x_2, \ldots$
- RBF/Gaussian $K = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$: infinite-dimensional ϕ. Large $\gamma \Rightarrow$ each point's influence is very local $\Rightarrow$ wiggly boundary, overfitting. Small $\gamma \Rightarrow$ smoother.
- A valid kernel must give a PSD Gram matrix for any data (Mercer's condition).

Complexity. Training kernel SVMs scales roughly between $O(n^2)$ and $O(n^3)$ — impractical above roughly 10^5 rows. Linear SVMs (liblinear, SGD) scale to millions. SVR (regression) uses an ϵ-insensitive tube instead of hinge.

5.5 Decision trees

Intuition. Play "20 questions" with the data: at each step, ask the yes/no question ("income > 50k?") that best separates the classes, then recurse on each side.

5.5.1 Splitting criteria

For a node with class proportions p_k:

- **Gini impurity** $G = 1 - \sum_k p_k^2$ (probability of misclassifying a random point labelled by the node's distribution). Max 0.5 for two balanced classes.
- **Entropy** $H = -\sum_k p_k \log_2 p_k$. Max 1 bit for two balanced classes. Split by maximising **information gain** $= H(\text{parent}) - \sum_{\text{child}} \frac{n_c}{n} H(c)$.
- Gain ratio (C4.5) normalises by split entropy to stop favouring many-valued attributes.
- Regression trees: minimise weighted variance (MSE) of children; a leaf predicts its mean.

Gini and entropy almost always choose the same split; Gini is slightly cheaper (no log).

Example 5.2 (Computing a split)**.** Parent: 10 customers, 5 churn / 5 stay. Gini $= 1 - 0.25 - 0.25 = 0.5$. Split on "contract = monthly": left has 6 (5 churn, 1 stay), right 4 (0 churn, 4 stay). Left Gini $= 1 - (5/6)^2 - (1/6)^2 = 10/36 \approx 0.278$; right Gini $= 0$. Weighted $= 0.6 \times 0.278 + 0 = 0.167$. Gini decrease $= 0.333$.

5.5.2 Algorithm and complexity

CART: greedy, binary splits, for each feature sort values and scan thresholds; cost per level about $O(nd \log n)$. Greedy splitting is not globally optimal (finding the optimal tree is NP-hard).

5.5.3 Controlling overfitting

A fully grown tree memorises training data. Pre-pruning: `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, minimum impurity decrease. Post-pruning: cost-complexity pruning, minimise $R(T) + \alpha|T|$ over subtrees, choose α by CV.

5.5.4 Pros and cons

Pros: interpretable, no scaling needed, handles mixed types and non-linearity and interactions automatically, robust to outliers in features. **Cons**: high variance (small data change → different tree), axis-aligned boundaries (a diagonal boundary needs a staircase), poor extrapolation in regression (predicts constants beyond the training range), biased toward high-cardinality features. Ensembles fix most of these.

5.6 Interview questions

Q5.1. Why is Naive Bayes "naive", and why does it still work?

Answer. It assumes conditional independence of features given the class. Classification only needs the right ordering of posteriors, which is often preserved even when probability values are badly off.

Q5.2. What are support vectors? What happens if you remove a non-support vector?

Answer. Points with $\alpha_i > 0$ (on or within the margin). Removing any other point leaves the solution unchanged.

Q5.3. SVM vs logistic regression?

Answer. Both linear (without kernels). LR gives probabilities and uses all points; SVM maximises margin, depends only on support vectors, more robust to points far from the boundary, kernelises naturally. With many features and few samples (text), linear SVMs are strong; when you need probabilities or interpretability, LR.

Q5.4. Effect of C and γ in an RBF SVM?

Answer. Larger $C \Rightarrow$ less regularisation; larger $\gamma \Rightarrow$ more local, complex boundary. Both large $\Rightarrow$ overfitting. Grid-search on a log scale.

Q5.5. Can a decision tree's training error be zero? Should it be?

Answer. Yes, if no two identical feature vectors have different labels. It should not — that is overfitting.

Q5.6. How does a decision tree handle a continuous feature?

Answer. Sorts its values and evaluates thresholds at midpoints between consecutive distinct values, choosing the best impurity decrease.

Q5.7. What is the time complexity of kNN prediction, and how do you speed it up?

Answer. $O(nd)$ per query. Use KD/ball trees in low dimensions, or approximate NN (HNSW, IVF-PQ) in high dimensions; reduce d with PCA.

Chapter 6

Ensembles: Bagging, Random Forests and Boosting

Intuition. A committee of diverse, reasonably good models beats any single member. **Bagging** builds many independent strong-but-noisy models and averages them (cuts variance). **Boosting** builds many weak models sequentially, each fixing the mistakes of those before (cuts bias). For tabular data — credit risk, churn, fraud, pricing — gradient-boosted trees remain the default winning approach in industry and on Kaggle.

6.1 Why averaging helps

If B models each have variance σ^2 and pairwise correlation ρ, the variance of their average is

$$\mathrm{Var}\left(\frac{1}{B}\sum_b f_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Derivation: $\frac{1}{B^2}[B\sigma^2 + B(B-1)\rho\sigma^2] = \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2$, which rearranges to the above. As $B \to \infty$ the second term vanishes but $\rho\sigma^2$ remains — so the key to good ensembles is *low correlation* between members. That motivates random forests' feature subsampling.

6.2 Bagging

Bootstrap aggregating: draw B bootstrap samples (size n, with replacement), train a model on each, average (regression) or vote (classification). Each bootstrap sample contains about $1-(1-1/n)^n \to 1-1/e \approx 63.2\%$ of unique points; the remaining $\approx 36.8\%$ are **out-of-bag (OOB)** and give a free validation estimate. Works best with high-variance, low-bias base learners (deep trees).

6.3 Random forests

Bagging of decision trees + at every split, consider only a random subset of `max_features` features (typical: $\sqrt{d}$ for classification, $d/3$ for regression). This decorrelates trees: otherwise one strong feature would sit at the root of every tree.

Key hyperparameters: `n_estimators` (more is never worse for accuracy, just slower), `max_features` (main decorrelation knob), `max_depth`, `min_samples_leaf`, `bootstrap`.

Feature importance.

- **Impurity-based (MDI)**: total impurity decrease from splits on a feature. Fast, but biased toward high-cardinality and continuous features, computed on training data.
- **Permutation importance**: shuffle one feature in validation data and measure the drop in score. Model-agnostic, more reliable; but correlated features share credit (shuffling one leaves its twin).
- **SHAP values**: see below.

Pros: strong out-of-the-box, hard to overfit badly with more trees, parallel, OOB estimate. Cons: large models, slower inference, cannot extrapolate, less accurate than well-tuned boosting on most tabular tasks. Extra-Trees choose split thresholds randomly too — even lower variance, faster.

6.4 Boosting

6.4.1 AdaBoost

Labels ± 1, sample weights w_i start at $1/n$. For $m = 1..M$:

1. Fit weak learner h_m (often a stump) on weighted data; weighted error $\epsilon_m = \sum_i w_i \mathbb{1}[h_m(\mathbf{x}_i) \neq y_i]$.
2. Learner weight $\alpha_m = \frac{1}{2} \log \frac{1-\epsilon_m}{\epsilon_m}$ (positive only if better than chance).
3. Update $w_i \leftarrow w_i \exp(-\alpha_m y_i h_m(\mathbf{x}_i))$, then normalise. Misclassified points gain weight.

Final $H(\mathbf{x}) = \operatorname{sign} \sum_m \alpha_m h_m(\mathbf{x})$. AdaBoost equals forward stagewise additive modelling under the **exponential loss** $e^{-yf(\mathbf{x})}$, which makes it sensitive to outliers and label noise.

6.4.2 Gradient boosting (derivation)

Build an additive model $F_M(\mathbf{x}) = F_0(\mathbf{x}) + \sum_{m=1}^{M} \eta\, h_m(\mathbf{x})$ for any differentiable loss ℓ. View it as gradient descent in function space:

1. Initialise $F_0 = \arg\min_c \sum_i \ell(y_i, c)$ (mean for MSE; log-odds of the base rate for log-loss).
2. For $m = 1..M$: compute **pseudo-residuals** $r_{im} = -\Big[\dfrac{\partial \ell(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\Big]_{F=F_{m-1}}$.
3. Fit a regression tree h_m to $\{(\mathbf{x}_i, r_{im})\}$.
4. For each leaf j, set the value $\gamma_{jm} = \arg\min_\gamma \sum_{\mathbf{x}_i \in R_{jm}} \ell(y_i, F_{m-1}(\mathbf{x}_i) + \gamma)$.
5. Update $F_m = F_{m-1} + \eta \sum_j \gamma_{jm} \mathbb{1}[\mathbf{x} \in R_{jm}]$.

For MSE $\ell = \frac{1}{2}(y - F)^2$: $r = y - F$, the plain residual — each tree literally fits what is left to explain. **For log-loss**: $r = y - p$. **For MAE**: $r = \operatorname{sign}(y - F)$.

The learning rate (shrinkage) $\eta \in [0.01, 0.3]$: smaller η needs more trees but generalises better. Use early stopping on a validation set. Unlike random forests, *more trees can overfit.*

Real-world example. Predicting delivery time for a food order: tree 1 learns "distance matters". Residuals show it under-predicts on rainy Friday evenings; tree 2 captures "rain × peak hour". Tree 3 picks up a specific slow restaurant. Each tree is small, but together they model complex interactions.

6.4.3 XGBoost: second-order boosting (derivation)

Objective at step t with regularised trees:

$$\mathcal{L}^{(t)} = \sum_i \ell(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t), \quad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2,$$

where T = number of leaves, w_j = leaf values. Second-order Taylor expansion with $g_i = \partial \ell / \partial \hat{y}$, $h_i = \partial^2 \ell / \partial \hat{y}^2$:

$$\mathcal{L}^{(t)} \approx \sum_i \left[g_i f_t(\mathbf{x}_i) + \tfrac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \gamma T + \tfrac{1}{2}\lambda \sum_j w_j^2 + \text{const.}$$

Group by leaf (I_j = samples in leaf j, $G_j = \sum_{i \in I_j} g_i$, $H_j = \sum_{i \in I_j} h_i$):

$$\mathcal{L}^{(t)} = \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2}(H_j + \lambda) w_j^2 \right] + \gamma T.$$

Minimising each quadratic: $w_j^* = -\dfrac{G_j}{H_j + \lambda}$, giving objective $-\frac{1}{2} \sum_j \frac{G_j^2}{H_j + \lambda} + \gamma T$. Hence the **split gain**

$$\text{Gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \Big] - \gamma.$$

A split is made only if Gain > 0: γ is a built-in pruning threshold. For log-loss, $g = p - y$, $h = p(1 - p)$.

Other XGBoost engineering: sparsity-aware split finding (learns a default direction for missing values), weighted quantile sketch for approximate splits, column blocks for parallel split search, cache awareness, out-of-core training, `min_child_weight` (minimum H in a leaf), row and column subsampling.

6.4.4 LightGBM

- **Histogram-based** splits: bin features into ≤ 255 buckets — fast and memory-light.
- **Leaf-wise growth**: split the leaf with the largest gain anywhere in the tree (XGBoost default is level-wise). Lower loss for the same number of leaves but can overfit small data; control with `num_leaves`, `min_data_in_leaf`, `max_depth`.
- **GOSS** (gradient-based one-side sampling): keep all large-gradient samples, sample the small-gradient ones and up-weight them.
- **EFB** (exclusive feature bundling): merge sparse features that are rarely non-zero together.
- Native categorical handling.

6.4.5 CatBoost

- **Ordered target statistics** for categorical features: encode each row using only rows "before" it in a random permutation, avoiding target leakage.
- **Ordered boosting** to reduce prediction shift (residuals computed by a model that has seen the row).
- **Oblivious (symmetric) trees**: same split at every node of a level — fast inference, regularising effect.
- Often strong with default parameters on data with many categoricals.

6.4.6 Tuning gradient boosting — a practical order

1. Fix a moderate learning rate (0.05–0.1) and use early stopping to choose the number of trees.
2. Tune tree complexity: `max_depth` (3–10) or `num_leaves`; and `min_child_weight` or `min_data_in_leaf`.
3. Tune randomness: `subsample`, `colsample_bytree` (0.5–1).
4. Tune regularisation: `lambda`, `alpha`, `gamma`.
5. Lower the learning rate and increase trees for the final model.

Bayesian optimisation (Optuna) usually beats grid search for this.

6.5 Stacking and blending

Train diverse base models; use their *out-of-fold* predictions as features for a meta-learner (often logistic/linear regression). Using in-fold predictions leaks. Blending uses a single holdout instead of OOF. Gains are usually small but real in competitions; in production the complexity cost often is not worth it.

6.6 Model interpretability

- **Partial dependence plot (PDP)**: average prediction as one feature varies; misleading when features are correlated. **ICE** plots show it per instance; **ALE** plots handle correlation better.
- **LIME**: fit a simple local surrogate (linear model) around one prediction using perturbed samples. Can be unstable.
- **SHAP**: Shapley values from cooperative game theory — the average marginal contribution of a feature over all orderings of features. Properties: local accuracy (contributions sum to prediction minus baseline), consistency, missingness. TreeSHAP computes exact values for trees in polynomial time. Standard for explaining credit and fraud models to risk teams and regulators.

6.7 Interview questions

Q6.1. Bagging vs boosting?

Answer. Bagging: parallel, independent models on bootstrap samples, averages to reduce variance, base learners are strong/deep. Boosting: sequential, each model fits the errors (gradients) of the ensemble so far, reduces bias, base learners are weak/shallow; more sensitive to noise and needs careful tuning.

Q6.2. Why do random forests use feature subsampling?

Answer. To decorrelate trees: the ensemble variance floor is $\rho\sigma^2$, so lower correlation ρ directly lowers ensemble variance.

Q6.3. Can random forests overfit? Can gradient boosting?

Answer. Adding trees to a random forest does not increase overfitting (the average converges), though individual deep trees on noisy data still give some overfit. Gradient boosting definitely overfits with too many trees or too high a learning rate — hence early stopping.

Q6.4. Why does XGBoost use the second derivative?

Answer. A second-order Taylor approximation gives closed-form optimal leaf weights and an exact gain formula for any twice-differentiable loss; it behaves like a Newton step, converging faster than first-order fitting of residuals.

Q6.5. In gradient boosting with MSE, what does each tree fit?

Answer. The current residuals $y - F_{m-1}(\mathbf{x})$, the negative gradient of $\frac{1}{2}(y - F)^2$.

Q6.6. Your XGBoost model has great AUC but its predicted probabilities seem off. Why?

Answer. Boosting with early stopping, class weighting or `scale_pos_weight`, and subsampling distort calibration. Recalibrate with isotonic or Platt scaling on a held-out set.

Q6.7. Why can't tree ensembles extrapolate?

Answer. Leaves predict averages of training targets, so predictions are bounded by the training target range. For a trending time series, model the trend separately (or difference the series) and let trees model the residual.

Q6.8. Which is faster to train: random forest or gradient boosting?

Answer. Random forest trees are independent and parallel across trees; boosting is sequential across trees (parallel only within a tree's split search). With histogram methods LightGBM is still often very fast in practice.

Chapter 7

Unsupervised Learning

7.1 k-means clustering

Intuition. Place k "centres" so that every point is close to its nearest centre. Alternate between assigning points to the nearest centre and moving each centre to the mean of its points.

Real-world example. A retailer clusters customers on recency, frequency and monetary value (RFM). Clusters come out as "loyal high spenders", "bargain hunters", "lapsed customers". Marketing then gives each segment a different campaign. Scaling matters: unscaled, the rupee-spend column dominates every distance.

Objective (within-cluster sum of squares, inertia):

$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2.$$

Lloyd's algorithm: (1) initialise centres; (2) assignment step: each point to its nearest centre; (3) update step: $\boldsymbol{\mu}_j$ = mean of its points; repeat until assignments stop changing.

Why it converges: each step cannot increase J — the assignment step picks the nearest centre for each point; the update step uses the mean, which minimises $\sum \|\mathbf{x}_i - \boldsymbol{\mu}\|^2$ (set gradient $-2\sum(\mathbf{x}_i - \boldsymbol{\mu}) = 0$). There are finitely many assignments, so it terminates — but at a *local* minimum. Cost per iteration $O(nkd)$.

k-means++ initialisation: pick the first centre at random; pick each next centre with probability proportional to $D(\mathbf{x})^2$ (squared distance to the nearest chosen centre). Spreads centres out; $O(\log k)$-competitive in expectation. Also run multiple restarts (`n_init`) and keep the best J.

Choosing k:

- **Elbow method**: plot J vs k, look for the bend. Often ambiguous.
- **Silhouette** for point i: $s = \frac{b-a}{\max(a,b)}$, a = mean distance to own cluster, b = mean distance to nearest other cluster. Range $[-1, 1]$; average over points; higher is better.
- Gap statistic, Davies–Bouldin, Calinski–Harabasz; and, most important, business interpretability.

Limitations: assumes roughly spherical, similar-sized clusters (it is a special case of a GMM with equal spherical covariances and hard assignments); sensitive to outliers and scale; must choose k; Euclidean only. Variants: k-medoids/PAM (centres are actual points, any distance, robust to outliers), mini-batch k-means (large data), k-modes (categorical).

7.2 Hierarchical clustering

Agglomerative: start with each point as its own cluster; repeatedly merge the closest pair. Linkage defines "closest":

- Single (min distance): finds elongated clusters, prone to chaining.
- Complete (max distance): compact clusters.
- Average: compromise.
- **Ward**: merge that least increases total within-cluster variance; usually the best default.

Produces a dendrogram; cut at a height to get clusters — no need to choose k upfront. Cost $O(n^2)$ memory and $O(n^2 \log n)$ to $O(n^3)$ time: unsuitable for very large n.

7.3 DBSCAN

Density-based. Parameters: radius ε, `minPts`. A **core point** has $\geq$ `minPts` neighbours within ε; points reachable from core points form a cluster; the rest are **noise**. Finds arbitrarily shaped clusters, no k needed, labels outliers. Struggles when clusters have very different densities (HDBSCAN fixes this) and in high dimensions. Pick ε from the "knee" of the sorted k-distance plot.

7.4 Gaussian mixture models and EM

Intuition. Each cluster is a Gaussian "blob" with its own shape. Points get *soft* memberships (probabilities). EM alternates: given blobs, compute how much each point belongs to each blob; given memberships, re-fit the blobs.

Model: $p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \, N(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, with $\sum \pi_k = 1$.
E-step (responsibilities):

$$\gamma_{ik} = \frac{\pi_k N(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j N(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

M-step (with $N_k = \sum_i \gamma_{ik}$):

$$\pi_k = \frac{N_k}{n}, \quad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i \gamma_{ik} \mathbf{x}_i, \quad \boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_i \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top.$$

Why EM works (sketch). With latent z, for any distribution $q(z)$, Jensen's inequality gives the lower bound

$$\log p(\mathbf{x} \mid \boldsymbol{\theta}) \geq \mathbb{E}_q[\log p(\mathbf{x}, z \mid \boldsymbol{\theta})] - \mathbb{E}_q[\log q(z)] = \text{ELBO},$$

with equality when $q(z) = p(z \mid \mathbf{x}, \boldsymbol{\theta})$. E-step sets q to that posterior (tightening the bound); M-step maximises the bound over $\boldsymbol{\theta}$. Each iteration therefore never decreases the likelihood. It converges to a local optimum; covariances can collapse onto a single point (likelihood $\to \infty$), so add a small diagonal regulariser. Choose K by BIC.

k-means vs GMM: k-means = hard assignments, spherical equal clusters; GMM = soft assignments, elliptical clusters with different sizes, gives a density (usable for anomaly detection: low $p(\mathbf{x})$ = anomaly).

7.5 Principal Component Analysis

Intuition. Rotate the coordinate system so that the first axis points along the direction where the data is most spread out, the second along the next most spread-out direction orthogonal to the first, and so on. Keep the first few axes; drop the rest as noise.

7.5.1 Derivation

Centre the data ($\mathbf{X}$ with column means zero). Sample covariance $\mathbf{\Sigma} = \frac{1}{n-1}\mathbf{X}^\top\mathbf{X}$. Projected variance along unit $\mathbf{u}$ is $\mathbf{u}^\top\mathbf{\Sigma}\mathbf{u}$. Maximising subject to $\|\mathbf{u}\| = 1$ (Lagrangian in Chapter 1) gives $\mathbf{\Sigma}\mathbf{u} = \lambda\mathbf{u}$: the principal components are eigenvectors of the covariance, and the variance captured by each equals its eigenvalue. Equivalently, PCA minimises the squared reconstruction error of projecting onto a k-dimensional subspace.

Via SVD (how it is computed in practice): $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Then $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top$, so the columns of $\mathbf{V}$ are principal directions, $\lambda_i = \sigma_i^2/(n-1)$, and the scores (projected data) are $\mathbf{X}\mathbf{V}_k = \mathbf{U}_k\mathbf{\Sigma}_k$. SVD avoids forming $\mathbf{X}^\top\mathbf{X}$, which squares the condition number.

Explained variance ratio $= \lambda_i / \sum_j \lambda_j$. Choose k to retain, say, 90–95%, or use the scree-plot elbow.

7.5.2 Practical points

- **Standardise first** when features have different units; otherwise the feature with the largest variance (e.g. salary in rupees) dominates PC1.
- PCA is linear and unsupervised: the direction of maximum variance may be useless for predicting y (use PLS or LDA for supervised reduction).
- Components are less interpretable than original features; loadings tell you which features drive each component.
- Sensitive to outliers (robust PCA exists).
- **Kernel PCA** applies the kernel trick for non-linear structure.
- Uses: compressing correlated features, removing multicollinearity (principal-component regression), visualisation, denoising, speeding up downstream models.

Real-world example. Yield curves: daily changes across 10 maturities of government bonds are highly correlated. PCA on them famously gives three components explaining the vast majority of variance — interpreted as level (all rates move together), slope (short vs long end) and curvature. Fixed-income risk teams hedge these factors rather than ten separate rates.

7.6 Non-linear embeddings for visualisation

t-SNE: converts high-dimensional distances into neighbour probabilities (Gaussian), low-dimensional ones into a heavy-tailed Student-t distribution, and minimises $D_{\mathrm{KL}}(P\|Q)$. The heavy tails let dissimilar points spread apart (reduces crowding). Preserves local neighbourhoods. Caveats: cluster sizes and inter-cluster distances in the plot are not meaningful; results depend on perplexity (roughly the effective number of neighbours, 5–50) and the random seed; it has no transform for new points; slow ($O(n \log n)$ with Barnes–Hut).

UMAP: builds a fuzzy k-NN graph and optimises a low-dimensional layout with a cross-entropy objective. Faster than t-SNE, preserves more global structure, supports transforming new points. Parameters: `n_neighbors`, `min_dist`.

Neither should be used as input features for downstream models without care; they are mainly for visualisation.

7.7 Anomaly detection

- **Statistical**: z-score > 3, IQR rule (below $Q_1 - 1.5\,\text{IQR}$ or above $Q_3 + 1.5\,\text{IQR}$), Mahalanobis distance for multivariate data.
- **Isolation Forest**: randomly split features at random thresholds; anomalies are isolated in fewer splits (shorter average path length). Fast, works in high dimensions.
- **Local Outlier Factor**: compares a point's local density with its neighbours'; good for anomalies in varying-density data.
- **One-class SVM**: learns a boundary around normal data.
- **Autoencoders**: train on normal data; high reconstruction error flags anomalies.
- Density models (GMM, KDE): low likelihood = anomaly.

Evaluation is hard without labels; if a few labels exist, use precision@k (the review team can investigate only k alerts a day).

7.8 Association rules (market basket analysis)

For rule $A \Rightarrow B$:

- Support $= P(A \cap B)$ (fraction of baskets containing both).
- Confidence $= P(B \mid A) = \text{supp}(A \cup B)/\text{supp}(A)$.
- **Lift** $= \dfrac{P(B \mid A)}{P(B)}$. Lift > 1: positive association; $= 1$: independent. Confidence alone misleads when B is very common (milk is in most baskets).

Apriori: any subset of a frequent itemset must be frequent, so candidates are pruned level by level. FP-Growth avoids candidate generation with a prefix tree.

Example 7.1. 1000 baskets: 200 contain bread, 150 contain butter, 100 contain both. Support $= 0.10$; confidence(bread⇒butter) $= 100/200 = 0.5$; lift $= 0.5/0.15 \approx 3.33$.

7.9 Interview questions

Q7.1. Does k-means always converge to the same answer?

Answer. It always converges (finitely many partitions, monotone objective) but to a local minimum that depends on initialisation. Use k-means++ and multiple restarts.

Q7.2. Why must you scale data before k-means or PCA?

Answer. Both use Euclidean geometry / variance, so large-scale features dominate distances and variance purely because of their units.

Q7.3. How do you evaluate clustering without labels?

Answer. Internal metrics (silhouette, Davies–Bouldin), stability across resamples, and above all whether clusters are actionable and interpretable for the business. With labels: adjusted Rand index, normalised mutual information.

Q7.4. PCA reduced 100 features to 10 with 95% variance retained, but model accuracy dropped. Why?

Answer. Variance is not predictive power: a low-variance direction can carry the signal. Try supervised selection, PLS, or keep the original features with regularisation.

Q7.5. Are principal components correlated?

Answer. No: scores of different components are uncorrelated by construction (orthogonal eigenvectors of the covariance).

Q7.6. k-means vs DBSCAN vs GMM — when to use which?

Answer. k-means: large data, roughly spherical clusters, known k. DBSCAN: arbitrary shapes, outliers present, unknown k, similar densities. GMM: elliptical overlapping clusters, need soft membership probabilities or a density.

Chapter 8

Time Series, Recommenders and Classical NLP

8.1 Time series

Intuition. In time series, the order of observations carries information and the past predicts the future. That breaks the i.i.d. assumption behind ordinary cross-validation, so both modelling and validation have to respect time.

8.1.1 Components and stationarity

A series = trend + seasonality + cycle + noise; decomposed additively ($y = T + S + R$) or multiplicatively ($y = T \times S \times R$, take logs to make it additive). STL is a robust decomposition.

(Weak) stationarity: constant mean, constant variance, autocovariance depends only on the lag. Most classical models (ARMA) assume it. Make a series stationary by differencing ($y_t - y_{t-1}$), seasonal differencing ($y_t - y_{t-12}$), log or Box–Cox transforms.

Tests: **ADF** (null: unit root, i.e. non-stationary — small p-value suggests stationarity); **KPSS** (null: stationary). Using both guards against a test's low power.

Real-world example. A stock price is (approximately) a random walk $P_t = P_{t-1} + \varepsilon_t$: non-stationary. Its returns are close to stationary. Regressing one random-walk price series on another often gives a high R^2 with no real relationship (**spurious regression**) — which is why analysts work with returns, or test for cointegration (the basis of pairs trading).

8.1.2 ACF and PACF

ACF at lag k: correlation of y_t with y_{t-k}. PACF at lag k: correlation after removing the effect of lags $1..k-1$.

- AR(p): PACF cuts off after lag p, ACF decays gradually.
- MA(q): ACF cuts off after lag q, PACF decays.
- Seasonal spikes at lags 12, 24 (monthly) or 7, 14 (daily data with weekly pattern).

8.1.3 Classical models

- **AR(p)**: $y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + \varepsilon_t$. AR(1) is stationary iff $|\phi_1| < 1$.
- **MA(q)**: $y_t = \mu + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j}$.

- **ARIMA**(p, d, q): ARMA on the d-times differenced series. **SARIMA**$(p, d, q)(P, D, Q)_s$ adds seasonal terms. **ARIMAX/SARIMAX** adds exogenous regressors (promotions, holidays). Select orders by ACF/PACF and AIC (auto-ARIMA).
- **Exponential smoothing**: simple ($\hat{y}_{t+1} = \alpha y_t + (1-\alpha)\hat{y}_t$, weights decay geometrically), Holt (adds trend), Holt–Winters (adds seasonality). ETS is the state-space formulation.
- **GARCH**: models time-varying volatility $\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2$ — volatility clustering in returns, used in VaR.
- **Prophet**: additive trend (piecewise linear/logistic) + Fourier seasonality + holidays; easy for business series with strong seasonality, not consistently more accurate than tuned alternatives.

8.1.4 Machine learning for forecasting

Turn the series into a supervised table: lag features (y_{t-1}, y_{t-7}), rolling means/stds, date features, known-future covariates (planned promotions, holidays), then fit gradient boosting. For many related series (thousands of SKUs) a single global model across series usually beats one model per series. Deep options: LSTMs, Temporal Convolutional Networks, N-BEATS, Temporal Fusion Transformer.

Pitfalls: computing rolling features with a window that includes the target day (leakage); for multi-step forecasts, either recursive (feed predictions back — errors compound) or direct (one model per horizon).

8.1.5 Validation and metrics

Use **walk-forward / rolling-origin** validation: train up to time t, forecast $t+1..t+h$, roll forward. Compare to naive baselines: last value, or seasonal naive (same day last week). **MASE** scales error by the in-sample naive forecast's error — below 1 means you beat naive. Use WAPE/sMAPE when MAPE breaks on zeros.

8.2 Recommender systems

Real-world example. On a streaming app, "because you watched X" rows come from item similarity, while the home page ranking blends predicted watch probability, freshness and diversity. Most large systems use a two-stage pipeline: *candidate generation* (retrieve a few hundred items from millions, fast) then *ranking* (a heavier model scores those few hundred with rich features).

8.2.1 Approaches

- **Popularity baseline**: strong and often embarrassingly hard to beat.
- **Content-based**: recommend items similar to those the user liked, using item features (genre, text embeddings). Handles new items; tends to over-specialise.
- **Collaborative filtering (CF)**: "users like you liked…". User–user or item–item similarity (cosine, Pearson on the rating matrix). Item–item is more stable (items change slower than users) and was popularised by Amazon.
- **Matrix factorisation**: $R \approx \mathbf{P}\mathbf{Q}^\top$, predict $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$; minimise squared error on *observed* entries with L_2 regularisation via SGD or alternating least squares (ALS: fix $\mathbf{Q}$ and solve ridge regressions for $\mathbf{P}$, then swap; parallelises well).

- **Implicit feedback** (clicks, views, no explicit ratings): treat missing as weak negatives with confidence weights (implicit ALS), or optimise ranking directly (BPR loss: score a consumed item above a sampled unconsumed one).
- **Neural**: two-tower models (user tower and item tower produce embeddings; dot product for retrieval with ANN search), wide-and-deep, DLRM, sequence models (SASRec, BERT4Rec) for "next item".

8.2.2 Problems and metrics

Cold start: new users (use popularity, onboarding questions, demographics, context) and new items (content features). **Sparsity**, **popularity bias**, **feedback loops** (the system only learns about what it shows — add exploration, e.g. bandits). Metrics: offline precision/recall@k, NDCG, MAP, hit rate, coverage, diversity, novelty; online CTR, watch time, retention via A/B tests. Offline gains often fail to translate online.

8.3 Classical NLP

- **Preprocessing**: tokenisation, lowercasing, stop-word removal, stemming (crude suffix chopping: "studies"→"studi") vs lemmatisation (dictionary form: "studies"→"study").
- **Bag of words**: count vector over the vocabulary; ignores order. n-grams capture short phrases ("not good").
- **TF-IDF**: $\mathrm{tf}(t,d) \times \log \frac{N}{\mathrm{df}(t)}$. Down-weights words that appear in every document. With logistic regression or linear SVM, a strong baseline for text classification.
- **Word2Vec**: dense embeddings learned by predicting context words (skip-gram) or the centre word from context (CBOW); trained with negative sampling. Captures analogies (king − man + woman ≈ queen). **GloVe**: factorises global co-occurrence statistics. **FastText**: sub-word n-grams, handles misspellings and unseen words.
- Static embeddings give one vector per word regardless of context ("bank" of a river vs a bank account); contextual models (ELMo, BERT) fix this — Chapter 12.
- Tasks: classification, named-entity recognition (sequence labelling; historically CRFs), topic modelling (LDA: documents are mixtures of topics, topics are distributions over words), sentiment analysis, similarity/search (BM25 for keyword search).

8.4 Interview questions

Q8.1. Why can't you use random k-fold CV for time series?

Answer. Shuffling lets the model train on the future and test on the past, leaking information and inflating scores. Use walk-forward validation with a gap if features use lookback windows.

Q8.2. How do you make a series stationary?

Answer. Remove trend by differencing or detrending; remove seasonality by seasonal differencing; stabilise variance with log/Box–Cox. Confirm with ADF/KPSS and ACF plots.

Q8.3. Explain the cold-start problem and solutions.

Answer. New users/items have no interaction history, so CF cannot place them. Use content features, popularity, contextual signals, onboarding preferences, hybrid models, and exploration.

Q8.4. TF-IDF vs word embeddings?

Answer. TF-IDF is sparse, interpretable, ignores semantics and order, strong for keyword-driven tasks. Embeddings are dense, capture semantic similarity, generalise to synonyms; contextual embeddings capture meaning in context.

Q8.5. A demand-forecasting model predicts well on average but badly on festival weeks. What do you do?

Answer. Add holiday/event features with lead and lag windows, model the event uplift separately, use known future covariates, weight recent festival periods, and evaluate specifically on those periods.

Part III

Deep Learning

Chapter 9

Neural Networks and How to Train Them

9.1 From the perceptron to multilayer networks

Intuition. A single neuron is logistic regression: weighted sum plus a squashing function. Stacking layers lets the network build features out of features — edges into shapes into objects, or characters into words into meaning — instead of a human hand-crafting them.

Perceptron (Rosenblatt): $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$, update $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \mathbf{x}_i$ on mistakes. Converges if data is linearly separable; cannot learn XOR. That limitation is why we need hidden layers and non-linear activations.

MLP forward pass for layer $l = 1..L$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = g^{(l)}(\mathbf{z}^{(l)}), \qquad \mathbf{a}^{(0)} = \mathbf{x}.$$

Why non-linearity? Without g, the composition of linear layers $\mathbf{W}_2 \mathbf{W}_1 \mathbf{x}$ is a single linear map — depth adds nothing.

Universal approximation theorem. A network with one hidden layer and enough units (with a non-polynomial activation) can approximate any continuous function on a compact set arbitrarily well. It says nothing about how many units are needed, or whether gradient descent will find the weights. In practice, depth gives exponentially more efficient representations of compositional functions than width.

Parameter count of a dense layer: $n_{in} \times n_{out} + n_{out}$. E.g. $784 \to 256 \to 10$: $(784 \cdot 256 + 256) + (256 \cdot 10 + 10) = 200{,}960 + 2{,}570 = 203{,}530$.

9.2 Backpropagation

Intuition. Backprop is the chain rule applied efficiently. Going forward we compute and cache every intermediate value; going backward we pass "how much does the loss change if this value changes?" from the output to the input, reusing work so that one backward pass costs about the same as one or two forward passes.

9.2.1 Derivation in matrix form

Define the error signal $\boldsymbol{\delta}^{(l)} = \partial L / \partial \mathbf{z}^{(l)}$. Then:

$$\begin{aligned}
\boldsymbol{\delta}^{(L)} &= \nabla_{\mathbf{a}^{(L)}} L \odot g'(\mathbf{z}^{(L)}) \quad \text{(for softmax + cross-entropy: } \boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}) \\
\boldsymbol{\delta}^{(l)} &= \left(\mathbf{W}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}\right) \odot g'(\mathbf{z}^{(l)}) \\
\frac{\partial L}{\partial \mathbf{W}^{(l)}} &= \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}.
\end{aligned}$$

For a mini-batch with rows as samples ($\mathbf{A}^{(l-1)} \in \mathbb{R}^{B \times n_{l-1}}$, $\boldsymbol{\Delta}^{(l)} \in \mathbb{R}^{B \times n_l}$): $\partial L / \partial \mathbf{W}^{(l)} = \boldsymbol{\Delta}^{(l)\top} \mathbf{A}^{(l-1)}$ (averaged over the batch).

Shape check trick (works in interviews): the gradient of a weight matrix always has the same shape as the matrix, so arrange the factors until shapes match.

9.2.2 Why softmax with cross-entropy gives prediction minus target

$L = -\sum_k y_k \log p_k$, $p_k = e^{z_k} / \sum_j e^{z_j}$. Using $\partial p_k / \partial z_i = p_k(\delta_{ki} - p_i)$:

$$\frac{\partial L}{\partial z_i} = -\sum_k y_k \frac{1}{p_k} p_k(\delta_{ki} - p_i) = -y_i + p_i \sum_k y_k = p_i - y_i.$$

9.2.3 A tiny worked example

One sigmoid neuron, $\mathbf{x} = (1, 2)$, $\mathbf{w} = (0.5, -0.3)$, $b = 0.1$, label $y = 1$, binary cross-entropy, learning rate 0.1. Forward: $z = 0.5 - 0.6 + 0.1 = 0$, $p = \sigma(0) = 0.5$, $L = -\log 0.5 = 0.693$. Backward: $\partial L / \partial z = p - y = -0.5$; $\partial L / \partial \mathbf{w} = -0.5\,\mathbf{x} = (-0.5, -1.0)$; $\partial L / \partial b = -0.5$. Update: $\mathbf{w} = (0.55, -0.2)$, $b = 0.15$. New $z = 0.55 - 0.4 + 0.15 = 0.30$, $p = 0.574$: the loss fell.

9.2.4 Automatic differentiation

Frameworks (PyTorch, JAX, TensorFlow) build a computational graph and run **reverse-mode** autodiff: one backward pass gives the gradient of a scalar loss w.r.t. all parameters. Forward-mode is efficient when there are few inputs and many outputs. Memory cost: activations must be stored for the backward pass; **gradient checkpointing** recomputes some of them to trade compute for memory. Numerical gradient checking: compare with $\frac{L(\theta+\epsilon) - L(\theta-\epsilon)}{2\epsilon}$.

9.3 Activation functions

Name	Formula	Notes
Sigmoid	$1/(1+e^{-z})$	Output $(0,1)$; saturates; max gradient $0.25 \Rightarrow$ vanishing gradients in deep nets; not zero-centred. Use for binary outputs and gates.
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	Zero-centred, $(-1,1)$; still saturates. Used in RNN/LSTM cells.
ReLU	$\max(0, z)$	Cheap, no saturation for $z > 0$, sparse activations. "Dying ReLU": units stuck at $z < 0$ get zero gradient forever (often from high learning rates).
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \approx 0.01$	Fixes dying ReLU. PReLU learns α.

ELU / SELU	z if $z > 0$ else $\alpha(e^z - 1)$	Smooth, negative outputs push mean toward zero; SELU self-normalises.
GELU	$z\,\Phi(z)$	Smooth ReLU-like; default in BERT/GPT-style Transformers.
SiLU / Swish	$z\,\sigma(z)$	Smooth, non-monotone; SwiGLU (gated variant) used in many modern LLM feed-forward layers.
Softmax	$e^{z_i} / \sum_j e^{z_j}$	Multi-class output distribution.

Output-layer rule: regression → linear (no activation); binary → sigmoid + BCE; multi-class single-label → softmax + CE; multi-label → one sigmoid per label + BCE.

9.4 Vanishing and exploding gradients

Backprop multiplies many Jacobians: $\boldsymbol{\delta}^{(1)} \propto \prod_l \mathbf{W}^{(l)\top} \operatorname{diag}(g'(\mathbf{z}^{(l)}))$. If these factors are mostly < 1 in norm the gradient shrinks exponentially with depth (vanishing); if > 1 it blows up (exploding).

Remedies: ReLU-family activations; careful initialisation; normalisation layers; residual connections (identity path lets gradients flow: $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x} + F(\mathbf{x})) = \mathbf{I} + F'$); gradient clipping (for exploding, esp. RNNs); LSTM/GRU gating; smaller learning rates/warm-up.

9.5 Weight initialisation

All-zero (or identical) weights make every neuron in a layer compute the same thing and receive the same gradient — symmetry is never broken. Use small random values, scaled so signal variance is preserved across layers.

Xavier/Glorot derivation (sketch). For $z = \sum_{i=1}^{n_{in}} w_i x_i$ with independent zero-mean w, x: $\operatorname{Var}(z) = n_{in} \operatorname{Var}(w) \operatorname{Var}(x)$. To keep $\operatorname{Var}(z) = \operatorname{Var}(x)$ forward we need $\operatorname{Var}(w) = 1/n_{in}$; backward we need $1/n_{out}$; the compromise is

$$\operatorname{Var}(w) = \frac{2}{n_{in} + n_{out}} \quad \text{(tanh/sigmoid)}.$$

He/Kaiming: ReLU zeros half its inputs, halving the variance, so double it: $\operatorname{Var}(w) = 2/n_{in}$. Use for ReLU-family networks. Biases are usually initialised to zero.

9.6 Optimisers

Let $g_t = \nabla L(\boldsymbol{\theta}_t)$ on a mini-batch.

- **SGD**: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta g_t$.
- **Momentum**: $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + g_t$, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v}_t$ ($\beta \approx 0.9$). Accumulates velocity in consistent directions, damps oscillation across ravines. **Nesterov**: evaluate the gradient at the look-ahead point.
- **AdaGrad**: divide by $\sqrt{\sum_{\tau \le t} g_\tau^2}$ per parameter — good for sparse features, but the learning rate decays to zero.
- **RMSProp**: replace the sum by an exponential moving average $s_t = \rho s_{t-1} + (1-\rho) g_t^2$; step $\eta g_t / (\sqrt{s_t} + \epsilon)$.

- **Adam**: momentum + RMSProp with bias correction:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

  Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. **Why bias correction:** $m_0 = 0$, and unrolling gives $m_t = (1-\beta_1)\sum_{i=1}^{t} \beta_1^{t-i} g_i$, so if $\mathbb{E}g_i = \mathbb{E}g$, then $\mathbb{E}m_t = \mathbb{E}g\,(1-\beta_1^t)$. Early on m_t is biased toward zero; dividing by $1-\beta_1^t$ removes this.

- **AdamW**: decoupled weight decay $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon) + \lambda\boldsymbol{\theta})$. In Adam, adding L_2 to the loss is *not* equivalent to weight decay because the penalty gradient gets rescaled by $1/\sqrt{\hat{v}}$. AdamW is the default for Transformers.

Folklore that is partly true: SGD with momentum sometimes generalises better than Adam on vision tasks; Adam(W) converges faster and is far less sensitive to tuning, dominating in NLP.

9.6.1 Learning-rate schedules

Step decay, exponential decay, cosine annealing, one-cycle, reduce-on-plateau. **Warm-up**: start with a tiny learning rate and ramp up over the first few hundred or thousand steps — stabilises Adam's early variance estimates and large-batch training; standard for Transformers (linear warm-up + cosine or inverse-sqrt decay). The learning rate is the single most important hyperparameter; an LR range test (increase LR exponentially, watch loss) finds a good range quickly.

9.6.2 Batch size

Larger batches: less gradient noise, better hardware utilisation, fewer steps; may generalise slightly worse and need larger learning rates (linear scaling rule: scale LR with batch size, with warm-up). Smaller batches: noisier updates that act as regularisation. Gradient accumulation simulates large batches with limited memory.

9.7 Normalisation

Batch normalisation (per feature, across the batch):

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma\hat{x} + \beta.$$

γ, β are learned so the network can undo the normalisation if useful. During **training** it uses batch statistics; it keeps running averages of μ, σ^2 and uses those at **inference** (forgetting `model.eval()` in PyTorch is a classic bug). Benefits: allows higher learning rates, smoother loss landscape, mild regularisation from batch noise. The original "internal covariate shift" explanation is disputed; smoothing of the optimisation landscape is the more accepted account. Weaknesses: depends on batch size (bad for tiny batches), awkward for variable-length sequences.

Layer normalisation: normalise across features within each sample — independent of batch size; standard in Transformers and RNNs. **RMSNorm**: divides by root-mean-square without subtracting the mean; cheaper, used in LLaMA-family models. **Group/Instance norm**: vision with small batches; style transfer.

Pre-LN vs post-LN Transformers: putting LayerNorm *before* each sub-layer (pre-LN) trains more stably for deep models.

9.8 Regularisation in deep learning

- **Weight decay** (L_2).
- **Dropout**: during training zero each unit with probability p and scale survivors by $1/(1-p)$ ("inverted dropout") so that expected activations match inference, where dropout is off. Interpretations: trains an implicit ensemble of sub-networks; prevents co-adaptation. Typical $p = 0.1$–0.5.
- **Early stopping** on validation loss.
- **Data augmentation**: flips, crops, colour jitter, mixup/CutMix (images); back-translation, synonym replacement (text); noise, time-warping (audio/time series).
- **Label smoothing**: target $1-\epsilon$ for the true class and $\epsilon/(K-1)$ for the rest; prevents over-confidence, improves calibration.
- **Gradient clipping** by global norm: if $\|g\| > c$, $g \leftarrow cg/\|g\|$.
- Stochastic depth, transfer learning from pre-trained weights (often the strongest regulariser when data is small).

9.9 Training at scale (ML engineer topics)

- **Mixed precision**: compute in FP16/BF16, keep a master copy of weights in FP32; FP16 needs *loss scaling* to keep small gradients from underflowing; BF16 has FP32's exponent range so usually doesn't.
- **Data parallelism**: replicate the model, split the batch, all-reduce gradients (PyTorch DDP). **ZeRO/FSDP** shard optimiser states, gradients and parameters across GPUs to save memory.
- **Model parallelism**: tensor parallelism (split a layer's matrices), pipeline parallelism (split layers across devices, micro-batches to reduce idle "bubbles").
- **Memory budget for Adam training**, roughly per parameter: weights + gradients + two Adam moments, plus activations. With mixed precision a common estimate is around 16 bytes per parameter before activations, so a 7B-parameter model needs on the order of 100+ GB for full fine-tuning — the motivation for LoRA and quantisation.

9.10 Debugging a network that won't learn

1. Overfit a single small batch first. If loss doesn't go near zero, there is a bug.
2. Check the initial loss: for K balanced classes it should be about $\log K$ (e.g. 2.30 for 10 classes).
3. Check data: labels aligned with inputs? normalisation applied? augmentation sane? Visualise a batch.
4. Learning rate too high (loss explodes, NaNs) or too low (flat).
5. Train/eval mode (dropout, BatchNorm), forgotten `optimizer.zero_grad()` (gradients accumulate), wrong loss for the output (softmax applied twice: `CrossEntropyLoss` in PyTorch expects raw logits).
6. Monitor gradient norms per layer; look for vanishing/exploding or dead ReLUs.

9.11 Interview questions

Q9.1. Why don't we initialise all weights to zero?

Answer. Symmetry: all neurons in a layer get identical gradients and stay identical, so the layer behaves like a single neuron.

Q9.2. What is the derivative of ReLU at 0?

Answer. Undefined mathematically; frameworks use a subgradient, typically 0.

Q9.3. How does dropout behave at test time?

Answer. It's turned off. With inverted dropout, scaling happened during training, so no change is needed at inference.

Q9.4. BatchNorm vs LayerNorm?

Answer. BN normalises each feature across the batch, needs running stats for inference, and depends on batch size. LN normalises across features within each sample, same computation at train and test, works with batch size 1 and variable-length sequences.

Q9.5. Why is Adam's bias correction needed?

Answer. Moment estimates start at zero, so early estimates are biased toward zero; dividing by $1 - \beta^t$ makes them unbiased.

Q9.6. Loss becomes NaN after a few hundred steps. Causes?

Answer. Learning rate too high, exploding gradients, log(0) or division by zero (compute log-softmax stably, add ϵ), FP16 overflow, bad data (NaN or inf in inputs).

Q9.7. Training loss decreases but validation loss rises from epoch 5. What do you do?

Answer. Overfitting: early stopping around epoch 5, more regularisation (dropout, weight decay, augmentation), more data, smaller model, or a pre-trained backbone.

Q9.8. How many parameters does a dense layer from 512 to 1024 units have?

Answer. $512 \times 1024 + 1024 = 525{,}312$.

Q9.9. Why are residual connections helpful?

Answer. They add an identity path, so gradients flow back unattenuated and each block only has to learn a correction $F(\mathbf{x})$ to the identity; very deep networks become trainable.

Chapter 10

Convolutional Neural Networks and Computer Vision

10.1 Why convolutions?

Intuition. A 224×224 RGB image has 150,528 inputs. A dense layer with 1000 units would need 150 million weights for one layer, and would have to learn "cat ear" separately at every location. A convolution slides one small filter (say 3×3) across the image: the same "ear detector" is reused everywhere.

Three built-in assumptions (inductive biases):

- **Local connectivity**: a unit looks at a small neighbourhood.
- **Weight sharing**: the same filter at every position — far fewer parameters.
- **Translation equivariance**: shifting the input shifts the feature map. Pooling adds a degree of local translation *invariance*.

10.2 The convolution layer

Deep-learning "convolution" is technically cross-correlation (no kernel flip): for a single channel, $Y[i,j] = \sum_{m,n} X[i+m, j+n]K[m,n] + b$. A layer with C_{in} input channels and C_{out} filters has filters of shape $K \times K \times C_{in}$; each filter produces one output channel.

10.2.1 Output size and parameter count (asked in almost every OA)

$$O = \Big\lfloor \frac{W - K + 2P}{S} \Big\rfloor + 1$$

for input width W, kernel K, padding P, stride S. "Same" padding with stride 1 uses $P = (K-1)/2$.

$$\text{Params} = K \times K \times C_{in} \times C_{out} + C_{out} \text{ (biases)}.$$

Multiply–accumulate operations (FLOPs, roughly) $\approx O_h \times O_w \times K^2 \times C_{in} \times C_{out}$.

Example 10.1. Input $224 \times 224 \times 3$, 64 filters of 7×7, stride 2, padding 3 (ResNet stem). $O = \lfloor (224-7+6)/2 \rfloor + 1 = 111 + 1 = 112$. Output $112 \times 112 \times 64$. Params $= 7 \cdot 7 \cdot 3 \cdot 64 + 64 = 9{,}472$. Then 3×3 max-pool stride 2 padding 1: $\lfloor (112 - 3 + 2)/2 \rfloor + 1 = 56$.

Example 10.2. Input $32 \times 32 \times 3$; conv 5×5, 6 filters, no padding, stride $1 \to 28 \times 28 \times 6$ (params 456); max-pool 2×2 stride $2 \to 14 \times 14 \times 6$ (0 params); conv 5×5, 16 filters $\to 10 \times 10 \times 16$ (params $5 \cdot 5 \cdot 6 \cdot 16 + 16 = 2{,}416$). This is LeNet-5's front end.

10.2.2 Other ingredients

- **Stride** > 1 downsamples. **Dilation** d spaces kernel taps apart: effective size $K+(K-1)(d-1)$ — larger receptive field without more parameters (used in segmentation, WaveNet).
- **Pooling**: max-pool keeps the strongest activation; average-pool smooths; no parameters. **Global average pooling** collapses $H \times W \times C$ to C, replacing huge flatten+dense layers.
- 1×1 **convolution**: a per-pixel dense layer across channels — changes the number of channels cheaply (bottlenecks), adds non-linearity.
- **Depthwise separable convolution** (MobileNet): a $K \times K$ depthwise conv per channel, then a 1×1 pointwise conv. Params $K^2 C_{in} + C_{in} C_{out}$ instead of $K^2 C_{in} C_{out}$. For $K = 3$, $C_{in} = C_{out} = 256$: 67,840 vs 589,824, about 8.7× fewer.
- **Transposed convolution** ("deconvolution"): learnable upsampling for segmentation and generators; can cause checkerboard artefacts (resize + conv avoids them).

10.2.3 Receptive field

The region of the input that influences a unit. With layers of kernel k_l and stride s_l: $r_l = r_{l-1} + (k_l - 1) \prod_{i<l} s_i$, $r_0 = 1$. Two stacked 3×3 convs (stride 1) see 5×5; three see 7×7.

Why VGG stacks 3×3 convs: three 3×3 layers have the receptive field of one 7×7 but $3 \cdot 9C^2 = 27C^2$ parameters versus $49C^2$, plus three non-linearities instead of one.

10.3 Landmark architectures

Model	Key idea
LeNet-5 (1998)	Conv–pool–conv–pool–dense for digit recognition.
AlexNet (2012)	Won ImageNet by a wide margin: ReLU, dropout, data augmentation, GPU training. Started the deep-learning era in vision.
VGG (2014)	Uniform 3×3 convs, depth 16–19; simple but heavy (about 138M params, mostly in the dense layers).
GoogLeNet / Inception (2014)	Parallel 1×1, 3×3, 5×5 branches concatenated; 1×1 bottlenecks to cut cost.
ResNet (2015)	Residual blocks $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$; trained 152 layers. Bottleneck block $1 \times 1 \to 3 \times 3 \to 1 \times 1$. Solved the *degradation* problem (deeper plain nets had higher *training* error — an optimisation problem, not overfitting).
DenseNet (2017)	Each layer receives concatenated features of all previous layers; feature reuse.
MobileNet, ShuffleNet	Depthwise separable convs for phones/edge devices.
EfficientNet (2019)	Compound scaling of depth, width and resolution together.
Vision Transformer (2020)	Split image into 16×16 patches, embed, feed to a Transformer. Weaker inductive bias, so needs more data or pre-training; matches/exceeds CNNs at scale.
ConvNeXt (2022)	Modernised ResNet borrowing Transformer design choices; competitive with ViTs.

10.4 Transfer learning

Real-world example. A manufacturing firm wants to detect scratches on metal parts but has only 800 labelled images. Training from scratch would overfit. Instead, take a ResNet-50 pre-trained on ImageNet, replace the final layer with a 2-class head, train the head first, then unfreeze the last block with a small learning rate. Early layers (edges, textures) transfer well across domains.

Strategy by data size and similarity:

- Small data, similar domain: freeze backbone, train only the head (feature extraction).
- Small data, different domain: fine-tune the later layers carefully; strong augmentation.
- Large data: fine-tune the whole network with a lower learning rate for earlier layers (discriminative learning rates).

Remember to use the preprocessing (normalisation statistics, input size) the backbone was trained with.

10.5 Beyond classification

10.5.1 Object detection

- **IoU** (intersection over union) $= |A \cap B|/|A \cup B|$ measures box overlap.
- **Non-maximum suppression**: sort boxes by confidence; keep the top box; discard others with IoU above a threshold (e.g. 0.5) with it; repeat.
- **Two-stage** (R-CNN $\to$ Fast R-CNN $\to$ Faster R-CNN with a Region Proposal Network): propose regions, then classify and refine. Accurate, slower.
- **One-stage** (YOLO family, SSD, RetinaNet): predict boxes and classes densely in one pass. Fast. RetinaNet's **focal loss** $-(1-p_t)^\gamma \log p_t$ down-weights easy background examples to fix extreme foreground/background imbalance.
- Anchor boxes vs anchor-free detectors; DETR uses a Transformer with set-based (Hungarian matching) loss and no NMS.
- Metric: **mAP** — area under the precision–recall curve per class at an IoU threshold, averaged over classes (COCO also averages IoU thresholds 0.5:0.95).

10.5.2 Segmentation

Semantic (a class per pixel), instance (separate each object: Mask R-CNN), panoptic (both). **U-Net**: encoder–decoder with skip connections that pass high-resolution detail to the decoder; standard in medical imaging. Loss: pixel-wise cross-entropy, Dice loss $1 - \frac{2|A \cap B|}{|A|+|B|}$ for imbalanced masks.

10.6 Interview questions

Q10.1. Input $128 \times 128 \times 3$, conv with 32 filters 3×3, stride 1, padding 1. Output shape and parameters?

Answer. $128 \times 128 \times 32$; params $3 \cdot 3 \cdot 3 \cdot 32 + 32 = 896$.

Q10.2. Same input, conv 5×5, stride 2, no padding, 16 filters?

Answer. $\lfloor (128-5)/2 \rfloor + 1 = 61 + 1 = 62$, so $62 \times 62 \times 16$; params $5 \cdot 5 \cdot 3 \cdot 16 + 16 = 1{,}216$.

Q10.3. How many parameters in a max-pooling layer?

Answer. Zero.

Q10.4. Why do CNNs generalise better than MLPs on images?

Answer. Weight sharing and locality drastically reduce parameters and encode translation equivariance, matching the structure of images.

Q10.5. What problem did ResNet solve?

Answer. Degradation: deeper plain networks got higher training error because they were hard to optimise. Identity shortcuts let blocks default to the identity and keep gradients flowing.

Q10.6. Is a CNN translation invariant?

Answer. Convolution is equivariant; pooling and global average pooling add approximate invariance. Strided layers break exact equivariance (small shifts can change outputs), which is why augmentation still helps.

Q10.7. When would you prefer a ViT over a CNN?

Answer. With large datasets or strong pre-trained checkpoints, and when long-range global context matters. With small data and no pre-training, CNNs' inductive biases usually win.

Chapter 11

Recurrent Networks and Sequence Models

11.1 Recurrent neural networks

Intuition. Read a sentence one word at a time while keeping a running summary (the hidden state) in your head. The RNN reuses the same weights at every step and updates the summary with each new input.

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h), \qquad \hat{\mathbf{y}}_t = \operatorname{softmax}(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y).$$

Configurations: many-to-one (sentiment of a review), one-to-many (image captioning), many-to-many aligned (part-of-speech tagging), many-to-many unaligned via encoder–decoder (translation).

11.1.1 Backpropagation through time and vanishing gradients

Unroll the network over T steps and backprop through the unrolled graph. The gradient from step t to step $k < t$ contains

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_i^2\right)\mathbf{W}_{hh}.$$

The same matrix $\mathbf{W}_{hh}$ is multiplied $t - k$ times. If its largest singular value (times the tanh derivative, ≤ 1) is below 1, the product shrinks exponentially — the network cannot learn long-range dependencies; above 1, gradients explode. Exploding gradients are handled by gradient clipping; vanishing gradients motivated gated cells. **Truncated BPTT** backpropagates only through the last k steps to save compute.

11.2 LSTM

Intuition. Add a separate "conveyor belt" — the cell state $\mathbf{c}_t$ — that information can ride along with only small, gated edits. Gates are sigmoids (between 0 and 1) that decide what to forget, what to write and what to reveal.

$$
\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{forget gate} \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{input gate} \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{candidate} \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{cell update (additive)} \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) && \text{output gate} \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)
\end{aligned}
$$

Why it helps: $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \operatorname{diag}(\mathbf{f}_t)$ (ignoring indirect paths). When the forget gate is near 1, gradients pass through almost unchanged — similar in spirit to a residual connection. Initialising the forget-gate bias to around 1 is a common trick.

Parameter count: four gate blocks, each $\mathbf{W} \in \mathbb{R}^{h \times (h+x)}$ plus bias: $4\,(h(h+x)+h)$. For $x = 100$, $h = 128$: $4(128 \cdot 228 + 128) = 117{,}248$.

Real-world example. In a sentence like "The *customers* who complained about the delayed shipment last week *were* refunded", the verb must agree with "customers" many tokens earlier. A plain RNN struggles to carry that number information across the gap; an LSTM can keep it in the cell state.

11.3 GRU

Merges forget and input gates into an update gate $\mathbf{z}_t$ and drops the separate cell state:

$$
\begin{aligned}
&\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\
&\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.
\end{aligned}
$$

Three gate blocks instead of four: about 25% fewer parameters, faster, usually comparable accuracy.

11.4 Variants and practicalities

- **Bidirectional RNN**: one pass left-to-right, one right-to-left, concatenate. Needs the full sequence, so not for real-time next-step prediction.
- **Stacked (deep) RNNs**: hidden states of one layer feed the next.
- **Padding and masking**: batch variable-length sequences by padding; mask pads out of the loss and attention (PyTorch `pack_padded_sequence`).
- Dropout on non-recurrent connections, or variational dropout (same mask every time step) on recurrent ones.
- 1D convolutions and Temporal Convolutional Networks (causal, dilated) are parallel alternatives for sequences.

11.5 Sequence-to-sequence and attention

Encoder–decoder: the encoder reads the source into a context vector; the decoder generates the target token by token. Problem: squeezing a whole long sentence into one fixed vector is a **bottleneck**; quality drops for long inputs.

Attention (Bahdanau, 2014): at each decoding step, compute a weighted average of *all* encoder states, with weights based on relevance to the current decoder state:

$$e_{t,i} = \text{score}(\mathbf{s}_{t-1}, \mathbf{h}_i), \quad \alpha_{t,i} = \text{softmax}_i(e_{t,i}), \quad \mathbf{c}_t = \sum_i \alpha_{t,i}\mathbf{h}_i.$$

Additive score $\mathbf{v}^\top \tanh(\mathbf{W}_1\mathbf{s} + \mathbf{W}_2\mathbf{h})$ (Bahdanau) or multiplicative $\mathbf{s}^\top\mathbf{W}\mathbf{h}$ (Luong). Attention weights also give rough alignments (which source words the model looks at when producing each target word). The Transformer (next chapter) keeps attention and drops recurrence entirely.

Teacher forcing: during training, feed the ground-truth previous token to the decoder rather than its own prediction. Fast, stable training, but causes **exposure bias**: at inference the model sees its own (possibly wrong) outputs, a distribution it never trained on. Scheduled sampling mixes the two.

11.6 Interview questions

Q11.1. Why do vanilla RNNs struggle with long sequences?

Answer. Repeated multiplication by the same recurrent Jacobian makes gradients vanish (or explode) exponentially with distance, so long-range dependencies don't get learned.

Q11.2. How does LSTM solve vanishing gradients?

Answer. The cell state is updated additively and scaled by the forget gate, giving a near-identity gradient path when the gate is open. It mitigates rather than eliminates the problem.

Q11.3. LSTM vs GRU?

Answer. GRU has two gates and no separate cell state — fewer parameters, faster; LSTM has three gates and a cell state — sometimes better on longer or more complex dependencies. Empirically close; try both.

Q11.4. Why did Transformers replace RNNs?

Answer. RNNs process tokens sequentially (no parallelism across time) and still struggle with very long dependencies. Self-attention connects any two positions in one step and parallelises across the sequence on GPUs, enabling much larger models and datasets.

Q11.5. What is gradient clipping and why is it used in RNNs?

Answer. Rescaling the gradient when its norm exceeds a threshold; prevents exploding gradients from causing huge, destabilising updates.

Chapter 12

Transformers and Large Language Models

12.1 Self-attention

Intuition. Each word asks a question (*query*), advertises what it contains (*key*), and carries information to share (*value*). A word's new representation is a weighted average of everyone's values, weighted by how well its query matches their keys. In "The bank raised interest rates", the token "bank" attends strongly to "interest rates" and its representation shifts toward the financial meaning.

12.1.1 Scaled dot-product attention

Input $\mathbf{X} \in \mathbb{R}^{n \times d}$ (n tokens). Learned projections $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ with $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\Big)\mathbf{V}.$$

$\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times n}$ holds every pairwise score; softmax is row-wise; $\mathbf{M}$ is a mask ($-\infty$ where attention is forbidden).

Why divide by $\sqrt{d_k}$? If components of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean 0 and variance 1, then $\mathbf{q}^\top\mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k. Large-magnitude scores push softmax into saturation (one weight ≈ 1, the rest ≈ 0), where gradients are tiny. Dividing by $\sqrt{d_k}$ restores unit variance.

Masks.

- **Causal (look-ahead) mask**: token i may attend only to positions $\leq i$ — required for autoregressive generation so the model cannot peek at the answer.
- **Padding mask**: ignore pad tokens in batched variable-length inputs.

Self- vs cross-attention. In self-attention $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ come from the same sequence. In cross-attention (encoder–decoder), queries come from the decoder and keys/values from the encoder output.

12.1.2 Multi-head attention

Run h attention heads in parallel with $d_k = d_v = d/h$, concatenate, project:

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O.$$

Different heads can specialise (syntax, coreference, position). Cost is about the same as a single full-width head. Parameters: $4d^2$ (+ biases) for $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$.

Multi-query (MQA) and grouped-query attention (GQA): share keys/values across all heads or across groups of heads, shrinking the KV cache and speeding up inference; GQA is common in recent open LLMs.

12.1.3 Complexity

Time $O(n^2 d)$, memory $O(n^2)$ for the attention matrix, versus $O(nd^2)$ for an RNN pass but with $O(n)$ sequential steps. Self-attention has $O(1)$ path length between any two tokens (RNN: $O(n)$). The quadratic cost in sequence length drives work on efficient attention: sparse/local attention (Longformer, BigBird), linear-attention approximations, and **FlashAttention** — exact attention computed in tiles that stay in fast on-chip memory, avoiding materialising the $n \times n$ matrix; it speeds things up by reducing memory traffic, not FLOPs.

12.2 Positional information

Attention is permutation-equivariant: shuffle the tokens and the outputs shuffle identically. Word order must be injected.

- **Sinusoidal** (original Transformer): $PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$, added to embeddings. Each dimension pair is a different frequency; relative offsets correspond to rotations, so relative position is linearly recoverable.
- **Learned absolute** embeddings (BERT, GPT-2): simple, but don't extrapolate beyond the trained maximum length.
- **Relative** positions (T5 biases) and **ALiBi** (a linear distance penalty on attention scores).
- **RoPE** (rotary): rotate query and key vectors by position-dependent angles so that $\mathbf{q}_m^\top \mathbf{k}_n$ depends only on $m - n$. Widely used in modern LLMs; context extension methods rescale its frequencies.

12.3 The Transformer block

Pre-LN decoder block (the common modern form):

$$\mathbf{H} = \mathbf{X} + \mathrm{MHA}(\mathrm{LN}(\mathbf{X}))$$
$$\mathbf{Y} = \mathbf{H} + \mathrm{FFN}(\mathrm{LN}(\mathbf{H})), \qquad \mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2\, g(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2,$$

with the FFN hidden size typically $4d$ (g = GELU, or a gated SwiGLU variant). The FFN acts position-wise and holds most of the parameters; it is often described as where much factual "memory" is stored.

Parameter estimate per block $\approx 4d^2$ (attention) $+8d^2$ (FFN with $4d$) $= 12d^2$. A model with L blocks has about $12Ld^2$ non-embedding parameters, plus Vd for token embeddings (vocabulary V).

Example 12.1 (GPT-2 small sanity check)**.** $d = 768$, $L = 12$, $V = 50{,}257$. Blocks: $12 \times 12 \times 768^2 \approx 84.9$M. Embeddings: $50{,}257 \times 768 \approx 38.6$M. Plus 1024 learned positions $\times 768 \approx 0.8$M. Total ≈ 124M — matching the published size.

12.4 Three families of Transformers

Family	Pre-training objective	Typical use
Encoder-only (BERT, RoBERTa, DeBERTa)	**Masked LM**: mask 15% of tokens (of these 80% → [MASK], 10% random token, 10% unchanged) and predict them using both left and right context. BERT also had next-sentence prediction, later shown to be of little use (RoBERTa dropped it).	Classification, NER, embeddings for search, extractive QA. Bidirectional but not naturally generative.
Decoder-only (GPT family, LLaMA, most chat LLMs)	**Causal LM**: predict the next token given all previous ones: $\max \sum_t \log p(x_t \mid x_{<t})$.	Text generation, chat, code, in-context learning. Dominant architecture for LLMs.
Encoder–decoder (T5, BART, original Transformer)	Span corruption / denoising (T5 masks spans; BART corrupts text and reconstructs).	Translation, summarisation, any input→output transformation.

For classification with BERT, a special [CLS] token's final representation (or mean pooling) is fed to a linear head and the whole network is fine-tuned.

12.5 Tokenisation

Word-level vocabularies are huge and cannot handle unseen words; character-level sequences are very long. **Sub-word tokenisation** balances the two:

- **BPE** (byte-pair encoding): start from characters (or bytes); repeatedly merge the most frequent adjacent pair until the target vocabulary size. Byte-level BPE (GPT-2 onwards) can encode any string.
- **WordPiece** (BERT): merges chosen by likelihood gain rather than raw frequency; continuation pieces marked "##".
- **Unigram LM / SentencePiece**: treats text as raw characters including spaces; language-independent.

Consequences worth mentioning: numbers and non-English scripts (including many Indian languages) can split into many tokens, raising cost and sometimes hurting arithmetic or spelling tasks.

12.6 Pre-training at scale

- Data: very large web-scale corpora, filtered and de-duplicated, plus code, books, papers.
- **Scaling laws**: loss falls predictably (roughly as a power law) with parameters, data and compute. The Chinchilla result (2022) found many earlier models under-trained: compute-optimal training uses on the order of 20 tokens per parameter. Many current models are trained far beyond that ratio because smaller, longer-trained models are cheaper to serve.
- Training compute $\approx 6ND$ FLOPs (N params, D tokens): 2 for the forward pass, 4 for the backward pass, per parameter per token.

- **Mixture of Experts (MoE)**: replace the FFN with many expert FFNs and a router that sends each token to the top-k experts. Total parameters grow while compute per token stays modest; needs load-balancing losses.

12.7 Adapting LLMs

12.7.1 Prompting and in-context learning

Zero-shot (instruction only), few-shot (examples in the prompt), chain-of-thought (ask for intermediate reasoning, which helps on multi-step problems), structured output (JSON schemas), system prompts. In-context learning means the model adapts behaviour from examples in its context window without any weight update.

12.7.2 Fine-tuning

- **Full fine-tuning**: update all weights. Most expensive; risk of catastrophic forgetting.
- **Supervised fine-tuning (SFT) / instruction tuning**: train on (instruction, good response) pairs; teaches the base model to follow instructions and adopt a format.
- **LoRA** (low-rank adaptation): freeze $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ and learn $\Delta\mathbf{W} = \mathbf{BA}$ with $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times k}$, $r \ll d$. Forward pass $\mathbf{W}_0\mathbf{x} + \frac{\alpha}{r}\mathbf{BAx}$. $\mathbf{B}$ is initialised to zero so training starts from the original model. For $d = k = 4096$, $r = 8$: 65,536 trainable params instead of 16.8M (about 0.4%). Adapters can be merged into $\mathbf{W}_0$ after training, so no inference latency is added. Rationale: fine-tuning updates appear to have low intrinsic rank.
- **QLoRA**: LoRA on top of a 4-bit quantised frozen base model — fine-tunes large models on a single GPU.
- Other PEFT methods: adapters (small bottleneck layers inserted), prefix/prompt tuning (learn virtual tokens).

12.7.3 Alignment: RLHF and DPO

RLHF pipeline: (1) SFT; (2) collect human preferences between pairs of responses and train a **reward model** with the Bradley–Terry loss $-\log\sigma(r(x, y_w) - r(x, y_l))$ (y_w preferred over y_l); (3) optimise the policy with RL (classically PPO) to maximise reward minus a KL penalty to the SFT model:

$$\max_{\pi}\ \mathbb{E}\big[r(x, y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi(\cdot \mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot \mid x)\big).$$

The KL term stops the policy drifting into text that exploits flaws in the reward model ("reward hacking").

DPO (direct preference optimisation) skips the explicit reward model and RL loop, optimising directly on preference pairs:

$$\mathcal{L}_{\mathrm{DPO}} = -\log\sigma\Big(\beta\Big[\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\Big]\Big).$$

Simpler and more stable to run; widely used. RL with verifiable rewards (e.g. unit tests for code, checkable maths answers) is another increasingly common approach for reasoning skills.

12.8 Inference and decoding

12.8.1 Decoding strategies

- **Greedy**: take the argmax token each step. Deterministic; can be repetitive.
- **Beam search**: keep the top-B partial sequences by cumulative log-probability. Good for translation/summarisation; produces bland text in open-ended generation. Length normalisation is needed because longer sequences accumulate more negative log-prob.
- **Temperature**: sample from $\text{softmax}(\mathbf{z}/T)$. $T < 1$ sharpens (more deterministic), $T > 1$ flattens (more random); $T \to 0$ equals greedy.
- **Top-k**: sample only among the k most likely tokens. **Top-p (nucleus)**: the smallest set whose cumulative probability $\geq p$ — adapts to how confident the model is.
- Repetition / frequency penalties.

12.8.2 KV cache

In autoregressive generation, keys and values of previous tokens don't change, so cache them and compute $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ only for the new token. Memory per token $= 2 \times L \times d\times$ bytes (for standard multi-head attention; GQA/MQA divide it by the head-sharing factor).

Example 12.2. A 7B model with $L = 32$, $d = 4096$, FP16: $2 \times 32 \times 4096 \times 2$ bytes $= 0.5$ MB per token. A 4096-token context then needs about 2 GB of KV cache *per sequence* — often the binding constraint on batch size in serving.

Prefill vs decode: processing the prompt is parallel and compute-bound; generating tokens one at a time is memory-bandwidth-bound. Serving systems use continuous batching and paged KV-cache memory (as in vLLM).

12.8.3 Making models cheaper

- **Quantisation**: store weights (and sometimes activations) in INT8/INT4 instead of FP16; post-training methods like GPTQ/AWQ keep quality loss small. A 7B model drops from about 14 GB (FP16) to about 3.5–4 GB at 4-bit.
- **Distillation**: train a small student to match a large teacher's output distribution (soft targets with temperature).
- **Pruning**: remove weights/heads/layers.
- **Speculative decoding**: a small draft model proposes several tokens; the large model verifies them in one parallel pass; output distribution is preserved exactly.

12.9 Retrieval-augmented generation (RAG)

Intuition. Instead of hoping the model memorised your company's policy documents, retrieve the relevant passages at question time and put them in the prompt. The model answers from evidence it can cite.

Pipeline: (1) chunk documents (by tokens/sections, with overlap); (2) embed chunks with an embedding model; (3) store in a vector index (FAISS, a vector database); (4) at query time embed the query, retrieve top-k by cosine similarity — often **hybrid** with keyword BM25 — then **rerank** with a cross-encoder; (5) generate with the retrieved context and ask for citations.

Failure points to discuss: poor chunking (an answer split across chunks), retrieval misses (vocabulary mismatch → hybrid search, query rewriting), stale index, too much irrelevant context, the model ignoring or contradicting context. Evaluate retrieval (recall@k, MRR) separately from generation (faithfulness/groundedness, answer relevance); LLM-as-judge plus human spot checks.

RAG vs fine-tuning: RAG for knowledge that changes or must be cited and access-controlled; fine-tuning for behaviour, style, format, or domain language. They combine well.

Agents and tool use: the model emits structured calls to tools (search, code execution, APIs), observes results and continues — the ReAct pattern of interleaving reasoning and actions. Key engineering concerns: reliability over many steps, cost, evaluation, and security (prompt injection from retrieved content).

12.10 Evaluating language models

- **Perplexity** $= \exp\big(-\frac{1}{N}\sum_t \log p(x_t \mid x_{<t})\big)$ — the effective branching factor; lower is better. Only comparable between models with the same tokeniser.
- **BLEU** (n-gram precision with brevity penalty; translation), **ROUGE** (n-gram recall/LCS; summarisation), **BERTScore** (embedding similarity). All correlate imperfectly with human judgement.
- Benchmarks (MMLU-style knowledge tests, coding benchmarks with unit tests, reasoning sets); beware **contamination** (test data in the pre-training corpus).
- **LLM-as-judge**: scalable but biased (position bias, verbosity bias, self-preference) — calibrate against human labels.
- **Hallucination**: fluent but false output. Mitigate with RAG, citations, constrained decoding, abstention when uncertain, verification steps.

12.11 Interview questions

Q12.1. Walk me through self-attention with shapes.

Answer. $\mathbf{X}$: $n \times d$. $\mathbf{Q}, \mathbf{K}$: $n \times d_k$; $\mathbf{V}$: $n \times d_v$. Scores $\mathbf{QK}^\top/\sqrt{d_k}$: $n \times n$. Row-wise softmax: $n \times n$. Times $\mathbf{V}$: $n \times d_v$. Multi-head concatenates h such outputs to $n \times d$ and applies $\mathbf{W}_O$.

Q12.2. Why scale by $\sqrt{d_k}$?

Answer. Dot products of random d_k-dimensional vectors have variance d_k; large scores saturate softmax and shrink gradients. Scaling keeps variance near 1.

Q12.3. Why do Transformers need positional encodings?

Answer. Self-attention is permutation-equivariant; without positions, "dog bites man" and "man bites dog" look identical.

Q12.4. BERT vs GPT?

Answer. BERT: encoder, bidirectional context, masked-LM pre-training, best for understanding tasks and embeddings. GPT: decoder, causal masking, next-token prediction, naturally generative.

Q12.5. What does the KV cache store and why does it help?

Answer. The keys and values of all previous tokens at every layer. Without it, each new token would recompute them for the whole prefix — quadratic total work instead of linear per step.

Q12.6. How does LoRA reduce trainable parameters, and does it slow inference?

Answer. Learns a rank-r update $\mathbf{BA}$ while freezing $\mathbf{W}_0$; parameters scale with $r(d+k)$ instead of dk. After training, $\mathbf{BA}$ can be merged into $\mathbf{W}_0$, so no extra latency.

Q12.7. Temperature 0 vs 1 vs 2?

Answer. 0: greedy, deterministic. 1: sample from the model's distribution as is. 2: flatter, more diverse and more error-prone.

Q12.8. Your RAG chatbot gives wrong answers. How do you debug?

Answer. Check retrieval first: is the right chunk in the top-k? If not, fix chunking, embeddings, hybrid search, query rewriting, reranking. If retrieved but answered wrongly, fix the prompt (instruct to answer only from context, cite), reduce noisy context, or use a stronger generator. Build a labelled evaluation set of question/answer/source triples.

Q12.9. What is the computational bottleneck of Transformers on long documents?

Answer. Attention's $O(n^2)$ time and memory in sequence length, and at inference the KV cache's linear growth in memory. Remedies: FlashAttention, sparse/sliding-window attention, GQA, retrieval instead of stuffing context.

Q12.10. Why is RLHF's KL penalty needed?

Answer. It keeps the policy close to the reference model so it doesn't exploit weaknesses of the learned reward model and keeps fluent, diverse language.

Chapter 13

Generative Models and Representation Learning

13.1 Autoencoders

An encoder compresses $\mathbf{x}$ to a code $\mathbf{z} = f(\mathbf{x})$; a decoder reconstructs $\hat{\mathbf{x}} = g(\mathbf{z})$; train on reconstruction loss $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. With a bottleneck smaller than the input the network must learn the important structure. A linear autoencoder with MSE learns the same subspace as PCA.

Variants: **denoising** (corrupt the input, reconstruct the clean version — learns robust features), **sparse** (penalise activations), **contractive**. Uses: dimensionality reduction, anomaly detection by reconstruction error, pre-training. A plain autoencoder is a poor generator: its latent space has gaps, so decoding a random $\mathbf{z}$ gives garbage.

13.2 Variational autoencoders (VAE)

Intuition. Make the latent space smooth and fill it in: the encoder outputs a small Gaussian cloud for each input rather than a point, and a penalty keeps all clouds close to a standard normal. Then any $\mathbf{z} \sim N(0, \mathbf{I})$ decodes to something plausible.

13.2.1 ELBO derivation

Model $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ is intractable. Introduce an encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$:

$$\begin{aligned}\log p_\theta(\mathbf{x}) &= \mathbb{E}_{q_\phi}\Big[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}\Big] + D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})\big) \\ &\geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\text{regulariser}} = \text{ELBO}.\end{aligned}$$

The inequality holds because $\mathrm{KL} \geq 0$. Maximising the ELBO jointly trains encoder and decoder.

Closed-form KL for $q = N(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$ and $p = N(0, \mathbf{I})$:

$$D_{\mathrm{KL}} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1).$$

Reparameterisation trick. Sampling $\mathbf{z} \sim q$ is not differentiable with respect to $\boldsymbol{\mu}, \boldsymbol{\sigma}$. Write $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim N(0, \mathbf{I})$: randomness moves into $\boldsymbol{\epsilon}$, and gradients flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

VAEs give a smooth, interpolable latent space and a likelihood bound but tend to produce blurry samples (Gaussian/MSE decoder averages over possibilities). β-VAE weights the KL term more to encourage disentangled factors.

13.3 Generative adversarial networks (GANs)

Intuition. A counterfeiter (generator G) tries to produce fake banknotes; a detective (discriminator D) tries to tell real from fake. Each improves by competing with the other until the fakes are indistinguishable.

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))].$$

Optimal discriminator. For fixed G with generated density p_g, maximise pointwise $p_{\text{data}} \log D + p_g \log(1 - D)$. Setting the derivative to zero gives

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}.$$

Substituting back: $V(D^*, G) = -\log 4 + 2\ \text{JSD}(p_{\text{data}} \| p_g)$, minimised (JSD $= 0$) exactly when $p_g = p_{\text{data}}$, where $D^* = 1/2$ everywhere.

Practical issues.

- **Non-saturating loss**: early on D easily rejects fakes and $\log(1 - D(G(\mathbf{z})))$ has tiny gradients; train G to maximise $\log D(G(\mathbf{z}))$ instead.
- **Mode collapse**: G produces a few outputs that fool D and ignores the rest of the distribution.
- **Unstable training**: a two-player game, not a single minimisation; oscillation and divergence are common.
- Fixes: **WGAN** (Wasserstein distance; critic constrained to be 1-Lipschitz via weight clipping or a gradient penalty — gives meaningful loss curves), spectral normalisation, DCGAN architectural rules, two-time-scale learning rates.
- Variants: conditional GAN (feed a class label), Pix2Pix (paired image translation), CycleGAN (unpaired, cycle-consistency loss), StyleGAN (high-fidelity faces).

13.4 Diffusion models

Intuition. Teach a network to remove a little noise from an image. Start from pure noise and apply the denoiser step by step; structure gradually emerges. Today this is the dominant approach for image, video and audio generation.

Forward (noising) process with schedule β_t: $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = N(\sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t \mathbf{I})$. With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \le t} \alpha_s$, any step is reachable in closed form:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim N(0, \mathbf{I}).$$

Training (DDPM): sample $\mathbf{x}_0$, a time t and noise $\boldsymbol{\epsilon}$; build $\mathbf{x}_t$; train a network (usually a U-Net or Transformer) to predict the noise:

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \big\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \big\|^2.$$

This simple loss is a reweighted form of the variational bound. **Sampling** runs the learned reverse process from $\mathbf{x}_T \sim N(0, \mathbf{I})$ down to $\mathbf{x}_0$; DDIM and other samplers cut the number of steps from about 1000 to tens.

Conditioning: text conditioning via cross-attention to a text encoder; **classifier-free guidance** trains with and without the condition and extrapolates $\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, \varnothing) + w\,[\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, c) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, \varnothing)]$; larger w follows the prompt more closely at the expense of diversity. **Latent diffusion** (Stable Diffusion) runs diffusion in a VAE's compressed latent space for efficiency.

13.5 Comparing generative models

Model	Sample quality	Training	Likelihood / latent
VAE	Blurry-ish	Stable	Lower bound; structured latent
GAN	Sharp	Unstable, mode collapse	No likelihood
Flow	Good	Stable, invertible layers	Exact likelihood
Diffusion	State of the art, diverse	Stable	Bound; slow iterative sampling
Autoregressive (GPT-style)	Excellent for discrete data (text)	Stable	Exact likelihood; sequential sampling

Image-generation metrics: Fréchet Inception Distance (FID: distance between Gaussian fits of Inception features of real vs generated images; lower is better), Inception Score, precision/recall for generative models, CLIP score for text alignment, and human evaluation.

13.6 Self-supervised and contrastive learning

- **Contrastive learning** (SimCLR, MoCo): two augmentations of the same image should have similar embeddings (positive pair); other images in the batch are negatives. InfoNCE loss:

$$\mathcal{L}_i = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}.$$

  Large batches or memory banks supply negatives; temperature τ controls sharpness.

- **CLIP**: contrastive training on (image, caption) pairs creates a shared image–text embedding space; enables zero-shot classification by comparing an image to embeddings of "a photo of a {class}".

- **Non-contrastive** (BYOL, DINO) and **masked modelling** (MAE: mask 75% of image patches and reconstruct) avoid explicit negatives.

- **Siamese networks / triplet loss** $\max(0, d(a,p) - d(a,n) + m)$ for face verification and similarity search.

13.7 Interview questions

Q13.1. Why can't you sample new data from a plain autoencoder?

Answer. Its latent space is not regularised; random latent points may fall in regions the decoder never learned. VAEs force the aggregate latent distribution toward $N(0, \mathbf{I})$.

Q13.2. What is the reparameterisation trick?

Answer. Express $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ with external noise $\boldsymbol{\epsilon}$ so sampling becomes a deterministic, differentiable function of $\boldsymbol{\mu}, \boldsymbol{\sigma}$; enables backprop through sampling.

Q13.3. What is mode collapse and how do you address it?

Answer. The generator covers only a few modes of the data. Mitigations: WGAN-GP, minibatch discrimination, unrolled GANs, spectral normalisation, or switching to diffusion.

Q13.4. Why have diffusion models largely overtaken GANs for image generation?

Answer. Stable training with a simple regression loss, better coverage of the data distribution (diversity), straightforward conditioning and guidance, and high quality; the main cost is slower sampling.

Q13.5. Derive the optimal discriminator for a GAN.

Answer. Pointwise, maximise $a \log D + b \log(1 - D)$ with $a = p_{\text{data}}$, $b = p_g$; derivative $a/D - b/(1 - D) = 0 \Rightarrow D = a/(a + b)$.

Part IV

Engineering, Coding and Interview Practice

Chapter 14

MLOps and ML System Design

ML-engineer interviews almost always include a design round: "Design a fraud detection system" or "Design the feed ranking for an app". Data-scientist interviews include a lighter version ("how would you build and measure X?"). Interviewers grade the *structure* of your thinking more than any single model choice.

14.1 A framework for any ML design question

1. **Clarify the business goal and constraints.** What decision does the model drive? Who consumes it? Latency (real-time ms vs nightly batch)? Scale (requests/sec, number of items/users)? Cost of errors (FP vs FN)? Regulatory or explainability needs?
2. **Frame it as an ML task.** Classification/regression/ranking? What exactly is the label, and how and when is it observed (delay)? Is ML even needed — would rules or a heuristic baseline do?
3. **Metrics.** Offline (AUC, PR-AUC, NDCG, RMSE) and online/business (revenue, fraud losses, CTR, retention), plus guardrails (latency, complaint rate). State how offline maps to online.
4. **Data.** Sources, labelling, sampling, class imbalance, leakage risks, privacy.
5. **Features.** Entity features, behavioural aggregates over windows, contextual features, embeddings. Where they are computed (batch vs streaming) and how training/serving consistency is ensured.
6. **Model.** Start with a simple baseline (logistic regression / GBDT), then justify complexity. Discuss trade-offs (accuracy, latency, interpretability).
7. **Serving architecture.** Batch vs online inference, retrieval + ranking stages, caching, fallbacks.
8. **Evaluation and launch.** Offline backtest → shadow mode → A/B test or canary rollout.
9. **Monitoring and iteration.** Drift, performance decay, retraining cadence, feedback loops.

14.2 Core MLOps concepts

- **Training–serving skew**: features computed differently offline (SQL on a warehouse) and online (application code) silently degrade performance. Fix: a shared feature definition, a **feature store** (offline store for training with point-in-time-correct joins, online low-latency store for serving), logging served features and training on those.

- **Point-in-time correctness**: when building training rows, use only feature values as they were at prediction time, not today's values.
- **Data drift** (covariate shift): $p(\mathbf{x})$ changes. **Concept drift**: $p(y \mid \mathbf{x})$ changes (fraudsters adapt). **Label/prior shift**: $p(y)$ changes. Detect with PSI, KS tests, monitoring prediction distributions, and delayed performance metrics once labels arrive.
- **Population Stability Index**: $\text{PSI} = \sum_i (a_i - e_i) \ln(a_i/e_i)$ across score bins; common rule of thumb in credit risk: < 0.1 stable, 0.1–0.25 monitor, > 0.25 significant shift.
- **Versioning**: code (git), data (DVC, table snapshots), models (registry, e.g. MLflow), and experiment tracking (parameters, metrics, artefacts).
- **Pipelines and orchestration**: Airflow, Kubeflow, Prefect; CI/CD for ML adds data validation and model-quality gates (CT: continuous training).
- **Deployment patterns**: batch scoring to a table; REST/gRPC microservice; streaming (Kafka + Flink); on-device (quantised models). Rollout: shadow (run silently alongside the old model), canary (small traffic share), blue–green, A/B.
- **Serving optimisations**: model compression, ONNX/TensorRT, batching requests, caching, GPU vs CPU trade-offs, autoscaling.
- **Containerisation**: Docker images for reproducible environments; Kubernetes for scaling.
- **Responsible ML**: fairness checks across groups (demographic parity, equalised odds), explainability (SHAP, reason codes in lending), privacy (PII handling, differential privacy), model documentation (model cards).

14.3 Worked design 1: real-time card fraud detection

Clarify. Score each card transaction before authorisation; latency budget around 50–100 ms; fraud rate well under 1%; a missed fraud costs the transaction amount and customer trust; a false decline costs a sale and annoys a customer; analysts can review a limited number of cases per day.

Framing. Binary classification producing a risk score, then a policy: approve / step-up authentication (OTP) / decline / send to review. The threshold is set from costs and review capacity, not 0.5.

Labels. Chargebacks and confirmed fraud reports — arriving with delays of weeks. Recent data has incomplete labels; handle with a label-maturity window.

Features. Transaction: amount, merchant category, channel, country. Velocity: count and sum of transactions on this card in the last 1 min / 1 hour / 24 hours (streaming aggregates). Deviation: amount vs the card's typical amount, new merchant, new device, distance from the last transaction's location divided by elapsed time ("impossible travel"). Network features: shared devices/IPs across cards (graph features).

Model. Gradient-boosted trees as the main model (strong on tabular data, fast to score, SHAP reason codes); rules for known patterns and regulatory blocks; optionally an anomaly detector for novel patterns. Class imbalance handled with weights and threshold tuning; evaluate with PR-AUC and recall at a fixed false-positive rate, plus money-weighted metrics (fraud value caught).

Serving. Streaming feature computation (Kafka + Flink) writing to an online store (e.g. Redis); model served in a low-latency service; fallback to rules if the model times out.

Monitoring. Score distribution, PSI on key features, decline rate, fraud caught once labels mature; adversarial drift is expected, so retrain frequently (weekly) and watch for feedback loops (declined transactions never get fraud labels).

14.4 Worked design 2: product recommendations on an e-commerce home page

Goal. Increase purchases (and longer-term retention) without hurting diversity or page latency.

Architecture. *Candidate generation* (hundreds from millions): co-purchase item–item similarity, two-tower embedding retrieval with approximate nearest neighbour search, trending items, items from recently viewed categories. *Ranking*: GBDT or deep model predicting $P(\text{click})$ and $P(\text{purchase} \mid \text{click})$ with user, item, context and cross features; combine into an expected-value score. *Re-ranking*: business rules, diversity, de-duplication, out-of-stock removal, sponsored slots.

Cold start. Popularity by region/segment for new users; content embeddings for new items; exploration slots.

Evaluation. Offline recall@k (retrieval), NDCG/AUC (ranking) on time-split data; online A/B test on purchases per session, revenue, with guardrails on latency and returns. Watch position bias in training data (items at the top get clicked more regardless of relevance) — use position as a feature during training and fix it at serving, or use randomised exploration data.

14.5 Worked design 3: customer churn for a telecom / subscription business

Define churn precisely (no recharge in 30 days after expiry). Predict churn probability 30 days ahead using usage trends (declining data usage, fewer calls), recharge history, complaints, network quality in the customer's area, tenure, plan. GBDT with SHAP for reasons. The real objective is *uplift*: whom does a retention offer actually change? A customer who would stay anyway wastes the discount; a "sure thing" to churn may be unpersuadable. Uplift modelling (two-model or T-learner approach, uplift trees) estimated from a randomised campaign is the strong answer. Measure with a holdout control group.

14.6 Interview questions

Q14.1. Your model's AUC dropped from 0.85 to 0.78 over three months. What do you check?

Answer. Data pipeline breaks (nulls, schema changes, unit changes), feature drift (PSI per feature), label definition or delay changes, concept drift (behaviour changed), upstream product changes (new user segment), and seasonality. Then retrain on recent data and consider a monitoring + automatic retraining setup.

Q14.2. Batch vs real-time inference — how do you choose?

Answer. Real time when the decision depends on fresh context and must happen in-session (fraud, search ranking). Batch when predictions can be precomputed and stay valid for hours/days (churn scores, daily demand forecasts); cheaper and simpler.

Q14.3. How would you safely replace a production model?

Answer. Offline backtest, shadow deployment comparing predictions on live traffic, canary to a small share of traffic with guardrail monitoring, then A/B test on the business metric, with a rollback plan.

Q14.4. What is a feature store and why use one?

Answer. A system that stores feature definitions and values with an offline store (historical, point-in-time-correct for training) and an online store (low-latency for serving), giving reuse across teams and consistency between training and serving.

Chapter 15

Python, NumPy and pandas for Assessments

Coding rounds for DS/ML roles test three things: general Python fluency, data manipulation (pandas/NumPy) under time pressure, and the ability to implement a classic algorithm from scratch. All code in this chapter was run before inclusion.

15.1 Python fundamentals that get asked

15.1.1 Data structures and complexity

Type	Properties	Average complexity
`list`	ordered, mutable, dynamic array	index $O(1)$; append $O(1)$ amortised; insert/delete at front $O(n)$; `x in list` $O(n)$
`tuple`	ordered, immutable, hashable if contents are	like list for reads
`dict`	hash map, insertion-ordered (3.7+)	get/set/delete/`in` $O(1)$
`set`	hash set, unique elements	add/`in` $O(1)$; union/intersection $O(n)$
`deque` (collections)	double-ended queue	append/pop at both ends $O(1)$
`heapq`	binary min-heap on a list	push/pop $O(\log n)$; top-k via `nlargest`

15.1.2 Gotchas

```
# 1. Mutable default arguments are created once
def add(item, bucket=[]):          # bug
    bucket.append(item); return bucket
print(add(1), add(2))              # [1, 2] [1, 2]
def add_ok(item, bucket=None):
    bucket = [] if bucket is None else bucket
    bucket.append(item); return bucket

# 2. Aliasing, shallow vs deep copy
import copy
a = [[1, 2], [3, 4]]
b = a                  # same object
c = a.copy()           # new outer list, same inner lists
d = copy.deepcopy(a)   # fully independent
```

```
a[0][0] = 99
print(b[0][0], c[0][0], d[0][0])   # 99 99 1

# 3. is vs ==
x = [1, 2]; y = [1, 2]
print(x == y, x is y)              # True False

# 4. Late binding in closures
fs = [lambda: i for i in range(3)]
print([f() for f in fs])           # [2, 2, 2]
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])           # [0, 1, 2]

# 5. Floating point
print(0.1 + 0.2 == 0.3, abs(0.1 + 0.2 - 0.3) < 1e-9)  # False True
```

15.1.3 Idioms worth having at your fingertips

```
from collections import Counter, defaultdict
words = "the quick fox and the lazy dog and the cat".split()
print(Counter(words).most_common(2))        # [('the', 3), ('and', 2)]

groups = defaultdict(list)
for w in words:
    groups[len(w)].append(w)
print(dict(groups))

squares = {n: n * n for n in range(5)}       # dict comprehension
evens = [n for n in range(10) if n % 2 == 0]
pairs = list(zip("abc", [1, 2, 3]))
for i, w in enumerate(words[:2]):
    print(i, w)

def gen_squares(n):                          # generator: lazy, O(1) memory
    for i in range(n):
        yield i * i
print(sum(gen_squares(1_000_000)))

def timer(fn):                               # decorator
    import time, functools
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        t = time.perf_counter(); out = fn(*args, **kwargs)
        print(f"{fn.__name__}: {time.perf_counter() - t:.4f}s"); return out
    return wrapper

@timer
def slow_sum(n): return sum(range(n))
slow_sum(10**6)

print(sorted(words, key=lambda w: (-len(w), w))[:3])  # sort by length desc, then
    alpha
```

The GIL. CPython's global interpreter lock lets only one thread execute Python bytecode at a time, so threads don't speed up CPU-bound pure-Python code; use `multiprocessing`, or vectorised libraries (NumPy releases the GIL in C code). Threads still help for I/O-bound work.

15.2 NumPy

15.2.1 Vectorisation and broadcasting

Python loops over arrays are slow (interpreter overhead per element); NumPy operations run in compiled C over contiguous memory. **Broadcasting rules**: compare shapes from the right; two dimensions are compatible if equal or one of them is 1; missing leading dimensions are treated as 1.

```
import numpy as np
X = np.arange(12, dtype=float).reshape(4, 3)  # 4 samples, 3 features
mu, sd = X.mean(axis=0), X.std(axis=0)        # axis=0 -> down the rows, per column
Z = (X - mu) / sd                             # (4,3) - (3,) broadcasts
print(Z.shape, Z.mean(axis=0).round(6))

# Pairwise squared distances between rows of A (n,d) and B (m,d) without loops
A = np.random.rand(5, 2); B = np.random.rand(3, 2)
D2 = (A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2 * A @ B.T   # (5,3)
D2_check = ((A[:, None, :] - B[None, :, :])**2).sum(-1)
print(np.allclose(D2, D2_check))

# Views vs copies
a = np.arange(6); v = a[1:4]; v[0] = 100
print(a)              # slicing returns a VIEW: a changed
f = a[[1, 2]]; f[0] = -1
print(a)              # fancy indexing returns a COPY: a unchanged

# Boolean masks, where, argmax, unique
x = np.array([3, -1, 4, -1, 5])
print(x[x > 0], np.where(x > 0, x, 0), x.argmax())
vals, counts = np.unique(x, return_counts=True)
print(dict(zip(vals.tolist(), counts.tolist())))
```

15.2.2 Numerically stable softmax and log-sum-exp

```
import numpy as np
def softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)   # shift: same result, no overflow
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

def log_softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)
    return z - np.log(np.exp(z).sum(axis=axis, keepdims=True))

print(softmax(np.array([1000.0, 1001.0, 1002.0])))   # naive exp would overflow
```

15.3 pandas

15.3.1 Core operations

```
import pandas as pd, numpy as np
orders = pd.DataFrame({
    "order_id": [1, 2, 3, 4, 5, 6],
    "cust_id":  [10, 10, 20, 30, 30, 30],
    "city":     ["Chennai", "Chennai", "Delhi", "Pune", "Pune", "Pune"],
```

```
    "amount":   [250.0, 400.0, np.nan, 120.0, 900.0, 300.0],
    "date": pd.to_datetime(["2026-01-03", "2026-01-15", "2026-01-20",
                            "2026-02-01", "2026-02-11", "2026-03-05"]),
})
customers = pd.DataFrame({"cust_id": [10, 20, 40],
                          "segment": ["gold", "silver", "gold"]})

# Selection: loc = labels, iloc = positions
print(orders.loc[orders["amount"] > 200, ["order_id", "amount"]])
print(orders.iloc[:2, :3])

# Missing values
print(orders["amount"].isna().sum())
orders["amount_f"] = orders["amount"].fillna(orders["amount"].median())

# groupby + named aggregation
summary = (orders.groupby("city")
           .agg(n_orders=("order_id", "count"),
                revenue=("amount", "sum"),          # sum skips NaN
                avg_order=("amount", "mean"))
           .sort_values("revenue", ascending=False)
           .reset_index())
print(summary)

# transform keeps the original shape: share of each order in its customer's spend
orders["share"] = orders["amount_f"] / orders.groupby("cust_id")["amount_f"].transform
    ("sum")

# Merges: how = inner / left / right / outer
m = orders.merge(customers, on="cust_id", how="left", indicator=True)
print(m[["order_id", "cust_id", "segment", "_merge"]])

# Pivot table and melt
orders["month"] = orders["date"].dt.to_period("M").astype(str)
pv = orders.pivot_table(index="city", columns="month", values="amount_f",
                        aggfunc="sum", fill_value=0)
print(pv)
long = pv.reset_index().melt(id_vars="city", var_name="month", value_name="rev")
print(long.head())
```

15.3.2 Time-aware and window-style operations

```
import pandas as pd
s = pd.DataFrame({"cust": [1, 1, 1, 2, 2],
                  "day": pd.to_datetime(["2026-01-01", "2026-01-05", "2026-01-20",
                                         "2026-01-02", "2026-01-03"]),
                  "spend": [100, 50, 70, 20, 30]}).sort_values(["cust", "day"])
s["prev_spend"] = s.groupby("cust")["spend"].shift(1)        # lag (like SQL LAG)
s["days_since_prev"] = s.groupby("cust")["day"].diff().dt.days
s["running_spend"] = s.groupby("cust")["spend"].cumsum()
s["rank_in_cust"] = s.groupby("cust")["spend"].rank(method="dense", ascending=False)
s["roll2_mean"] = (s.groupby("cust")["spend"]
                   .transform(lambda x: x.rolling(2, min_periods=1).mean()))
print(s)

# Top-1 row per group (highest spend per customer)
print(s.sort_values("spend", ascending=False).drop_duplicates("cust"))
# or: s.loc[s.groupby("cust")["spend"].idxmax()]
```

15.3.3 pandas pitfalls

- **Chained assignment** `df[df.a>0]["b"] = 1` may modify a temporary copy (pandas raises a *SettingWithCopy* warning). Use `df.loc[df.a>0, "b"] = 1`.
- `apply` with a Python function row by row is slow; prefer vectorised column operations, `np.where`, `np.select`, `map`.
- Merges on keys with duplicates on both sides multiply rows (many-to-many). Check with `validate="one_to_one"` or `"many_to_one"`.
- `count` excludes NaN, `size` includes it; `mean`/`sum` skip NaN by default.
- Integer columns with NaN become float (or use the nullable `Int64` dtype).
- `groupby` drops NaN keys by default (`dropna=False` keeps them).

15.4 scikit-learn: leakage-safe pipelines

```
import numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score

rng = np.random.default_rng(0)
n = 500
df = pd.DataFrame({"age": rng.integers(18, 70, n).astype(float),
                   "income": rng.normal(50, 15, n),
                   "plan": rng.choice(["basic", "pro", "max"], n)})
df.loc[rng.choice(n, 30, replace=False), "income"] = np.nan
y = (df["income"].fillna(50) + rng.normal(0, 10, n) > 55).astype(int)

num, cat = ["age", "income"], ["plan"]
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=1000))])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(model, df, y, cv=cv, scoring="roc_auc")
print(scores.mean().round(3))   # imputer and scaler are re-fit inside every fold
```

Hyperparameters inside a pipeline are addressed as `"clf__C"` in `GridSearchCV`.

15.5 Implement-from-scratch questions

15.5.1 Linear regression with gradient descent (and closed form)

```
import numpy as np
rng = np.random.default_rng(42)
X = rng.normal(size=(200, 3)); true_w = np.array([2.0, -1.0, 0.5])
y = X @ true_w + 4.0 + rng.normal(0, 0.1, 200)

def fit_linreg_gd(X, y, lr=0.1, epochs=500):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        err = X @ w + b - y               # (n,)
        w -= lr * (2 / n) * X.T @ err     # gradient of MSE
        b -= lr * (2 / n) * err.sum()
    return w, b

w, b = fit_linreg_gd(X, y)
Xb = np.c_[np.ones(len(X)), X]
beta = np.linalg.lstsq(Xb, y, rcond=None)[0]   # closed form, stable solver
print(w.round(3), round(b, 3), beta.round(3))
```

15.5.2 Logistic regression

```
import numpy as np
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def fit_logreg(X, y, lr=0.1, epochs=2000, l2=0.0):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        grad_w = X.T @ (p - y) / n + l2 * w
        grad_b = (p - y).mean()
        w -= lr * grad_w; b -= lr * grad_b
    return w, b

def log_loss(y, p, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

rng = np.random.default_rng(1)
X = rng.normal(size=(500, 2))
y = (X[:, 0] - 2 * X[:, 1] + rng.normal(0, 0.5, 500) > 0).astype(float)
w, b = fit_logreg(X, y)
p = sigmoid(X @ w + b)
print(w.round(2), round(log_loss(y, p), 3), ((p > 0.5) == y).mean())
```

15.5.3 k-means

```
import numpy as np
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)]            # random init
    for _ in range(iters):
        d2 = ((X[:, None, :] - C[None, :, :])**2).sum(-1)  # (n,k)
        labels = d2.argmin(1)
```

```
        newC = np.array([X[labels == j].mean(0) if np.any(labels == j) else C[j]
                         for j in range(k)])
        if np.allclose(newC, C): break
        C = newC
    inertia = ((X - C[labels])**2).sum()
    return labels, C, inertia

rng = np.random.default_rng(3)
X = np.vstack([rng.normal(c, 0.3, size=(50, 2)) for c in [(0, 0), (3, 3), (0, 4)]])
labels, C, inertia = kmeans(X, 3)
print(np.round(C[np.argsort(C[:, 0] + C[:, 1])], 1), round(inertia, 1))
```

15.5.4 k-nearest neighbours

```
import numpy as np
from collections import Counter
def knn_predict(Xtr, ytr, Xte, k=3):
    d2 = ((Xte[:, None, :] - Xtr[None, :, :])**2).sum(-1)
    idx = np.argsort(d2, axis=1)[:, :k]
    return np.array([Counter(ytr[row]).most_common(1)[0][0] for row in idx])

Xtr = np.array([[0, 0], [0, 1], [5, 5], [6, 5]]); ytr = np.array([0, 0, 1, 1])
print(knn_predict(Xtr, ytr, np.array([[0.2, 0.4], [5.5, 4.8]]), k=3))   # [0 1]
```

15.5.5 Metrics by hand: confusion matrix and ROC-AUC

```
import numpy as np
def confusion(y, yhat):
    tp = int(((y == 1) & (yhat == 1)).sum()); fp = int(((y == 0) & (yhat == 1)).sum())
    fn = int(((y == 1) & (yhat == 0)).sum()); tn = int(((y == 0) & (yhat == 0)).sum())
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * prec * rec / (prec + rec) if prec + rec else 0.0
    return dict(tp=tp, fp=fp, fn=fn, tn=tn, precision=prec, recall=rec, f1=f1)

def roc_auc(y, score):
    # AUC = P(score_pos > score_neg); ties count 1/2. Rank-based O(n log n).
    from scipy.stats import rankdata
    r = rankdata(score)
    n_pos = y.sum(); n_neg = len(y) - n_pos
    return (r[y == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)

y = np.array([1, 0, 1, 1, 0, 0, 1, 0])
s = np.array([0.9, 0.4, 0.65, 0.3, 0.2, 0.55, 0.8, 0.1])
print(confusion(y, (s >= 0.5).astype(int)))
print(roc_auc(y, s))   # 0.875 (14 of 16 pos-neg pairs ordered correctly)
```

15.5.6 A two-layer neural network with manual backprop

```
import numpy as np
rng = np.random.default_rng(0)
X = rng.normal(size=(400, 2))
y = ((X[:, 0]**2 + X[:, 1]**2) > 1.5).astype(int)        # circle: not linearly
    separable
```

```
Y = np.eye(2)[y]                                          # one-hot (n,2)

h = 16
W1 = rng.normal(0, np.sqrt(2 / 2), (2, h)); b1 = np.zeros(h)   # He init
W2 = rng.normal(0, np.sqrt(2 / h), (h, 2)); b2 = np.zeros(2)
lr = 0.5
for epoch in range(2000):
    Z1 = X @ W1 + b1; A1 = np.maximum(0, Z1)              # ReLU
    Z2 = A1 @ W2 + b2
    Z2 -= Z2.max(1, keepdims=True); P = np.exp(Z2); P /= P.sum(1, keepdims=True)
    loss = -np.log(P[np.arange(len(y)), y]).mean()
    dZ2 = (P - Y) / len(X)                                # softmax + CE gradient
    dW2 = A1.T @ dZ2; db2 = dZ2.sum(0)
    dA1 = dZ2 @ W2.T; dZ1 = dA1 * (Z1 > 0)
    dW1 = X.T @ dZ1; db1 = dZ1.sum(0)
    W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2
print(round(loss, 3), (P.argmax(1) == y).mean())
```

15.5.7 PCA via SVD

```
import numpy as np
def pca(X, k):
    Xc = X - X.mean(0)
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    explained = S**2 / (S**2).sum()
    return Xc @ Vt[:k].T, Vt[:k], explained[:k]

rng = np.random.default_rng(0)
t = rng.normal(size=300)
X = np.c_[t, 2 * t + rng.normal(0, 0.1, 300), rng.normal(0, 0.1, 300)]
Z, comps, ev = pca(X, 2)
print(ev.round(3))      # first component carries almost all the variance
```

15.6 Typical DSA patterns in DS coding rounds

```
# Two-sum with a hash map: O(n)
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen: return [seen[target - x], i]
        seen[x] = i

# Sliding window: longest substring without repeating characters, O(n)
def longest_unique(s):
    last, start, best = {}, 0, 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start: start = last[ch] + 1
        last[ch] = i; best = max(best, i - start + 1)
    return best

# Top-k frequent elements with a heap: O(n log k)
import heapq
from collections import Counter
def top_k(nums, k):
```

```
    return [x for x, _ in heapq.nlargest(k, Counter(nums).items(), key=lambda t: t[1])
    ]

# Moving average of a stream
from collections import deque
class MovingAverage:
    def __init__(self, size): self.q, self.size, self.s = deque(), size, 0.0
    def next(self, val):
        self.q.append(val); self.s += val
        if len(self.q) > self.size: self.s -= self.q.popleft()
        return self.s / len(self.q)

# Binary search
def lower_bound(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo

print(two_sum([2, 7, 11, 15], 9), longest_unique("abcabcbb"),
      top_k([1, 1, 1, 2, 2, 3], 2), lower_bound([1, 3, 5, 7], 4))
m = MovingAverage(3); print([round(m.next(v), 2) for v in [1, 10, 3, 5]])
```

Other common ones: merging intervals (sort by start), prefix sums for range queries, BFS/DFS on grids, dynamic programming basics (climbing stairs, coin change, longest common subsequence), stack for valid parentheses.

15.7 Interview questions

Q15.1. List vs tuple vs set vs dict — when do you use which?

Answer. List for ordered mutable sequences; tuple for fixed records and dictionary keys; set for fast membership and de-duplication; dict for key–value lookup. Membership test is $O(1)$ for set/dict and $O(n)$ for list.

Q15.2. What does `axis=0` mean in NumPy/pandas aggregations?

Answer. Aggregate down the rows, producing one value per column. `axis=1` aggregates across columns, one value per row.

Q15.3. `groupby().transform` vs `groupby().agg` vs `apply`?

Answer. `agg` returns one row per group; `transform` returns a result aligned to the original rows (useful for group-normalised features); `apply` is the most flexible and the slowest.

Q15.4. Why is a NumPy vectorised operation faster than a Python loop?

Answer. It runs in compiled code over contiguous typed memory, avoiding per-element interpreter overhead and type checks, and can use SIMD instructions.

Q15.5. How do you avoid leakage when scaling features?

Answer. Fit the scaler on training data only (inside each CV fold), then transform validation/test with it. A scikit-learn `Pipeline` enforces this.

Chapter 16

SQL for Data Science Interviews

SQL is the most reliably tested skill in DS online assessments. Most questions reduce to a handful of patterns: join, aggregate, filter groups, and window functions. Every query below was executed on SQLite against the sample tables; syntax is standard and works on PostgreSQL/MySQL 8 unless noted (date functions differ most between dialects).

16.1 Sample schema used in this chapter

```
employees(emp_id, name, dept, salary, manager_id)
customers(cust_id, name, city, signup_date)
orders(order_id, cust_id, order_date, amount)
logins(user_id, login_date)
```

16.2 Logical order of execution

`FROM/JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` (window functions evaluated here) → `DISTINCT` → `ORDER BY` → `LIMIT`.

Consequences that interviewers test:

- `WHERE` filters rows before grouping and cannot use aggregates; `HAVING` filters groups after aggregation.
- A column alias defined in `SELECT` cannot be used in `WHERE` (standard SQL), but can in `ORDER BY`.
- You cannot filter on a window function in the same query's `WHERE`; wrap it in a subquery/CTE.

16.3 Joins

- `INNER JOIN`: only matching rows.
- `LEFT JOIN`: all rows of the left table, NULLs where no match.
- `RIGHT JOIN`, `FULL OUTER JOIN`: symmetric versions (SQLite added these only in recent versions; MySQL has no FULL OUTER — emulate with `LEFT UNION RIGHT`).
- `CROSS JOIN`: Cartesian product ($m \times n$ rows).
- **Self join**: a table joined to itself (employee and manager).

- **Anti join**: rows in A with no match in B — `LEFT JOIN ... WHERE b.key IS NULL` or `NOT EXISTS`.

Row-count puzzle: Table A has key values (1,1,2), B has (1,1,1,3). INNER JOIN on key gives $2 \times 3 = 6$ rows; LEFT JOIN gives $6 + 1$ (the 2 with NULL) $= 7$. Duplicate keys multiply.

Join-condition trap: putting a filter on the right table in `WHERE` after a `LEFT JOIN` silently turns it into an inner join (NULL rows fail the filter). Put it in the `ON` clause instead.

16.4 NULL semantics

- `NULL = NULL` is unknown, not true; use `IS NULL`.
- `COUNT(*)` counts rows; `COUNT(col)` counts non-NULL values; `AVG`/`SUM` ignore NULLs.
- `NOT IN (subquery)` returns no rows if the subquery contains a NULL. Prefer `NOT EXISTS`.
- `COALESCE(x, 0)` replaces NULL; `NULLIF(a, b)` returns NULL if equal (avoid divide-by-zero: `x / NULLIF(y,0)`).

16.5 Window functions

`func() OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)` computes across related rows *without collapsing them.*

- Ranking: `ROW_NUMBER()` (1,2,3,4 — unique), `RANK()` (1,2,2,4 — gaps after ties), `DENSE_RANK()` (1,2,2,3 — no gaps), `NTILE(n)` (buckets), `PERCENT_RANK()`.
- Offset: `LAG(col, k)`, `LEAD(col, k)`, `FIRST_VALUE`, `LAST_VALUE`.
- Aggregates as windows: `SUM/AVG/COUNT/MIN/MAX(...) OVER (...)`.
- Frames: `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` (moving window of 3 rows). With `ORDER BY` and no explicit frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — a running total in which *ties in the ORDER BY value are included together.* Use `ROWS` when you need a strict row-by-row running total. `LAST_VALUE` with the default frame returns the current row, a common surprise.

16.6 Classic interview queries

16.6.1 Second-highest (and N-th highest) salary

```
-- Using a subquery
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- N-th highest distinct salary (N = 3) with DENSE_RANK
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 3;
```

16.6.2 Top-N per group

```
-- Top 2 earners in each department (ties broken arbitrarily by ROW_NUMBER)
WITH ranked AS (
  SELECT dept, name, salary,
         ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn
  FROM employees
)
SELECT dept, name, salary
FROM ranked
WHERE rn <= 2
ORDER BY dept, salary DESC;
```

16.6.3 Employees earning more than their manager (self join)

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

16.6.4 Customers who never ordered (anti join)

```
SELECT c.cust_id, c.name
FROM customers c
LEFT JOIN orders o ON o.cust_id = c.cust_id
WHERE o.order_id IS NULL;

-- equivalent
SELECT cust_id, name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.cust_id);
```

16.6.5 Duplicates: find and keep one

```
-- Which (cust_id, order_date) pairs appear more than once?
SELECT cust_id, order_date, COUNT(*) AS n
FROM orders
GROUP BY cust_id, order_date
HAVING COUNT(*) > 1;

-- Keep only the first order_id per duplicated pair
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY cust_id, order_date
                                 ORDER BY order_id) AS rn
  FROM orders o) t
WHERE rn = 1;
```

16.6.6 Department aggregates with HAVING and share of total

```
SELECT dept,
       COUNT(*) AS headcount,
       ROUND(AVG(salary), 1) AS avg_salary,
       ROUND(100.0 * SUM(salary) / SUM(SUM(salary)) OVER (), 1) AS pct_of_payroll
```

```
FROM employees
GROUP BY dept
HAVING COUNT(*) >= 2
ORDER BY avg_salary DESC;
```

Note `SUM(SUM(salary)) OVER ()`: the inner `SUM` is the group aggregate, the outer is a window over all groups. Subtle point: windows are evaluated *after* `HAVING`, so the percentage is a share of the departments that survive the filter, not of the whole company. If you want company-wide shares, compute the total in a separate CTE. Also note `100.0` — integer division (`100 * a / b` with integers) truncates in many dialects.

16.6.7 Running total, moving average, previous value

```
SELECT cust_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY cust_id ORDER BY order_date, order_id
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
    running_total,
       ROUND(AVG(amount) OVER (PARTITION BY cust_id ORDER BY order_date, order_id
                         ROWS BETWEEN 1 PRECEDING AND CURRENT ROW), 1) AS moving_avg_2,

       LAG(amount) OVER (PARTITION BY cust_id ORDER BY order_date, order_id) AS
    prev_amount
FROM orders
ORDER BY cust_id, order_date, order_id;
```

16.6.8 Month-over-month revenue growth

```
WITH monthly AS (
  SELECT strftime('%Y-%m', order_date) AS month,   -- PostgreSQL: DATE_TRUNC('month',
    order_date)
         SUM(amount) AS revenue
  FROM orders
  GROUP BY 1
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / LAG(revenue) OVER (ORDER BY month), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

16.6.9 Pivot with conditional aggregation

```
SELECT c.city,
       SUM(CASE WHEN strftime('%m', o.order_date) = '01' THEN o.amount ELSE 0 END) AS
    jan,
       SUM(CASE WHEN strftime('%m', o.order_date) = '02' THEN o.amount ELSE 0 END) AS
    feb,
       SUM(CASE WHEN strftime('%m', o.order_date) = '03' THEN o.amount ELSE 0 END) AS
    mar
FROM orders o
JOIN customers c ON c.cust_id = o.cust_id
GROUP BY c.city
ORDER BY c.city;
```

16.6.10 Consecutive days (gaps and islands)

Real-world example. "Find users who logged in on at least 3 consecutive days" is a favourite at consumer-tech firms. The trick: within a run of consecutive dates, `date − row_number` is constant.

```
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins),
g AS (
  SELECT user_id, login_date,
         julianday(login_date)
           - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS grp
         -- PostgreSQL: login_date - ROW_NUMBER() OVER (...) * INTERVAL '1 day'
  FROM d
)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end,
       COUNT(*) AS streak_len
FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

16.6.11 Retention: share of users who returned the next day

```
WITH firsts AS (
  SELECT user_id, MIN(login_date) AS first_day FROM logins GROUP BY user_id
)
SELECT ROUND(100.0 * COUNT(DISTINCT l.user_id) / (SELECT COUNT(*) FROM firsts), 1)
         AS day1_retention_pct
FROM firsts f
JOIN logins l
  ON l.user_id = f.user_id
 AND l.login_date = date(f.first_day, '+1 day');   -- PostgreSQL: f.first_day + 1
```

16.6.12 Monthly cohort analysis

```
WITH first_order AS (
  SELECT cust_id, strftime('%Y-%m', MIN(order_date)) AS cohort
  FROM orders GROUP BY cust_id
),
activity AS (
  SELECT DISTINCT o.cust_id, f.cohort, strftime('%Y-%m', o.order_date) AS month
  FROM orders o JOIN first_order f ON f.cust_id = o.cust_id
)
SELECT cohort, month, COUNT(*) AS active_customers
FROM activity
GROUP BY cohort, month
ORDER BY cohort, month;
```

16.6.13 Median

```
-- Portable median via row numbers (PostgreSQL also has PERCENTILE_CONT(0.5))
WITH r AS (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
```

```
        COUNT(*) OVER () AS n
  FROM employees
)
SELECT AVG(salary) AS median_salary
FROM r
WHERE rn IN ((n + 1) / 2, (n + 2) / 2);   -- integer division picks 1 or 2 middle rows
```

16.6.14 Customers whose every order exceeded 200 (relational division style)

```
SELECT cust_id
FROM orders
GROUP BY cust_id
HAVING MIN(amount) > 200;
```

16.7 Other things you may be asked

- `UNION` removes duplicates (sort/hash cost); `UNION ALL` keeps them and is faster.
- `DELETE` (row by row, can be rolled back, fires triggers) vs `TRUNCATE` (removes all rows quickly) vs `DROP` (removes the table).
- CTEs (`WITH`) improve readability; recursive CTEs walk hierarchies (org charts).
- **Indexes** (B-trees) speed lookups and joins on the indexed columns at the cost of slower writes and storage. A function applied to an indexed column in `WHERE` (`WHERE YEAR(order_date)=2026`) often prevents index use; write a range condition instead.
- Normalisation (1NF–3NF) reduces redundancy; analytical warehouses often denormalise into star schemas (fact tables + dimension tables).
- **Primary key** (unique, not null) vs **foreign key** (references another table's key) vs **unique** constraint.
- `EXPLAIN` shows the query plan.

16.8 Interview questions

Q16.1. WHERE vs HAVING?

Answer. WHERE filters rows before aggregation; HAVING filters groups after aggregation and can use aggregate functions.

Q16.2. RANK vs DENSE_RANK vs ROW_NUMBER for salaries 100, 90, 90, 80?

Answer. ROW_NUMBER: 1,2,3,4. RANK: 1,2,2,4. DENSE_RANK: 1,2,2,3.

Q16.3. Why might `SELECT * FROM a WHERE id NOT IN (SELECT id FROM b)` return nothing?

Answer. If `b.id` contains a NULL, `x NOT IN (..., NULL)` is never true. Use `NOT EXISTS` or filter NULLs.

Q16.4. How do you compute DAU/MAU (stickiness)?

Answer. Average daily distinct active users in a month divided by distinct active users in that month: count distinct users per day, average them, divide by the monthly distinct count.

Q16.5. Your LEFT JOIN returned fewer rows than the left table. Why?

Answer. Impossible from the join alone (a LEFT JOIN never drops left rows); a WHERE condition on a right-table column filtered out the NULL-extended rows. Move it into the ON clause.

Chapter 17

Rapid-Fire Question Bank

Use this chapter for timed self-testing: cover the answer, answer aloud in under a minute, then compare. Answers are deliberately compact; the earlier chapters have the derivations.

17.1 Statistics and probability

Q17.1. Explain a p-value to a non-technical manager.

Answer. If the change truly had no effect, how surprising would results this extreme be? A p-value of 0.01 means results like ours would happen about 1% of the time by chance alone. It is not the probability that the change does nothing.

Q17.2. Type I vs Type II error in a fraud model's alert threshold?

Answer. Type I: flagging a genuine transaction (false alarm). Type II: missing fraud. Lowering the threshold trades Type II for Type I.

Q17.3. What is statistical power and what increases it?

Answer. Probability of detecting a real effect. Increases with sample size, effect size, significance level α, and lower variance (CUPED, better metrics).

Q17.4. Standard deviation vs standard error?

Answer. SD measures spread of individual observations; SE $= \sigma/\sqrt{n}$ measures uncertainty of the sample mean.

Q17.5. When would you use the median over the mean?

Answer. Skewed data or outliers (income, house prices, session lengths).

Q17.6. Covariance vs correlation?

Answer. Correlation is covariance scaled by both standard deviations — unit-free, in $[-1, 1]$; measures linear association only.

Q17.7. What does a Q–Q plot show?

Answer. Sample quantiles against theoretical quantiles; points on a straight line indicate the assumed distribution fits; curved tails show skew or heavy tails.

Q17.8. A coin gives 60 heads in 100 tosses. Is it fair at 5%?

Answer. Under H_0: mean 50, SD $= \sqrt{100 \times 0.25} = 5$; $z = (60 - 50)/5 = 2 > 1.96$. Reject at 5% (two-sided p ≈ 0.046; slightly larger with continuity correction — borderline).

Q17.9. Expected value of a fair die roll and its variance?

Answer. 3.5; $\mathbb{E}X^2 = 91/6$, Var $= 91/6 - 12.25 = 35/12 \approx 2.92$.

Q17.10. Sum of two dice: probability of 7?

Answer. $6/36 = 1/6$.

Q17.11. You run 100 A/B tests with no true effects at $\alpha = 0.05$. How many "significant"?

Answer. About 5 expected false positives.

17.2 Classical machine learning

Q17.12. Explain overfitting with an everyday analogy.

Answer. A student who memorises past exam papers word for word scores perfectly on those papers but fails new questions; they learned the noise, not the subject.

Q17.13. L1 vs L2 regularisation in one line each.

Answer. L1: absolute weights, produces exact zeros (feature selection), Laplace prior. L2: squared weights, shrinks smoothly, handles collinearity, Gaussian prior.

Q17.14. Why standardise before regularised regression?

Answer. The penalty treats all coefficients equally, but a coefficient's size depends on the feature's units; unscaled features get unequal penalisation.

Q17.15. What is the kernel trick?

Answer. Computing dot products in a high-dimensional feature space via a kernel function on the original inputs, so linear algorithms that only need dot products learn non-linear boundaries cheaply.

Q17.16. Gini vs entropy?

Answer. Both measure node impurity; nearly always choose the same split; Gini is cheaper to compute.

Q17.17. Why do random forests rarely need feature scaling?

Answer. Tree splits are thresholds on single features, invariant to monotone transformations.

Q17.18. What is out-of-bag error?

Answer. Each tree is evaluated on the roughly 37% of samples not in its bootstrap sample; aggregating these gives a validation estimate without a separate holdout.

Q17.19. How do you pick k in kNN?

Answer. Cross-validation; odd k for binary classification avoids ties; small k overfits, large k underfits; rule-of-thumb starting point $\sqrt{n}$.

Q17.20. What does a learning curve tell you?

Answer. Training and validation scores vs training-set size. Large persistent gap: high variance (more data helps). Both low and converged: high bias (more data won't help; need a richer model).

Q17.21. Why is logistic regression called linear?

Answer. Its log-odds (and hence its decision boundary) are linear in the features.

Q17.22. What is multicollinearity and how do you detect it?

Answer. Strong correlation among predictors, inflating coefficient variance. Detect with correlation matrices, VIF, condition number; handle with ridge, dropping/combining variables, PCA.

Q17.23. Hard vs soft voting in ensembles?

Answer. Hard: majority of predicted labels. Soft: average predicted probabilities — usually better when models are calibrated.

Q17.24. Generative vs discriminative example pair?

Answer. Naive Bayes vs logistic regression (they form a classic pair: NB converges faster with little data; LR wins asymptotically).

Q17.25. How would you explain gradient boosting to a business stakeholder?

Answer. A sequence of small decision trees where each new tree focuses on the cases the previous ones got wrong; together they become very accurate.

Q17.26. How do you deal with outliers?

Answer. First ask whether they are errors or genuine rare events (fraud is an "outlier" you want). Options: cap/winsorise, log transform, robust losses (Huber, MAE), robust scalers, tree models, or separate modelling.

Q17.27. Which algorithms are sensitive to class imbalance and how do you adjust each?

Answer. Most are; use class weights (LR, SVM, trees, boosting via `scale_pos_weight`), resampling in the training fold, threshold tuning and appropriate metrics.

Q17.28. What is the difference between a parameter and a hyperparameter?

Answer. Parameters are learned from data (weights); hyperparameters are set before training and tuned on validation data (learning rate, depth, λ).

Q17.29. Grid search vs random search vs Bayesian optimisation?

Answer. Grid: exhaustive, wasteful in high dimensions. Random: more efficient because only a few hyperparameters usually matter. Bayesian (e.g. Optuna's TPE): models the score surface to choose promising settings; most sample-efficient.

17.3 Deep learning

Q17.30. Why do we need activation functions?

Answer. Without non-linearity, any stack of linear layers collapses into one linear map.

Q17.31. Sigmoid vs softmax output?

Answer. Sigmoid: independent probabilities per output (binary or multi-label). Softmax: a distribution across mutually exclusive classes summing to 1.

Q17.32. What is an epoch, a batch, an iteration?

Answer. Epoch: one pass through the training set. Batch: the samples used for one gradient step. Iterations per epoch $= \lceil n/B \rceil$.

Q17.33. 1000 samples, batch size 32: how many iterations per epoch?

Answer. $\lceil 1000/32 \rceil = 32$ (the last batch has 8 samples unless dropped).

Q17.34. What happens if the learning rate is too high? Too low?

Answer. Too high: loss oscillates or diverges (NaN). Too low: slow progress, may stall in poor regions.

Q17.35. What is the vanishing gradient problem and three fixes?

Answer. Gradients shrink exponentially through layers so early layers stop learning. Fixes: ReLU activations, residual connections, normalisation, proper initialisation, LSTM gates.

Q17.36. Why is BatchNorm problematic with batch size 2?

Answer. Batch statistics become very noisy, harming training; use LayerNorm or GroupNorm.

Q17.37. Transfer learning: freeze or fine-tune?

Answer. Freeze with little, similar data; fine-tune more layers with more data or a different domain; use smaller learning rates for pre-trained layers.

Q17.38. What is data augmentation, and when is it harmful?

Answer. Label-preserving transformations to enlarge training data. Harmful when the transformation changes the label (flipping a "6" into a "9", horizontally flipping text or road signs).

Q17.39. Parameters in a 3×3 conv, 64 input channels, 128 filters?

Answer. $3 \cdot 3 \cdot 64 \cdot 128 + 128 = 73{,}856$.

Q17.40. What is a 1×1 convolution used for?

Answer. Mixing channels and changing channel depth cheaply (bottlenecks), adding non-linearity.

Q17.41. Why does self-attention scale quadratically?

Answer. It computes a score for every pair of positions: n^2 entries.

Q17.42. What is the [CLS] token?

Answer. A special token prepended in BERT whose final hidden state is used as a summary representation for classification.

Q17.43. Encoder–decoder attention: where do Q, K and V come from?

Answer. Queries from the decoder; keys and values from the encoder output.

Q17.44. What is the context window of an LLM?

Answer. The maximum number of tokens (prompt plus generated output) the model can attend to at once.

Q17.45. Why do LLMs hallucinate?

Answer. They are trained to produce probable continuations, not verified facts; gaps in knowledge or ambiguous prompts still yield fluent text. Mitigate with retrieval, citations, verification, and allowing "I don't know".

Q17.46. Fine-tuning vs RAG for a company FAQ bot?

Answer. RAG: answers change, must be up-to-date and cite sources. Fine-tuning: teach tone/format or domain-specific behaviour. Often both.

Q17.47. What is knowledge distillation?

Answer. Training a small student to match a large teacher's softened output probabilities (temperature > 1), transferring "dark knowledge" about class similarities.

17.4 Tricky MCQ-style items

Q17.48. Increasing the regularisation parameter λ in ridge regression will: (a) increase variance (b) increase bias (c) increase training fit.

Answer. (b) increase bias (and reduce variance; training fit worsens).

Q17.49. Which is NOT affected by feature scaling: (a) kNN (b) SVM with RBF kernel (c) random forest (d) PCA?

Answer. (c).

Q17.50. A model with AUC 0.3 is:

Answer. Worse than random — but informative: inverting its scores gives AUC 0.7. Often a sign of a label flip bug.

Q17.51. Adding more training data helps most when the model has: high bias or high variance?

Answer. High variance.

Q17.52. k-means with $k = n$ gives inertia:

Answer. Zero (each point is its own centre).

Q17.53. Which loss is most robust to outliers in regression: MSE, MAE, or Huber?

Answer. MAE (Huber is a compromise; MSE is least robust).

Q17.54. Dropout with $p = 0.5$ in a layer of 100 units: expected active units during training?

Answer. 50; survivors are scaled by 2 (inverted dropout).

Q17.55. Output size of 7×7 input, 3×3 kernel, stride 2, padding 0?

Answer. $\lfloor (7-3)/2 \rfloor + 1 = 3$.

Q17.56. Max pooling 2×2 stride 2 on $32 \times 32 \times 16$?

Answer. $16 \times 16 \times 16$.

Q17.57. Softmax of $[2, 2, 2]$?

Answer. $[1/3, 1/3, 1/3]$; softmax is invariant to adding a constant.

Q17.58. If precision $= 0.8$ and recall $= 0.5$, F1?

Answer. $2(0.4)/1.3 \approx 0.615$.

Q17.59. Which of these is a lazy learner: decision tree, kNN, Naive Bayes?

Answer. kNN.

Q17.60. Correlation between X and $Y = -3X + 5$?

Answer. -1.

Q17.61. For an unbiased die, P(at least one 6 in 4 rolls)?

Answer. $1 - (5/6)^4 \approx 0.518$.

17.5 Case and open-ended questions

Q17.62. How would you build a model to predict which loan applicants will default?

Answer. Clarify the definition of default (e.g. 90+ days past due within 12 months) and the decision (approve/reject/price). Data: application data, bureau data, bank transactions; beware that labels exist only for approved loans (reject inference). Features: income stability, debt-to-income, utilisation, past delinquencies, tenure. Model: logistic regression scorecard (explainable, regulator-friendly) benchmarked against GBDT; evaluate AUC/Gini, KS, calibration, stability (PSI) across time; set cut-offs from expected loss = PD $\times$ LGD $\times$ EAD against margin; fairness checks; monitor drift.

Q17.63. Sales dropped 15% last week. How do you investigate?

Answer. Check data first (tracking/pipeline issues). Then decompose: which regions, channels, products, customer segments? Traffic vs conversion vs average order value? Internal changes (price, release, outage, campaign end) vs external (holidays, competitor promotion, weather, seasonality — compare to the same week last year). Form hypotheses, test with data, quantify each driver.

Q17.64. How would you measure the impact of a feature you cannot A/B test?

Answer. Quasi-experimental methods: difference-in-differences with a comparable control group, synthetic control, regression discontinuity (if a threshold determines exposure), interrupted time series, propensity-score matching — and state their assumptions (e.g. parallel trends).

Q17.65. What metric would you use for a churn model used by a retention team that can call 500 customers a week?

Answer. Precision@500 (or lift in the top decile) and, ideally, expected saved revenue; AUC is secondary because only the top of the ranking is acted on.

17.6 Talking about your own projects

Interviewers ask about projects on your CV more than any textbook topic. A structure that works:

1. **Problem and why it mattered** (one or two sentences, business terms).
2. **Data**: source, size, granularity, issues you had to fix.
3. **Approach**: baseline first, then the model, and *why* you chose it over alternatives.
4. **Validation**: how you split (time-based for forecasting), which metric and why.
5. **Result**: a number against the baseline, stated honestly.
6. **What you would do differently / next.**

Real-world example. For a working-capital forecasting project like your Sheela Foam internship, expect follow-ups such as: Why a time-series model rather than regression on drivers? How did you check stationarity? What was your naive baseline and did you beat it (MASE)? How did you validate — walk-forward? Since you used a public proxy dataset for the college deliverable, be ready to explain how well the proxy represents the real business and what would change with the actual data. Owning such limitations openly tends to read as maturity, not weakness.

Q17.66. What if you don't know the answer in an interview?

Answer. Say so plainly, then reason aloud from first principles you do know; interviewers often value the reasoning more than recall. Do not bluff — follow-up questions expose it quickly.

Appendix A

Formula Cheat Sheet

Probability and statistics

Bayes	$P(A \mid B) = P(B \mid A)P(A)/P(B)$
Variance	$\operatorname{Var} X = \mathbb{E}X^2 - (\mathbb{E}X)^2$; $\operatorname{Var}(aX + bY) = a^2 \operatorname{Var} X + b^2 \operatorname{Var} Y + 2ab \operatorname{Cov}(X, Y)$
Standard error	$\sigma/\sqrt{n}$
95% CI for a mean	$\bar{x} \pm 1.96\, s/\sqrt{n}$ (use t_{n-1} for small n)
Two-sample t	$(\bar{x}_1 - \bar{x}_2)/\sqrt{s_1^2/n_1 + s_2^2/n_2}$
A/B sample size per arm	$n \approx (z_{1-\alpha/2} + z_{1-\beta})^2[p_1(1-p_1) + p_2(1-p_2)]/\delta^2$; rule of thumb $16\sigma^2/\delta^2$
Entropy, cross-entropy	$H(p) = -\sum p \log p$; $H(p, q) = -\sum p \log q$
KL divergence	$D_{\mathrm{KL}}(p\|q) = \sum p \log(p/q) = H(p, q) - H(p) \geq 0$
Chebyshev	$P(\|X - \mu\| \geq k\sigma) \leq 1/k^2$

Linear models

OLS	$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$; simple slope $= \operatorname{Cov}(x, y)/\operatorname{Var}(x)$
Ridge	$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$
Lasso coordinate update	$S(\rho, \lambda) = \operatorname{sign}(\rho) \max(\|\rho\| - \lambda, 0)$
R^2, adjusted	$1 - SS_{res}/SS_{tot}$; $1 - (1 - R^2)\frac{n-1}{n-d-1}$
VIF	$1/(1 - R_j^2)$
Logistic	$p = \sigma(\mathbf{w}^\top \mathbf{x})$; gradient $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})$; Hessian $\mathbf{X}^\top \operatorname{diag}(p(1-p))\mathbf{X}$
Odds ratio	e^{w_j} per unit increase in x_j

Metrics

Precision, recall	$TP/(TP+FP)$, $TP/(TP+FN)$
F_1, F_β	$2PR/(P+R)$; $(1+\beta^2)PR/(\beta^2 P+R)$
Specificity, FPR	$TN/(TN+FP)$; $1-$specificity
AUC	$P(\text{score}_+ > \text{score}_-)$; $\text{Gini} = 2\,\text{AUC} - 1$
Log-loss	$-\frac{1}{n}\sum[y\log p + (1-y)\log(1-p)]$
RMSE, MAE, MAPE	$\sqrt{\frac{1}{n}\sum e^2}$, $\frac{1}{n}\sum\lvert e\rvert$, $\frac{1}{n}\sum\lvert e/y\rvert$
NDCG	DCG/IDCG, $\text{DCG} = \sum_i (2^{rel_i}-1)/\log_2(i+1)$
PSI	$\sum(a_i - e_i)\ln(a_i/e_i)$
Silhouette	$(b-a)/\max(a,b)$
Lift (association rules)	$P(B \mid A)/P(B)$

Trees and ensembles

Gini, entropy	$1-\sum p_k^2$; $-\sum p_k \log_2 p_k$
Bagging unique fraction	$1-(1-1/n)^n \to 63.2\%$
Ensemble variance	$\rho\sigma^2 + (1-\rho)\sigma^2/B$
AdaBoost weight	$\alpha = \frac{1}{2}\log\frac{1-\epsilon}{\epsilon}$
XGBoost leaf, gain	$w^* = -G/(H+\lambda)$; $\frac{1}{2}[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}] - \gamma$

Deep learning

Sigmoid derivative	$\sigma(1-\sigma)$, max 0.25
Softmax + CE gradient	$\hat{\mathbf{y}} - \mathbf{y}$
Dense layer params	$n_{in}n_{out} + n_{out}$
Conv output size	$\lfloor (W-K+2P)/S \rfloor + 1$
Conv params	$K^2 C_{in} C_{out} + C_{out}$
Depthwise separable params	$K^2 C_{in} + C_{in} C_{out}$
LSTM / GRU params	$4(h(h+x)+h)$ / $3(h(h+x)+h)$ (one bias set)
Xavier, He variance	$2/(n_{in}+n_{out})$; $2/n_{in}$
Adam	$\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$, step $\eta\hat{m}/(\sqrt{\hat{v}}+\epsilon)$
Attention	$\text{softmax}(\mathbf{QK}^\top/\sqrt{d_k})\mathbf{V}$
Transformer block params	$\approx 12d^2$
Training FLOPs	$\approx 6ND$
KV cache per token	$2 \times L \times d\times$ bytes (MHA)
LoRA trainable params	$r(d+k)$ per adapted matrix
Perplexity	exp(average negative log-likelihood per token)
VAE KL (Gaussian)	$\frac{1}{2}\sum(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$
Diffusion forward	$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$
Optimal GAN discriminator	$p_{\text{data}}/(p_{\text{data}} + p_g)$

Final advice

- In OAs, MCQs on shapes, parameter counts and metric calculations are free marks: practise until they are mechanical.
- In interviews, say the trade-off, not just the definition: "X is better when…, worse when…".
- Always start design questions with the business objective and a baseline.

- When unsure, reason from first principles aloud, and say what you would check.